MachinaState

Visual UML State Machine Designer for Visual Studio Code

User Manual

Draw, simulate and generate hierarchical state machines
in C++, C# and C — switch-based or table-based.

Version 1.2.0 • November 9, 2026

Contents

I Getting Started 1

1 Introduction 2
- 1.1 What is MachinaState? 2
- 1.2 Feature overview 2
- 1.3 How it works 3
- 1.4 How to read this manual 3

2 Installation 5
- 2.1 Requirements 5
- 2.2 Installing from a VSIX package 5
- 2.3 Building from source 5
- 2.4 Verifying the installation 5

3 A Tour of the Editor 6
- 3.1 The toolbar 7
- 3.2 Diagram notation 7

II Tutorials 8

4 Tutorial 1: A Traffic Light 9
- 4.1 Step 1 — Create the diagram 9
- 4.2 Step 2 — Add the other states 10
- 4.3 Step 3 — Connect the states 10
- 4.4 Step 4 — Add entry and exit actions 11
- 4.5 Step 5 — Simulate 12
- 4.6 Step 6 — Generate the code 13
- 4.7 Step 7 — Use the generated class 14
- 4.8 What you have learned 14

5 Tutorial 2: Hierarchy, History and Internal Transitions 15
- 5.1 Step 1 — Create the composite state 15
- 5.2 Step 2 — Power and fault handling 15
- 5.3 Step 3 — Resume with history 16
- 5.4 Step 4 — An internal transition 16

5.5 Step 5 — Simulate the fault scenario 17
5.6 Step 6 — Generate table-based code with tracing 18
5.7 Step 7 — Run it 19

6 Tutorial 3: A Thread-Safe Door Lock in C# 22
6.1 Step 1 — States and transitions 22
6.2 Step 2 — Understand the priority of guarded transitions 23
6.3 Step 3 — Event parameters 23
6.4 Step 4 — Simulate 24
6.5 Step 5 — Generate C# with tracing and thread safety 25
6.6 Step 6 — Implement and run 26

7 Tutorial 4: A Vending Machine with Choices, Timeouts and a Queue 29
7.1 Step 1 — Boot with a completion transition 29
7.2 Step 2 — Collect coins through a choice point 30
7.3 Step 3 — Give up after 30 seconds 30
7.4 Step 4 — Dispense, then return change 31
7.5 Step 5 — Simulate 31
7.6 Step 6 — Generate with trace and queue 33
7.7 Step 7 — Implement and run 34

8 Tutorial 5: A Washing Machine with Orthogonal Regions 38
8.1 Step 1 — The outer states 38
8.2 Step 2 — Divide Running into two regions 39
8.3 Step 3 — Fill the regions 39
8.4 Step 4 — Leaving the orthogonal state 39
8.5 Step 5 — Simulate 40
8.6 Step 6 — Record and replay scenarios 41
8.7 Step 7 — Generate 43

III Reference 44

9 Editor Reference 45
9.1 Toolbar 45
9.2 Mouse and keyboard 45
9.3 Properties panel 46
9.4 Working with several states 47
9.5 Colours 48
9.6 Problems list 48
9.7 Exporting images 49

9.8 Automatic layout 49
9.9 Commands 50
9.10 Settings 50
9.11 Getting help 50
9.12 Working with the JSON source 51

10 Modelling Reference **52**
10.1 States 52
10.2 Composite states and regions 52
10.3 Orthogonal regions 52
10.4 Initial and final states 53
10.5 Transitions 53
10.6 Internal transitions 54
10.7 History 54
10.8 Completion transitions 55
10.9 Timeouts 55
10.10 Choice points 55
10.11 Event parameters 56
10.12 Supported and unsupported UML features 57

11 Execution Semantics **58**
11.1 Starting 58
11.2 Dispatching an event 58
11.3 Firing a transition 59
11.4 Orthogonal regions 59
11.5 Worked example 60
11.6 Stateflow semantics 60
11.7 Rules for hooks 61

12 Simulator **62**
12.1 Scenarios 63

13 Code Generation **64**
13.1 Generating 64
13.2 Generated interface 64
13.3 Switch-based versus table-based 65
13.4 C++ specifics 66
13.5 C specifics 66
13.6 C# specifics 68
13.7 Tracing 68

13.8 Thread safety 68
13.9 Event queue 69
13.10 Timers 69
13.11 Hook stubs 70
13.12 Integration patterns 71
13.12.1 Unit testing 71

14 Import and Export **72**
14.1 PlantUML 72
14.2 Mermaid 73
14.3 SCXML 74
14.4 Stateflow (Simulink models) 75
14.4.1 Example: a gear shift chart 76
14.5 Round trips 77

15 Validation Rules **78**

16 File Format **81**

17 Troubleshooting and FAQ **83**

18 Development and Testing **85**

A Generated Code: Flat Traffic Light, C++ Switch-Based **86**

B Generated Code: Flat Traffic Light, C++ Table-Based **91**

C Generated Code: Flat Traffic Light, C# Switch-Based **96**

D Generated Code: Door Lock, C# Table-Based **100**

E Generated Code: Vending Machine, C++ Switch-Based with Trace and Queue **107**

F Generated Code: Flat Traffic Light, C Header (Switch-Based with Trace) **118**

Part I

Getting Started

Chapter 1

Introduction

1.1 What is MachinaState?

MachinaState is a Visual Studio Code extension for designing *state machines* visually and turning them into production code. You draw a UML state machine diagram on a canvas, check its behaviour in a built-in simulator, and generate a C++ or C# class, or a C module, that implements it. Your own logic — what happens when a light switches on or a door unlocks — lives in a hand-written subclass (in C, in functions you register), so the diagram can be changed and regenerated at any time without touching your code.

A state machine describes a system that is always in one of a finite number of *states* (with orthogonal regions, in one state per region) and moves between them in response to *events*. Typical examples are protocol handlers, user interface flows, device controllers, game characters and workflow engines. Writing such logic by hand quickly leads to deeply nested `if` statements and flags; a diagram makes the behaviour explicit, reviewable and testable.

1.2 Feature overview

- **Visual editor**:
  - states, composite (nested) states, initial and final states
  - orthogonal (parallel) regions
  - history pseudostates and choice points
  - transitions with events, guards and actions; internal transitions, completion transitions and `after(...)` timeouts
  - colours
- **Productive editing**:
  - multi-selection, align and distribute
  - copy, cut, paste and duplicate (also between diagrams)
  - arrow-key nudging and snap to grid
  - automatic layout with the Eclipse Layout Kernel
- **Continuous validation** in the editor and in VS Code's **Problems** panel.
- **Interactive simulator** running the same reference engine that the test suite checks against the generated code:
  - highlights the active states and records a trace of every exit, action and entry
  - guard results as check boxes, event arguments and running timers
  - scenarios that are recorded, saved in the diagram and replayed
  - transition coverage
- **Code generation**:
  - C++ (C++17, single header, no heap allocation), C# (C# 8+) and C (C99, a header and a source file, no heap allocation)

  - each as a *switch-based* or *table-based* implementation; all six variants behave identically and are verified against each other by an automated test suite
  - optional regeneration on every save

- **Hierarchical semantics**:
  - event bubbling from inner to outer states
  - exit/entry ordering via the least common ancestor
  - default entry and shallow/deep history
  - run-to-completion processing including completion transitions
- **Event parameters** passed to guards and actions.
- **Options**:
  - a tracing hook
  - thread safety through user-implemented `Lock`/`Unlock` functions
  - an **event queue** so that actions can raise follow-up events
  - **timer hooks** for timeouts
- **PlantUML, Mermaid and SCXML** import and export; **Copy as Mermaid** for Markdown documentation.
- **PNG and SVG export** and a diff-friendly JSON file format.

1.3 How it works

Figure 1.1 shows the workflow. The diagram is stored in a `.machina` file. The generator turns it into an abstract class that contains the complete state handling. You derive from that class and implement the *hooks* — guards, actions and optionally tracing and locking.

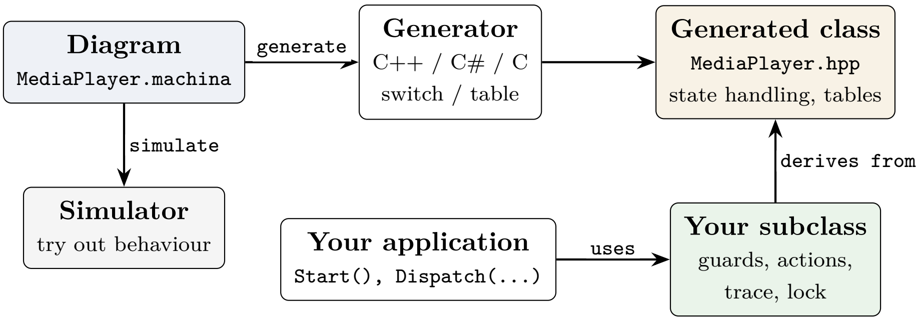


Diagram
MediaPlayer.machina
generate
Generator
C++ / C# / C
switch / table
Generated class
MediaPlayer.hpp
state handling, tables
simulate
Simulator
try out behaviour
derives from
Your application
Start(), Dispatch(...)
uses
Your subclass
guards, actions,
trace, lock

Figure 1.1: From diagram to running code.

1.4 How to read this manual

- **Part I** installs the extension and gives a guided tour of the editor.
- **Part II** contains five hands-on tutorials that build complete, compiling examples:
  - a simple traffic light (C++, switch-based)
  - a hierarchical traffic light with fault handling and history (C++, table-based with tracing)
  - a thread-safe door lock with event parameters (C#, table-based)

  - a vending machine with choice points, completion transitions, timeouts and the event queue (C++, switch-based)
  - a washing machine with orthogonal regions, recorded scenarios and coverage (C#)

- **Part III** is the reference: every editor feature, every modelling element, the exact execution semantics, the generated code, validation rules, the file format and the test suite.

Throughout the manual, **bold sans-serif** text names buttons and fields, `monospace` text is code or file content, and keys are shown as `Ctrl`+`S`. The screenshots use VS Code's *Light Modern* theme; the editor follows whatever colour theme you use.

All code listings in the tutorials and appendices were generated by MachinaState from the example diagrams in the `examples` folder and compiled with `g++ -std=c++17 -Wall -Wextra -Werror`, `gcc -std=c99 -Wall -Wextra -Werror -pedantic` or `dotnet` with warnings treated as errors. The program outputs shown are the real outputs.

Chapter 2

Installation

2.1 Requirements

Component	Requirement
Editor	Visual Studio Code 1.85 or later.
Generated C++ code	A C++17 compiler (GCC 7+, Clang 5+, MSVC 2017 15.7+). Header-only, no dependencies beyond `<cstddef>`.
Generated C# code	C# 8 or later (.NET Core 3.0+, .NET 5+). Uses only `System`.
Generated C code	A C99 compiler. Uses only `<stdbool.h>` and `<stddef.h>`; no heap allocation.

2.2 Installing from a VSIX package

1. Open the **Extensions** view (`Ctrl`+`Shift`+`X`).
2. Click the `...` menu at the top of the view and choose **Install from VSIX...**.
3. Select `machina-state-1.2.0.vsix` and reload VS Code if asked.

Alternatively, install from a terminal:

```
code --install-extension machina-state-1.2.0.vsix
```

2.3 Building from source

```
git clone <repository> MachinaState
cd MachinaState
npm install
npm test                    # compile, unit/golden tests, compile and run generated code
npm run package             # produces machina-state-<version>.vsix
```

To try the extension without installing it, open the folder in VS Code and press `F5`. An *Extension Development Host* window opens with the `examples` folder.

2.4 Verifying the installation

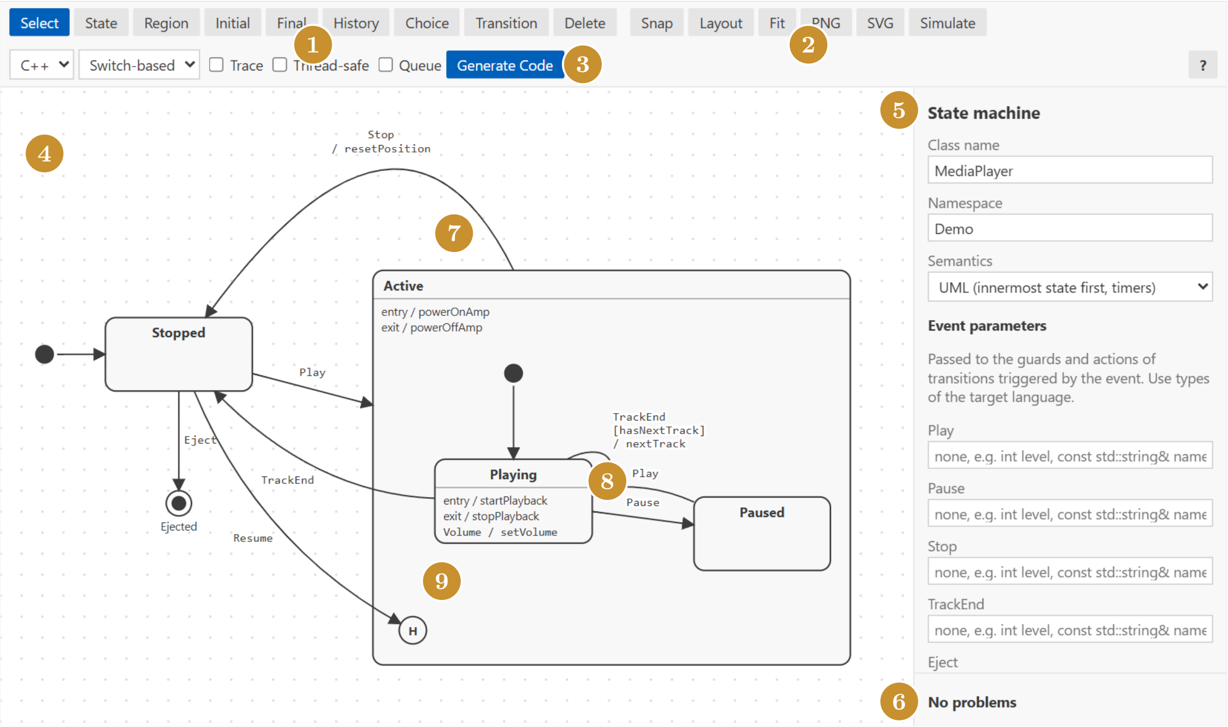
Open the Command Palette (`Ctrl`+`Shift`+`P`) and run **MachinaState: New State Machine Diagram**. After choosing a file name, a diagram with an initial pseudostate and a state named `Idle` opens in the diagram editor. Files ending in `.machina` always open in the diagram editor; use **Reopen Editor With...** → **Text Editor** to see the JSON source.

For a short introduction inside VS Code, run **MachinaState: Get Started**, or press **?** in the diagram editor's toolbar for the shortcuts (Section 9.11).

Chapter 3

A Tour of the Editor

Figure 3.1 shows the editor with the `MediaPlayer` example from the `examples` folder. The numbered areas are explained below.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
1
2
3
4
5
6
7
8
9
Stop
/ resetPosition
Stopped
Play
Eject
Ejected
TrackEnd
Resume
Active
entry / powerOnAmp
exit / powerOffAmp
Playing
entry / startPlayback
exit / stopPlayback
Volume / setVolume
TrackEnd
[hasNextTrack]
/ nextTrack
Play
Pause
Paused
H

C++ | Switch-based | ☐ Trace ☐ Thread-safe ☐ Queue

State machine

Class name: MediaPlayer

Namespace: Demo

Semantics: UML (innermost state first, timers)

Event parameters

Passed to the guards and actions of transitions triggered by the event. Use types of the target language.

Play: none, e.g. int level, const std::string& name

Pause: none, e.g. int level, const std::string& name

Stop: none, e.g. int level, const std::string& name

TrackEnd: none, e.g. int level, const std::string& name

Eject

No problems

Figure 3.1: The diagram editor showing `examples/MediaPlayer.machina`.

① **Drawing tools.** **Select** moves and resizes elements. **State**, **Region**, **Initial**, **Final**, **History** and **Choice** add elements with a single click on the canvas; **Transition** connects two elements by dragging. **Delete** removes the selection.

② **View, export and simulation.** **Snap** toggles the 10-pixel grid, **Layout** arranges the diagram automatically, **Fit** zooms to the whole diagram, **PNG** and **SVG** save an image and **Simulate** starts the simulator. The **?** button at the right end of the toolbar opens the help (Section 9.11).

③ **Code generation options.** Target language (C++, C# or C), implementation style, the optional **Trace**, **Thread-safe** and **Queue** features, and the **Generate Code** button.

④ **Canvas.** Drag empty space to pan, use the mouse wheel to zoom.

⑤ **Properties panel.** Shows the properties of the selected element — or, with nothing selected, the class name, namespace and event parameters.

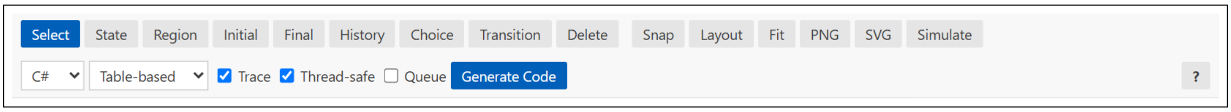
⑥ **Problems list.** All validation problems; click one to select the element concerned. The same problems appear in VS Code's **Problems** panel.

⑦ **Composite state.** `Active` contains other states, its own initial pseudostate and a history pseudostate. Entry and exit actions are listed below the name.

⑧ **Internal transition.** `Volume / setVolume` is handled inside `Playing` without leaving it.

⑨ **History pseudostate.** The `Resume` transition re-enters `Active` in the substate that was active when it was left.

3.1 The toolbar


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?

C# Table-based ☑ Trace ☑ Thread-safe ☐ Queue

Figure 3.2: The toolbar with C#, table-based, **Trace** and **Thread-safe** selected. In a narrow window the toolbar wraps onto two rows.

The language, style and the three check boxes are stored in the diagram, so everyone who opens the file generates the same code. Chapter 9 describes every button in detail.

3.2 Diagram notation

MachinaState uses standard UML notation (Figure 3.3).

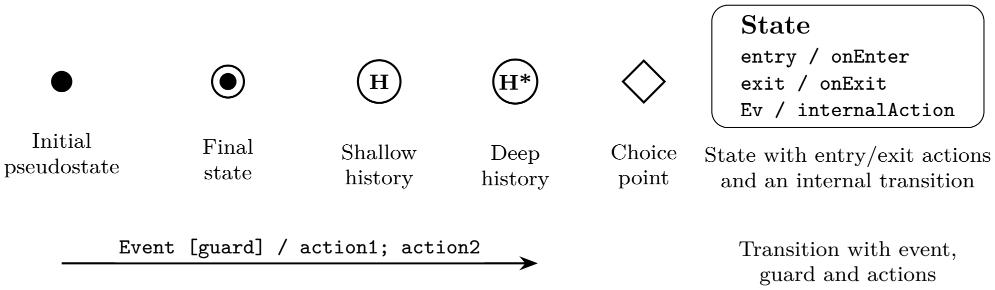

Initial pseudostate
Final state
H
Shallow history
H*
Deep history
Choice point
State
entry / onEnter
exit / onExit
Ev / internalAction
State with entry/exit actions and an internal transition
Event [guard] / action1; action2
Transition with event, guard and actions

Figure 3.3: Elements of the notation.

Part II

Tutorials

Chapter 4

Tutorial 1: A Traffic Light

In this tutorial you build the classic three-colour traffic light, simulate it and run it as a small C++ program. It introduces the editing workflow; later tutorials build on it.

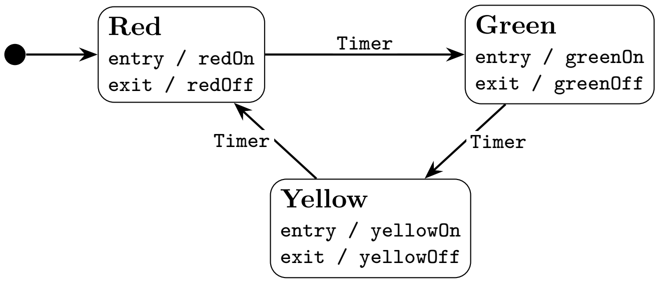
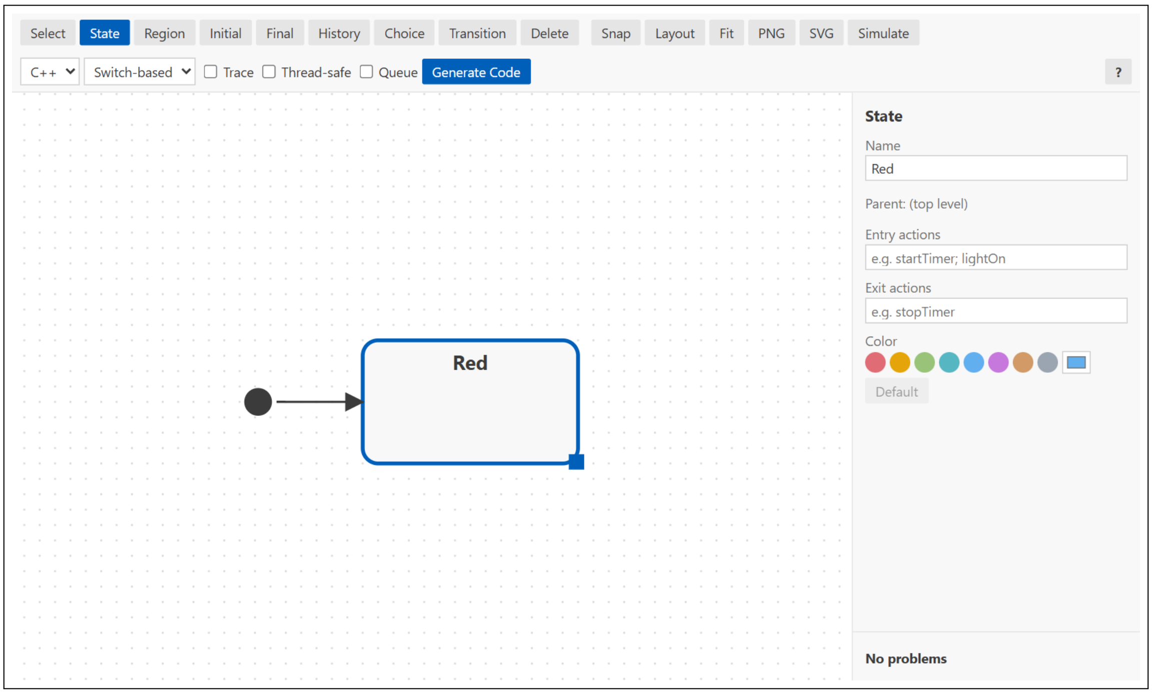

Red
entry / redOn
exit / redOff
Timer
Green
entry / greenOn
exit / greenOff
Timer
Timer
Yellow
entry / yellowOn
exit / yellowOff

Figure 4.1: Goal of Tutorial 1.

4.1 Step 1 — Create the diagram

1. Run **MachinaState: New State Machine Diagram** and save it as `TrafficLight.machina`.
2. The new diagram contains an initial pseudostate connected to a state named `Idle`. Double-click `Idle` and rename it to `Red`; press `Enter` to apply.
3. With nothing selected (click on empty canvas), set **Class name** to `TrafficLight` and **Namespace** to `Demo`.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Red
Default

C++ · Switch-based · ☐ Trace ☐ Thread-safe ☐ Queue

State

Name: Red

Parent: (top level)

Entry actions: e.g. startTimer; lightOn

Exit actions: e.g. stopTimer

Color

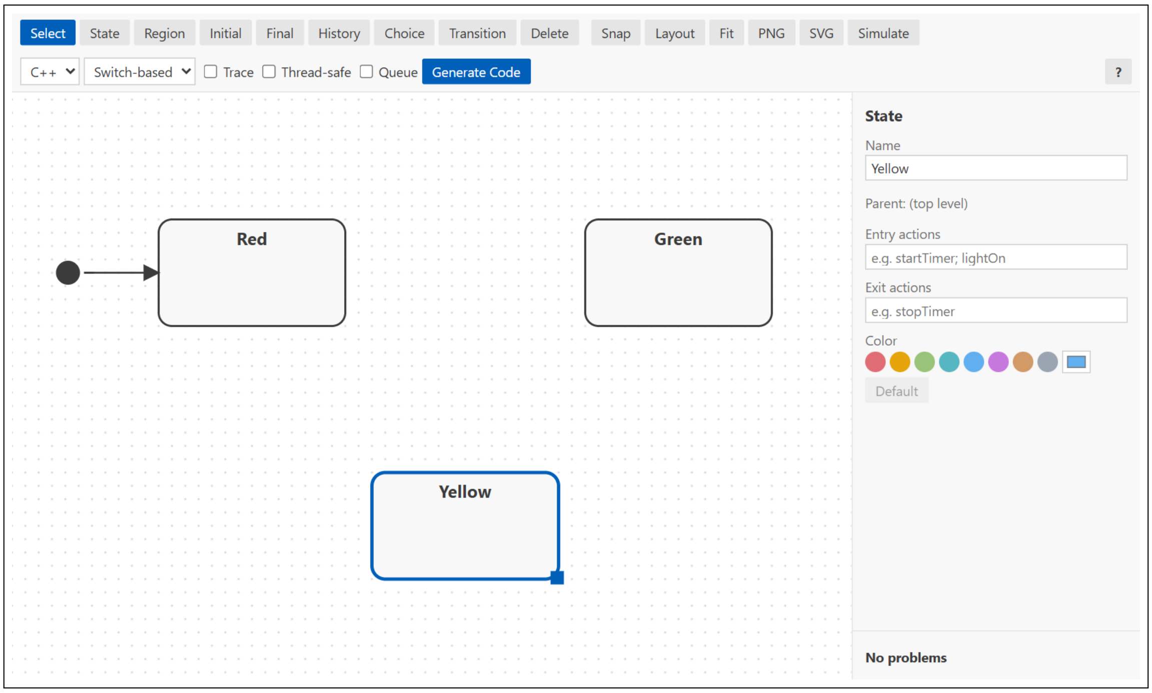
No problems

Figure 4.2: Step 1: the initial pseudostate points at `Red`. The **State** tool is active, ready to place the next state. The square at the bottom-right corner of the selected state resizes it.

4.2 Step 2 — Add the other states

1. Click **State** in the toolbar and click on the canvas to the right of `Red`. A state named `State1` appears and is selected.
2. Type `Green` in the **Name** field and press `Enter`.
3. Repeat for `Yellow`, placing it below and between the two.

After adding an element, the editor switches back to the **Select** tool so that you can immediately move or edit it. Drag states by their body to arrange them.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
C++
Switch-based
☐ Trace
☐ Thread-safe
☐ Queue
Generate Code
?
Red
Green
Yellow

State

Name: Yellow

Parent: (top level)

Entry actions: e.g. startTimer; lightOn

Exit actions: e.g. stopTimer

Color

Default

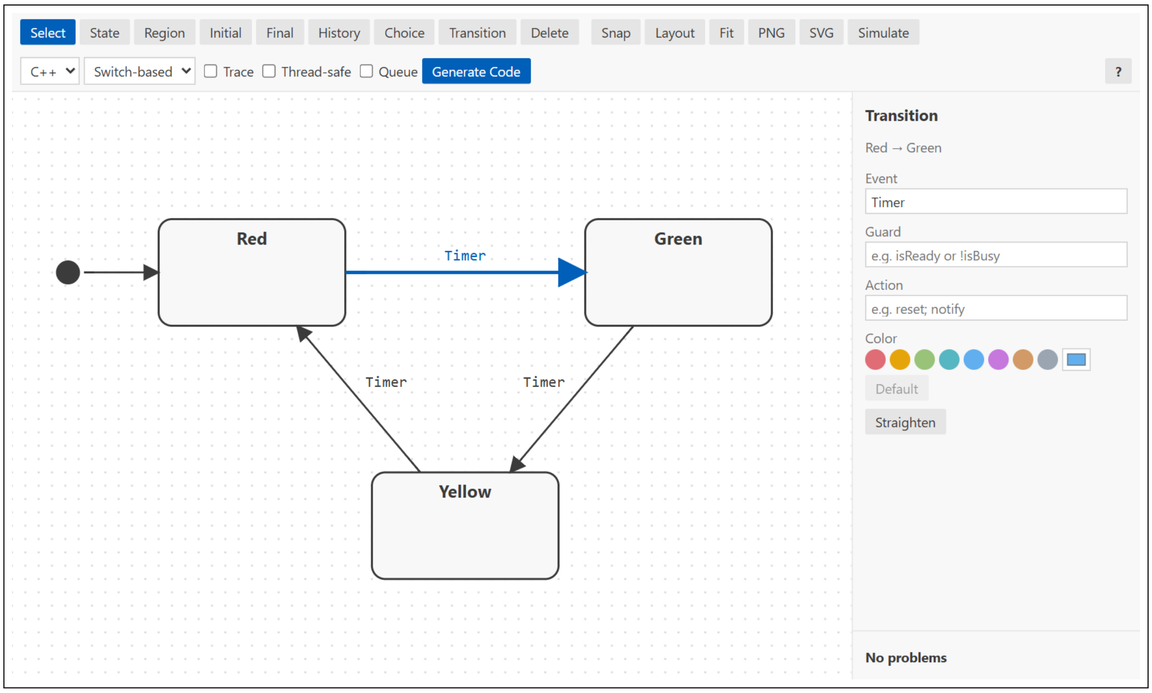
No problems

Figure 4.3: Step 2: three states, `Yellow` selected.

> **Tip**
>
> State names become enumeration values in the generated code, so they must be valid identifiers and unique within the whole diagram. The problems list tells you immediately if a name is not allowed.

4.3 Step 3 — Connect the states

1. Click **Transition**. Press the mouse button on `Red`, drag to `Green` and release.
2. The new transition is selected and has the placeholder event `Event`. Change **Event** to `Timer` and press `Enter`.
3. Create `Green` → `Yellow` and `Yellow` → `Red` the same way, both with the event `Timer`.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Red
Green
Yellow
Timer
Timer
Timer

C++ Switch-based ☐ Trace ☐ Thread-safe ☐ Queue

Transition

Red → Green

Event: Timer

Guard: e.g. isReady or !isBusy

Action: e.g. reset; notify

Color

Default

Straighten

No problems

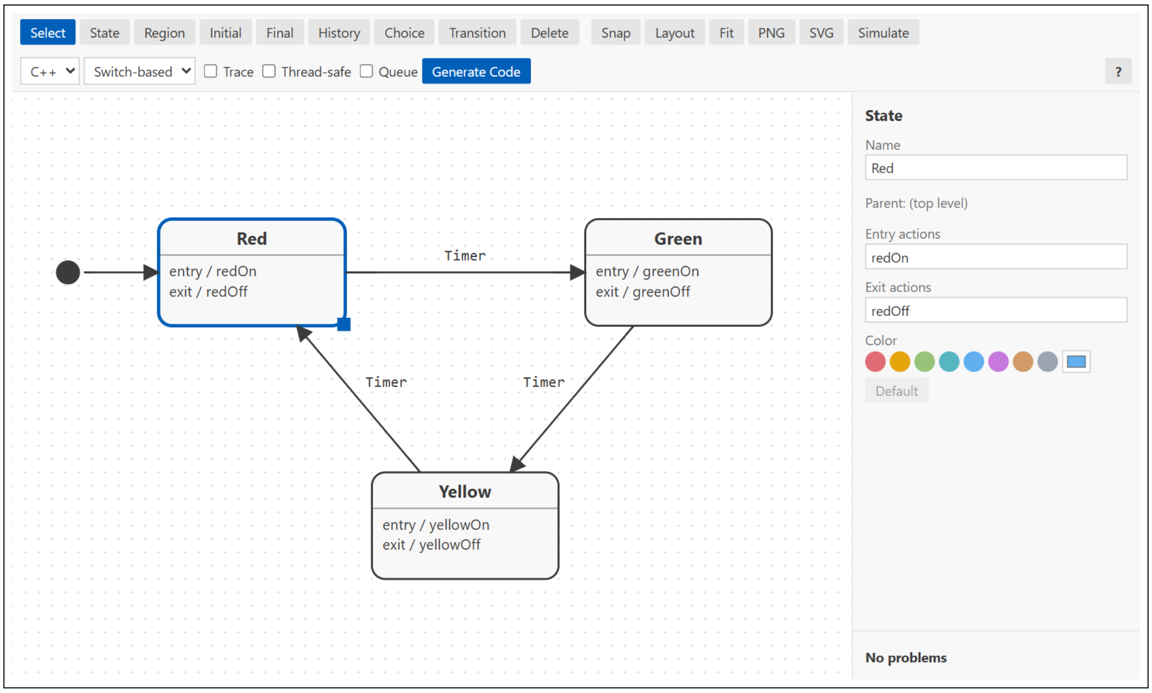
Figure 4.4: Step 3: the `Red` → `Green` transition is selected and shows its properties.

Tip

Drag a transition's label to curve it — useful when two transitions connect the same states in opposite directions. Drag the label back close to the straight line, or press **Straighten**, to make it straight again.

4.4 Step 4 — Add entry and exit actions

A light should switch on when its state is entered and off when it is left. Select `Red` and enter `redOn` in **Entry actions** and `redOff` in **Exit actions**. Do the same for `Green` (`greenOn`, `greenOff`) and `Yellow` (`yellowOn`, `yellowOff`).


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Red
entry / redOn
exit / redOff
Timer
Green
entry / greenOn
exit / greenOff
Timer
Timer
Yellow
entry / yellowOn
exit / yellowOff
Default

C++ Switch-based ☐ Trace ☐ Thread-safe ☐ Queue

State

Name: Red

Parent: (top level)

Entry actions: redOn

Exit actions: redOff

Color

No problems

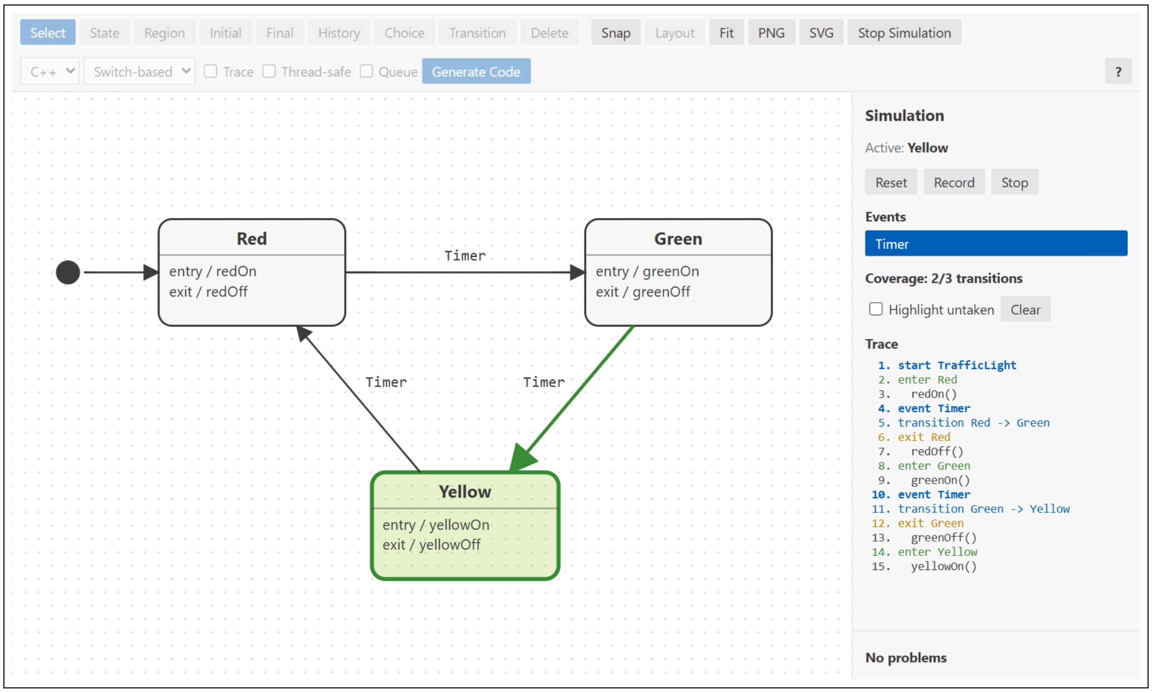
Figure 4.5: Step 4: `Red` with entry and exit actions.

Actions are *names* of methods. The generator declares each name as an abstract method in the generated class, and you implement it later. To run several actions, separate them with semicolons: `redOn; startTimer`.

4.5 Step 5 — Simulate

Before writing any code, check that the machine behaves as intended.

1. Click **Simulate**. Editing is disabled, the machine is started, and `Red` is highlighted.
2. The **Events** list has a button for every event. Click **Timer** twice.
3. The **Trace** shows each step: the event, the exit of the old state with its exit action, and the entry of the new state with its entry action.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Stop Simulation
C++
Switch-based
☐ Trace
☐ Thread-safe
☐ Queue
Generate Code
?
Red
entry / redOn
exit / redOff
Timer
Green
entry / greenOn
exit / greenOff
Timer
Timer
Yellow
entry / yellowOn
exit / yellowOff
Reset
Record
Stop
Events
Timer
☐ Highlight untaken
Clear

Simulation

Active: **Yellow**

Coverage: 2/3 transitions

Trace

```
 1. start TrafficLight
 2. enter Red
 3.   redOn()
 4. event Timer
 5. transition Red -> Green
 6. exit Red
 7.   redOff()
 8. enter Green
 9.   greenOn()
10. event Timer
11. transition Green -> Yellow
12. exit Green
13.   greenOff()
14. enter Yellow
15.   yellowOn()
```

No problems

Figure 4.6: Step 5: after two `Timer` events the light is `Yellow`. The transition taken in the last step is highlighted.

Click **Reset** to start over or **Stop** to return to editing.

4.6 Step 6 — Generate the code

Select **C++** and **Switch-based** in the toolbar and click **Generate Code**. MachinaState writes `TrafficLight.hpp` next to the diagram and opens it beside the editor. The class looks like this (abridged; Appendix A contains the full file):

```cpp
namespace Demo {
    class TrafficLight {
      public:
        enum class State { Red, Green, Yellow, None };
        enum class Event { Timer };

        void Start();                         // enter the initial state
        bool Dispatch(Event event);           // process one event
        State Current() const;                // innermost active state
        bool IsIn(State state) const;
        bool IsFinished() const;
        static const char* StateName(State state);
        static const char* EventName(Event event);

      protected:
        // Actions
        virtual void redOn() = 0;
        virtual void redOff() = 0;
        virtual void greenOn() = 0;
        // ...
    };
}
```

4.7 Step 7 — Use the generated class

Create `main.cpp`, derive from `TrafficLight` and implement the six actions:

```
// Tutorial 1: drive the generated flat TrafficLight (switch-based C++).
#include <cstdio>

#include "TrafficLightFlat_switch.hpp"

class ConsoleTrafficLight : public Demo::TrafficLight {
  protected:
    void redOn() override     { std::puts("  [RED    on ]"); }
    void redOff() override    { std::puts("  [RED    off]"); }
    void yellowOn() override  { std::puts("  [YELLOW on ]"); }
    void yellowOff() override { std::puts("  [YELLOW off]"); }
    void greenOn() override   { std::puts("  [GREEN  on ]"); }
    void greenOff() override  { std::puts("  [GREEN  off]"); }
};

int main() {
    ConsoleTrafficLight light;
    light.Start();
    std::printf("state: %s\n", Demo::TrafficLight::StateName(light.Current()));

    for (int tick = 1; tick <= 4; ++tick) {
        light.Dispatch(Demo::TrafficLight::Event::Timer);
        std::printf("tick %d -> state: %s\n", tick, Demo::TrafficLight::StateName(light.Current
    ()));
    }
}
```

Compile and run it (the generated header was saved as `TrafficLightFlat_switch.hpp` for this manual):

```
g++ -std=c++17 -Wall -Wextra -o traffic main.cpp && ./traffic
```

Listing 4.1: Output of the Tutorial 1 program.

```
  [RED    on ]
state: Red
  [RED    off]
  [GREEN  on ]
tick 1 -> state: Green
  [GREEN  off]
  [YELLOW on ]
tick 2 -> state: Yellow
  [YELLOW off]
  [RED    on ]
tick 3 -> state: Red
  [RED    off]
  [GREEN  on ]
tick 4 -> state: Green
```

4.8 What you have learned

- Adding states and transitions, naming events and attaching entry and exit actions.
- Checking behaviour in the simulator before writing code.
- Generating a class and implementing its actions in a subclass.

Chapter 5

Tutorial 2: Hierarchy, History and Internal Transitions

Real traffic lights can be switched off and fall back to a flashing mode when a fault is detected. After the repair they should continue with the colour they showed before. This tutorial extends the traffic light with a *composite state*, a *history pseudostate* and an *internal transition*, and generates table-based C++ code with tracing.

5.1 Step 1 — Create the composite state

1. Open `TrafficLight.machina` from Tutorial 1 (or `examples/TrafficLight.machina`, which contains the finished result).
2. Click **State** and add a new state `Operating` in an empty area. Resize it with the corner handle until it is large enough to hold the three colours.
3. Drag `Red`, `Green` and `Yellow` into `Operating`. A state becomes a child of the state under its *centre* when you drop it.

Tip

You can also draw the large state *around* existing states: when you resize a state so that a sibling lies completely inside it, the sibling becomes its child.

Every composite state needs its own initial pseudostate that defines the default substate:

4. Select the old top-level initial pseudostate and move it into `Operating`, next to `Red`. It now belongs to `Operating`'s region.
5. Add a new state `Off` outside `Operating`, then add a new initial pseudostate at the top level and connect it to `Off` with the **Transition** tool.

5.2 Step 2 — Power and fault handling

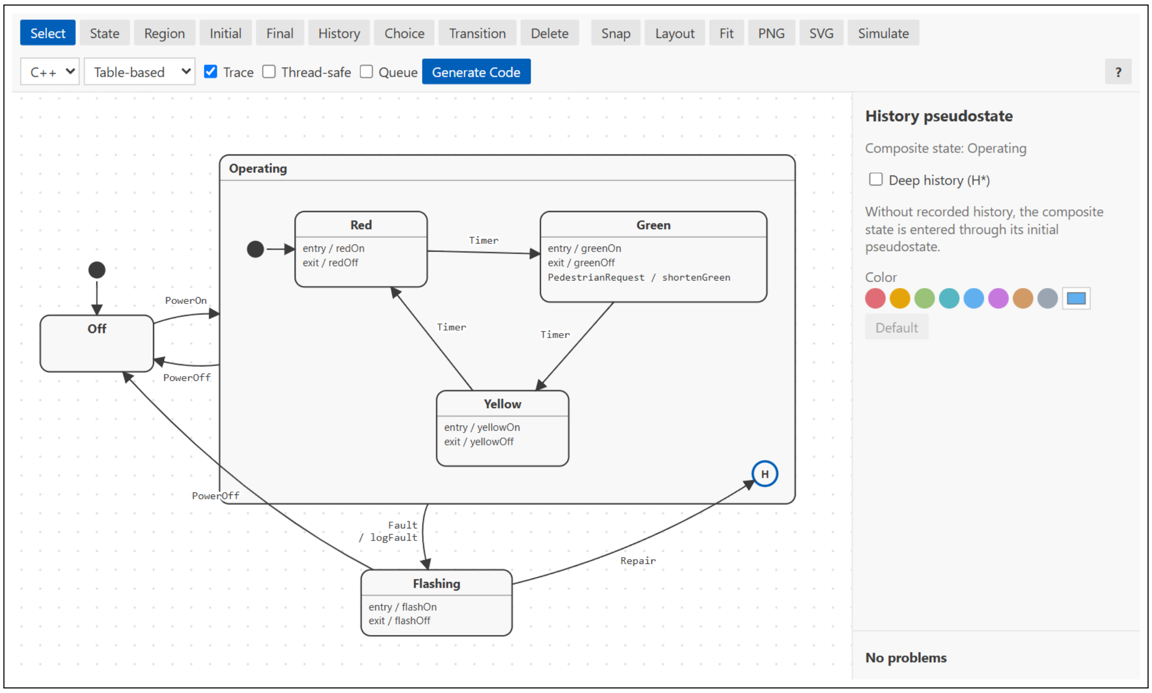
1. Connect `Off` → `Operating` with the event `PowerOn`, and `Operating` → `Off` with `PowerOff`.
2. Add a top-level state `Flashing` with entry action `flashOn` and exit action `flashOff`.
3. Connect `Operating` to `Flashing` with event `Fault` and action `logFault`, and `Flashing` to `Off` with `PowerOff`.

Because `PowerOff` and `Fault` start at the *composite* state `Operating`, they are available no matter which colour is active: an event that the active substate does not handle *bubbles up* to its parent (Section 11.2).

5.3 Step 3 — Resume with history

1. Click **History** and click inside `Operating` to add a history pseudostate (`H`).
2. Connect `Flashing` to `H` with the event `Repair`.

A transition to `H` re-enters `Operating` in the colour that was active when `Operating` was last left. Without history it would always restart at `Red`.



Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Operating
Red
Green
Yellow
Off
Flashing
H
Timer
Timer
Timer
PowerOn
PowerOff
PowerOff
Fault
/ logFault
Repair
Color
Default

C++ Table-based ☑ Trace ☐ Thread-safe ☐ Queue

Red: entry / redOn, exit / redOff

Green: entry / greenOn, exit / greenOff, PedestrianRequest / shortenGreen

Yellow: entry / yellowOn, exit / yellowOff

Flashing: entry / flashOn, exit / flashOff

History pseudostate

Composite state: Operating

☐ Deep history (H*)

Without recorded history, the composite state is entered through its initial pseudostate.

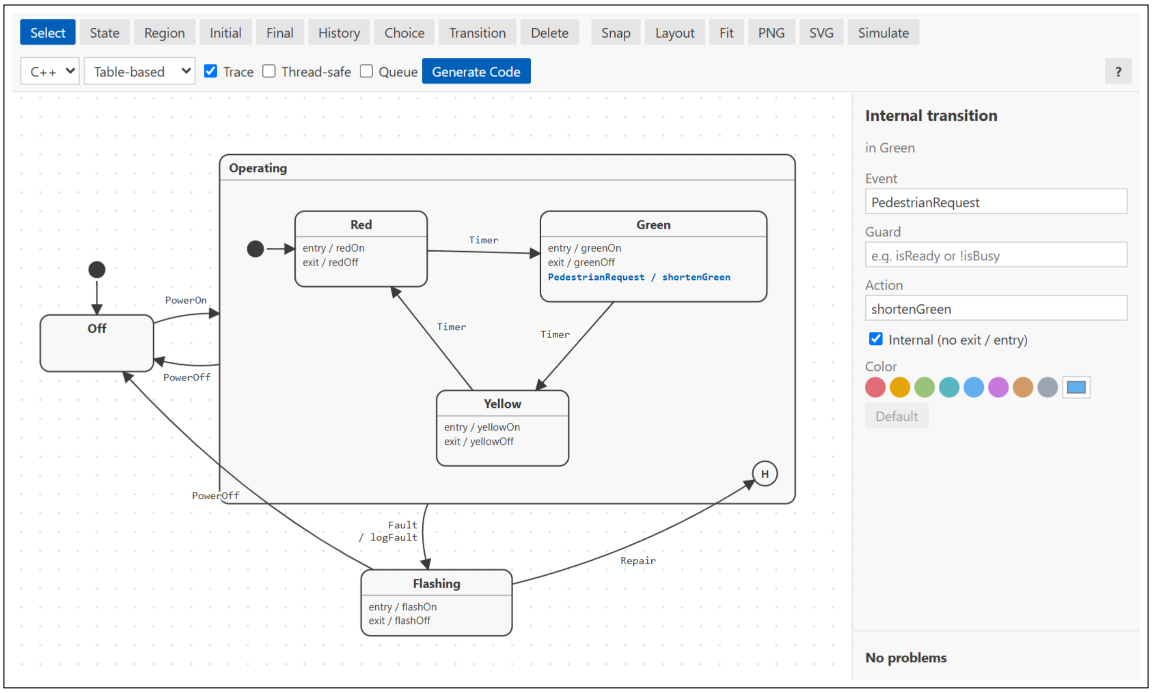
No problems

Figure 5.1: The history pseudostate is selected. **Deep history (H*)** would resume the innermost state instead of the direct substate; both behave the same here because the colours have no substates of their own.

5.4 Step 4 — An internal transition

Pedestrians can request a shorter green phase. The request must not restart the green phase — which an ordinary self transition would do, running `greenOff` and `greenOn`.

1. With the **Transition** tool, drag from `Green` back onto `Green` to create a self transition.
2. Set **Event** to `PedestrianRequest` and **Action** to `shortenGreen`.
3. Tick **Internal (no exit / entry)**. The loop disappears and the transition is listed inside `Green` instead.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Operating
Red
entry / redOn
exit / redOff
Timer
Green
entry / greenOn
exit / greenOff
PedestrianRequest / shortenGreen
Timer
Timer
Yellow
entry / yellowOn
exit / yellowOff
PowerOn
Off
PowerOff
PowerOff
H
Fault
/ logFault
Repair
Flashing
entry / flashOn
exit / flashOff

C++ Table-based ☑ Trace ☐ Thread-safe ☐ Queue

Internal transition

in Green

Event: PedestrianRequest

Guard: e.g. isReady or !isBusy

Action: shortenGreen

☑ Internal (no exit / entry)

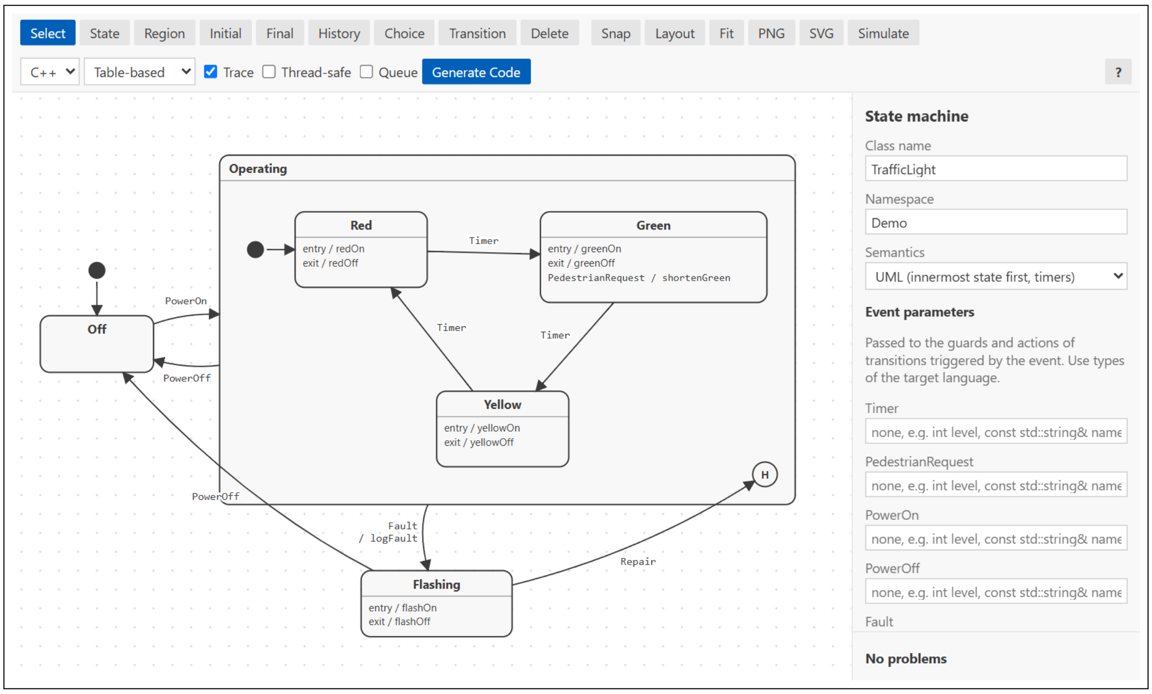
Color

Default

No problems

Figure 5.2: The internal transition is selected by clicking its line inside `Green`.

The finished diagram is shown in Figure 5.3.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Operating
Red
entry / redOn
exit / redOff
Timer
Green
entry / greenOn
exit / greenOff
PedestrianRequest / shortenGreen
Timer
Timer
Yellow
entry / yellowOn
exit / yellowOff
PowerOn
Off
PowerOff
PowerOff
H
Fault
/ logFault
Repair
Flashing
entry / flashOn
exit / flashOff

C++ Table-based ☑ Trace ☐ Thread-safe ☐ Queue

State machine

Class name: TrafficLight

Namespace: Demo

Semantics: UML (innermost state first, timers)

Event parameters

Passed to the guards and actions of transitions triggered by the event. Use types of the target language.

Timer: none, e.g. int level, const std::string& name

PedestrianRequest: none, e.g. int level, const std::string& name

PowerOn: none, e.g. int level, const std::string& name

PowerOff: none, e.g. int level, const std::string& name

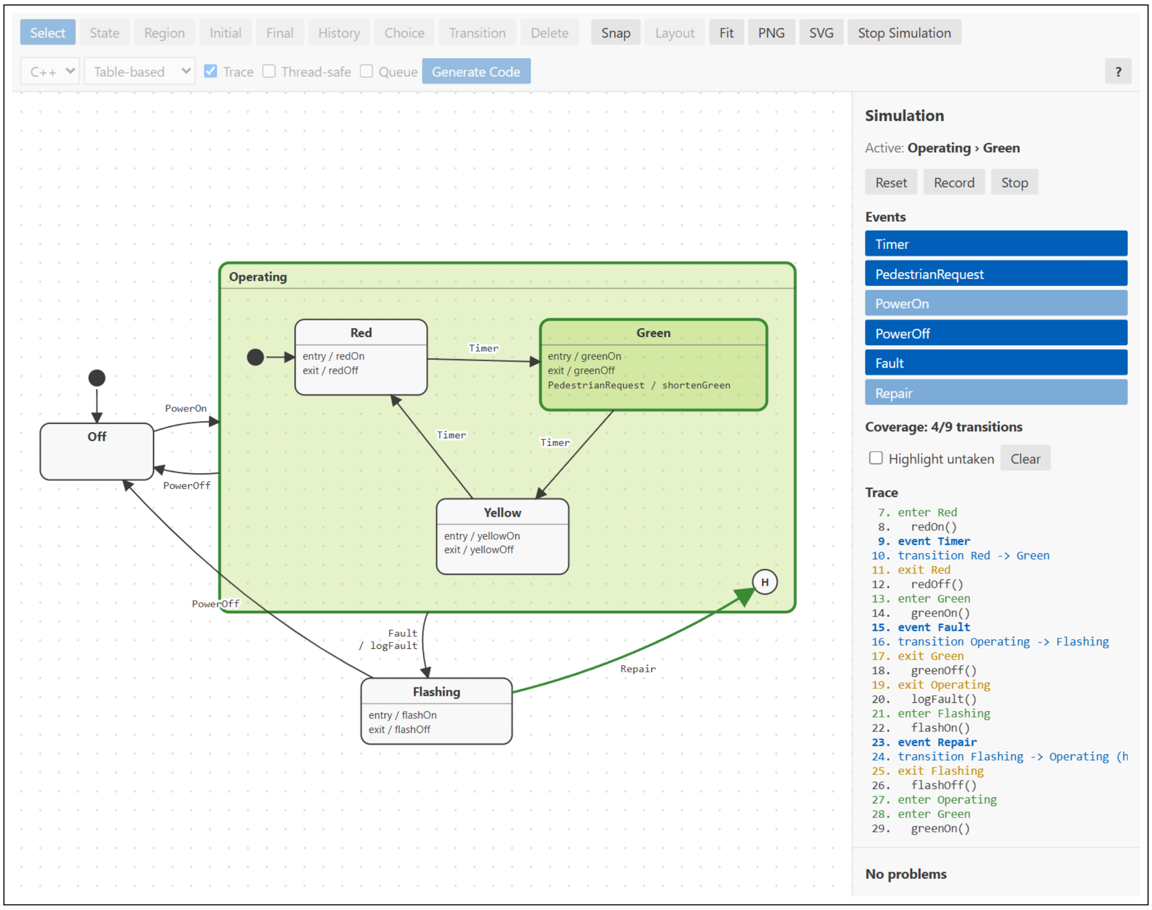
Fault

No problems

Figure 5.3: The hierarchical traffic light (`examples/TrafficLight.machina`).

5.5 Step 5 — Simulate the fault scenario

Start the simulator and fire `PowerOn`, `Timer`, `Fault` and `Repair`. The trace shows that `Fault` exits `Green` *and* `Operating` before entering `Flashing`, and that `Repair` resumes `Green` from history: it is traced as `transition Flashing -> Operating (history)`, followed directly by `enter Green`.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Stop Simulation
Generate Code
?
Operating
Red
entry / redOn
exit / redOff
Green
entry / greenOn
exit / greenOff
PedestrianRequest / shortenGreen
Yellow
entry / yellowOn
exit / yellowOff
Off
Flashing
entry / flashOn
exit / flashOff
H
Timer
Timer
Timer
PowerOn
PowerOff
PowerOff
Fault
/ logFault
Repair
Simulation
Reset
Record
Stop
Events
Timer
PedestrianRequest
PowerOn
PowerOff
Fault
Repair
Highlight untaken
Clear
Trace

C++ | Table-based | ☑ Trace ☐ Thread-safe ☐ Queue

Active: **Operating › Green**

Coverage: 4/9 transitions

☐ Highlight untaken

```
 7. enter Red
 8.   redOn()
 9. event Timer
10. transition Red -> Green
11. exit Red
12.   redOff()
13. enter Green
14.   greenOn()
15. event Fault
16. transition Operating -> Flashing
17. exit Green
18.   greenOff()
19. exit Operating
20.   logFault()
21. enter Flashing
22.   flashOn()
23. event Repair
24. transition Flashing -> Operating (h
25. exit Flashing
26.   flashOff()
27. enter Operating
28. enter Green
29.   greenOn()
```

No problems

Figure 5.4: After `Repair` the light is green again, resumed from history.

5.6 Step 6 — Generate table-based code with tracing

Select **C++**, **Table-based**, tick **Trace** and click **Generate Code**. The table-based class keeps the hierarchy, entry/exit handlers and transitions in constant arrays. The core of `Dispatch` is a short loop over the transition table, and `TryRow` fires a matching row:

Listing 5.1: `HandleEvent` in the table-based flat traffic light.

```
bool HandleEvent(State state, Event event) {
    for (const TransitionRow& row : kTransitions) {
        if (row.source == state && row.event == event && TryRow(row)) {
            return true;
        }
    }
    return false;
}
```

Listing 5.2: `TryRow`: guard check, exit, action, entry.

```
bool TryRow(const TransitionRow& row) {
    if (row.guard != nullptr && (this->*row.guard)() == row.negate) {
        return false;
    }
    State root = row.source;
    while (ParentOf(root) != row.lca) {
        root = ParentOf(root);
    }
    ExitState(root);
    Invoke(row.action);
    EnterDown(row.lca, row.target);
    return true;
}
```

With history, `ExitState` records in an array the state that was last exited from each region, so the array always describes the configuration that every composite state had when it was last left. A transition to the history pseudostate enters `Operating` through `EnterHistory`, which re-enters the recorded substate instead of following the initial transition:

Listing 5.3: History restoration in the generated hierarchical traffic light.

```
void EnterHistory(State state, bool deep) {
    const std::size_t first = FirstRegionOf(state);
    for (std::size_t region = first; region < first + RegionsOf(state); ++region) {
        const State remembered = history_[region];
        if (remembered == State::None) {
            EnterRegion(region);
        } else if (deep) {
            EnterState(remembered);
            EnterHistory(remembered, true);
        } else {
            EnterDown(state, remembered, false);
        }
    }
}
```

5.7 Step 7 — Run it

The program below implements the hooks and the `Trace` method and plays the same scenario as the simulator, followed by `PowerOff`.

```
// Tutorial 2: hierarchical TrafficLight (table-based C++ with tracing).
#include <cstdio>

#include "TrafficLight.hpp"

using Demo::TrafficLight;

class Light : public TrafficLight {
  protected:
    void redOn() override        { std::puts("    redOn()"); }
    void redOff() override       { std::puts("    redOff()"); }
    void yellowOn() override     { std::puts("    yellowOn()"); }
    void yellowOff() override    { std::puts("    yellowOff()"); }
    void greenOn() override      { std::puts("    greenOn()"); }
    void greenOff() override     { std::puts("    greenOff()"); }
    void flashOn() override      { std::puts("    flashOn()"); }
    void flashOff() override     { std::puts("    flashOff()"); }
    void logFault() override     { std::puts("    logFault()"); }
    void shortenGreen() override { std::puts("    shortenGreen()"); }

    void Trace(TraceKind kind, const char* name) override {
        static const char* const kinds[] = { "start", "event", "ignored", "transition", "exit",
     "enter" };
        std::printf("  %-10s %s\n", kinds[static_cast<int>(kind)], name);
    }
};
```

```
int main() {
    Light light;
    light.Start();
    const TrafficLight::Event script[] = {
        TrafficLight::Event::PowerOn, TrafficLight::Event::Timer, TrafficLight::Event::
    PedestrianRequest,
        TrafficLight::Event::Fault,   TrafficLight::Event::Repair, TrafficLight::Event::
    PowerOff,
    };
    for (TrafficLight::Event e : script) {
        std::printf("Dispatch(%s)\n", TrafficLight::EventName(e));
        light.Dispatch(e);
    }
    std::printf("final state: %s\n", TrafficLight::StateName(light.Current()));
}
```

Listing 5.4: Output of the Tutorial 2 program.

```
  start      TrafficLight
  enter      Off
Dispatch(PowerOn)
  event      PowerOn
  transition Off -> Operating
  exit       Off
  enter      Operating
  enter      Red
    redOn()
Dispatch(Timer)
  event      Timer
  transition Red -> Green
  exit       Red
    redOff()
  enter      Green
    greenOn()
Dispatch(PedestrianRequest)
  event      PedestrianRequest
  transition Green (internal)
    shortenGreen()
Dispatch(Fault)
  event      Fault
  transition Operating -> Flashing
  exit       Green
    greenOff()
  exit       Operating
    logFault()
  enter      Flashing
    flashOn()
Dispatch(Repair)
  event      Repair
  transition Flashing -> Operating (history)
  exit       Flashing
    flashOff()
  enter      Operating
  enter      Green
    greenOn()
Dispatch(PowerOff)
  event      PowerOff
  transition Operating -> Off
  exit       Green
    greenOff()
  exit       Operating
  enter      Off
final state: Off
```

Compare the output with the simulator trace in Figure 5.4: they describe the same sequence of exits, actions and entries. Note in particular:

- `PowerOn` enters `Operating` and then follows its initial transition to `Red`.
- `PedestrianRequest` only calls `shortenGreen()` — no exit, no entry.

- `Fault` exits the innermost state first (`Green`, then `Operating`), runs the transition action `logFault()`, and only then enters `Flashing`.
- `Repair` enters `Operating` and resumes `Green` instead of `Red`.

Chapter 6

Tutorial 3: A Thread-Safe Door Lock in C#

This tutorial builds an electronic door lock that is opened with a numeric code. It shows *event parameters*, *guards*, transition priority, and the optional *tracing* and *thread safety* features, using C# and the table-based style.

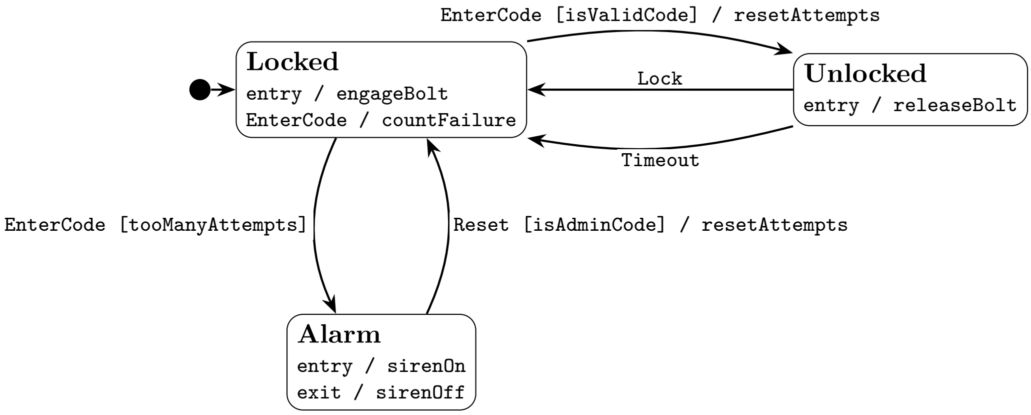
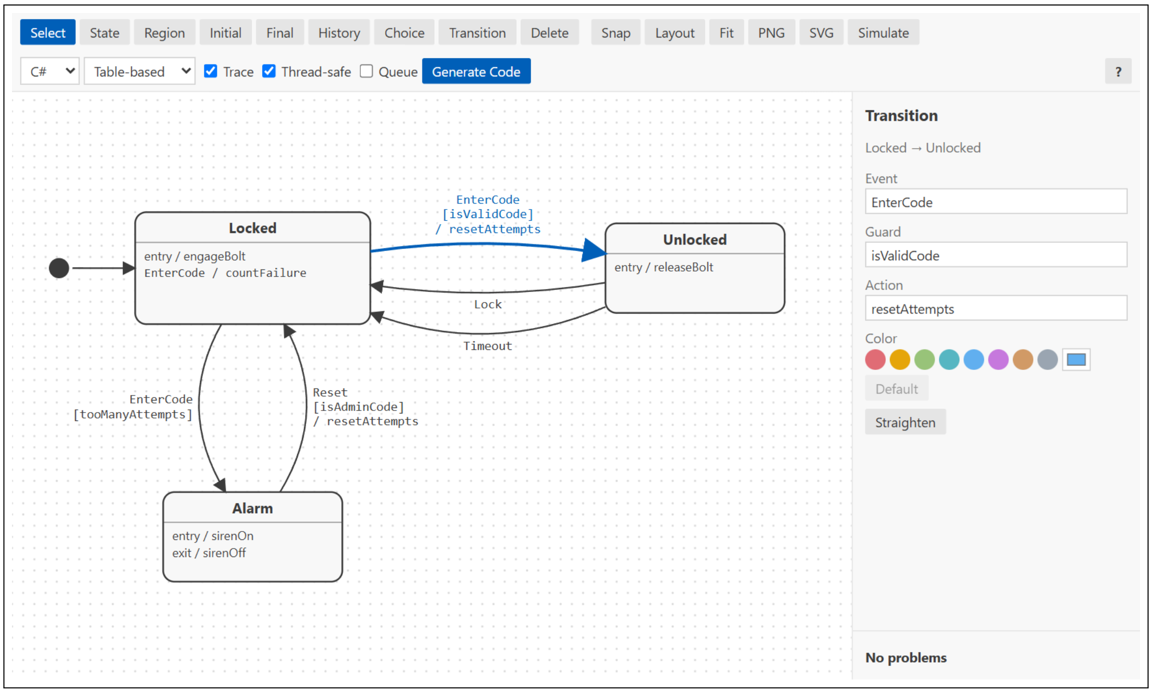

EnterCode [isValidCode] / resetAttempts
Locked
entry / engageBolt
EnterCode / countFailure
Lock
Unlocked
entry / releaseBolt
Timeout
EnterCode [tooManyAttempts]
Reset [isAdminCode] / resetAttempts
Alarm
entry / sirenOn
exit / sirenOff

Figure 6.1: Goal of Tutorial 3 (`examples/DoorLock.machina`).

6.1 Step 1 — States and transitions

Create a new diagram `DoorLock.machina` with class name `DoorLock` and namespace `Security`, and draw the states and transitions shown above:

- `Locked` (entry `engageBolt`), `Unlocked` (entry `releaseBolt`) and `Alarm` (entry `sirenOn`, exit `sirenOff`).
- Two transitions from `Locked` with the event `EnterCode`: to `Unlocked` with guard `isValidCode` and action `resetAttempts`, and to `Alarm` with guard `tooManyAttempts`.
- An internal transition in `Locked`: `EnterCode / countFailure`.
- `Unlocked` → `Locked` with `Lock` and with `Timeout`; `Alarm` → `Locked` with `Reset [isAdminCode] / resetAttempts`.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Locked
entry / engageBolt
EnterCode / countFailure
Unlocked
entry / releaseBolt
Alarm
entry / sirenOn
exit / sirenOff
EnterCode
[isValidCode]
/ resetAttempts
Lock
Timeout
EnterCode
[tooManyAttempts]
Reset
[isAdminCode]
/ resetAttempts
Color
Default
Straighten

C# Table-based ☑ Trace ☑ Thread-safe ☐ Queue

Transition

Locked → Unlocked

Event: EnterCode

Guard: isValidCode

Action: resetAttempts

No problems

Figure 6.2: The guarded transition `EnterCode [isValidCode] / resetAttempts`.

6.2 Step 2 — Understand the priority of guarded transitions

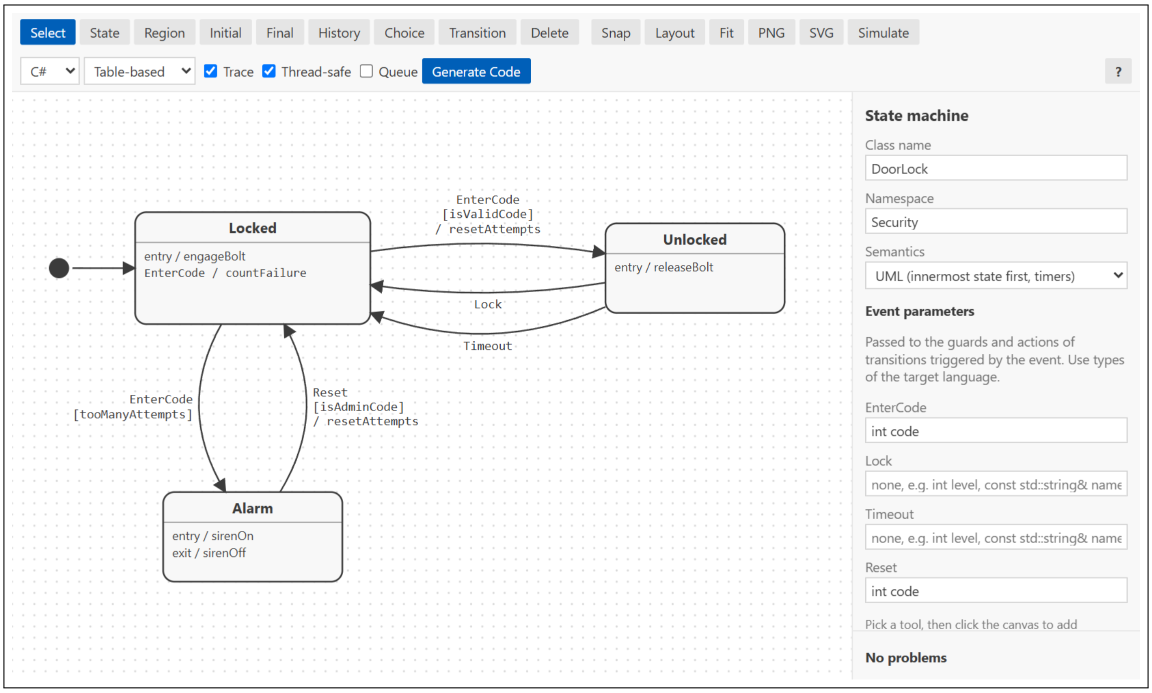
`Locked` now has three transitions for `EnterCode`. When the event arrives, MachinaState tries them in a fixed order:

1. guarded transitions first, in the order they appear in the diagram — `[isValidCode]`, then `[tooManyAttempts]`;
2. then the unguarded one — the internal `countFailure`.

The first transition whose guard is true fires. A wrong code therefore falls through both guards and is counted by `countFailure`; after too many failures `tooManyAttempts` becomes true and the alarm goes off. A state may have at most one unguarded transition per event, because a second one could never fire.

6.3 Step 3 — Event parameters

The guards need to see the code that was entered. Click on empty canvas to show the machine properties. Under **Event parameters**, enter `int code` for `EnterCode` and for `Reset`.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Locked
entry / engageBolt
EnterCode / countFailure
Unlocked
entry / releaseBolt
Alarm
entry / sirenOn
exit / sirenOff
EnterCode
[isValidCode]
/ resetAttempts
Lock
Timeout
EnterCode
[tooManyAttempts]
Reset
[isAdminCode]
/ resetAttempts

C# | Table-based | ☑ Trace ☑ Thread-safe ☐ Queue

State machine

Class name: DoorLock

Namespace: Security

Semantics: UML (innermost state first, timers)

Event parameters

Passed to the guards and actions of transitions triggered by the event. Use types of the target language.

EnterCode: int code

Lock: none, e.g. int level, const std::string& name

Timeout: none, e.g. int level, const std::string& name

Reset: int code

Pick a tool, then click the canvas to add

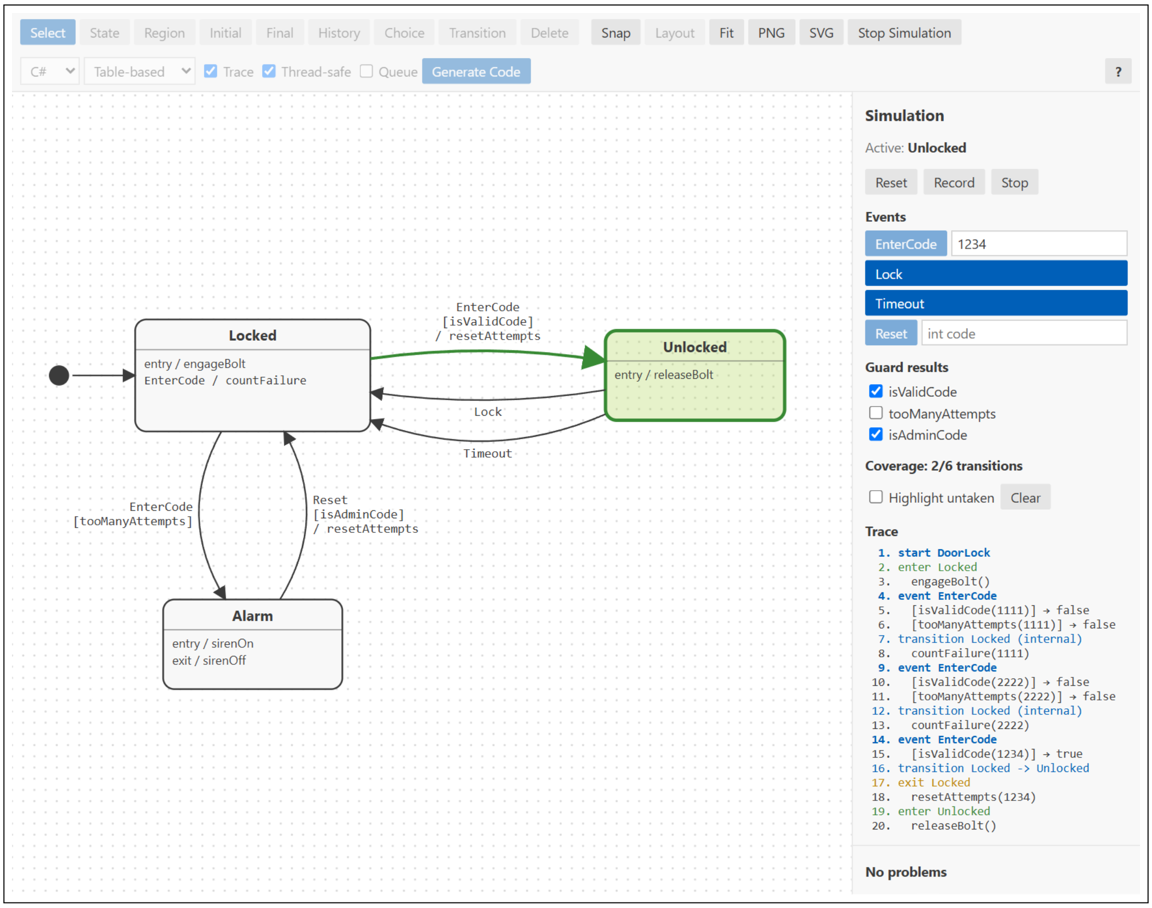
No problems

Figure 6.3: Event parameters are declared once per event in the machine properties.

All guards and actions of transitions triggered by `EnterCode` now receive the parameter: `isValidCode(int code)`, `tooManyAttempts(int code)`, `resetAttempts(int code)` and `countFailure(int code)`. Write parameter types in the syntax of the language you generate — `int code` is valid in both C++ and C#, while e.g. `const std::string& name` is C++ only.

6.4 Step 4 — Simulate

Because the simulator cannot run your code, *you* decide what each guard returns: the **Guard results** list has a check box per guard. Untick **isValidCode** and **tooManyAttempts**, type a code into the argument field of `EnterCode` and fire it twice — both guards are false, so the internal transition counts the failures. Then tick **isValidCode** and fire `EnterCode` with `1234`. The trace shows every guard call with its argument and result:


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Stop Simulation
Generate Code
?
Locked
entry / engageBolt
EnterCode / countFailure
EnterCode
[isValidCode]
/ resetAttempts
Unlocked
entry / releaseBolt
Lock
Timeout
EnterCode
[tooManyAttempts]
Reset
[isAdminCode]
/ resetAttempts
Alarm
entry / sirenOn
exit / sirenOff
Simulation
Reset
Record
Stop
Events
EnterCode
Lock
Timeout
Reset
Clear

C# Table-based ☑ Trace ☑ Thread-safe ☐ Queue

Active: **Unlocked**

EnterCode: 1234

Reset: int code

Guard results

☑ isValidCode
☐ tooManyAttempts
☑ isAdminCode

Coverage: 2/6 transitions

☐ Highlight untaken

Trace

```
 1. start DoorLock
 2. enter Locked
 3.   engageBolt()
 4. event EnterCode
 5.   [isValidCode(1111)] → false
 6.   [tooManyAttempts(1111)] → false
 7. transition Locked (internal)
 8.   countFailure(1111)
 9. event EnterCode
10.   [isValidCode(2222)] → false
11.   [tooManyAttempts(2222)] → false
12. transition Locked (internal)
13.   countFailure(2222)
14. event EnterCode
15.   [isValidCode(1234)] → true
16. transition Locked -> Unlocked
17. exit Locked
18.   resetAttempts(1234)
19. enter Unlocked
20.   releaseBolt()
```

No problems

Figure 6.4: Two failed attempts are counted, then the valid code unlocks the door.

6.5 Step 5 — Generate C# with tracing and thread safety

Select **C#**, **Table-based**, tick **Trace** and **Thread-safe**, and click **Generate Code**. The interesting parts of the generated `DoorLock.cs`:

Listing 6.1: The transition table is built in the constructor. Arguments are passed to hooks from fields set by the typed dispatch method.

```
protected DoorLock()
{
    regionActions = new Action?[] { null };
    entryActions = new Action?[] { engageBolt, releaseBolt, sirenOn };
    exitActions = new Action?[] { null, null, sirenOff };
    // source, event, guard, action, target, least common ancestor, trace name
    transitions = new TransitionRow[]
    {
        new TransitionRow(State.Locked, Event.EnterCode, () => isValidCode(EnterCode_code), ()
    => { resetAttempts(EnterCode_code); }, State.Unlocked, State.None, "Locked -> Unlocked"),
        new TransitionRow(State.Locked, Event.EnterCode, () => tooManyAttempts(EnterCode_code),
     null, State.Alarm, State.None, "Locked -> Alarm"),
        new TransitionRow(State.Locked, Event.EnterCode, null, () => { countFailure(
    EnterCode_code); }, State.None, State.None, "Locked (internal)"),
        new TransitionRow(State.Unlocked, Event.Lock, null, null, State.Locked, State.None, "
    Unlocked -> Locked"),
        new TransitionRow(State.Unlocked, Event.Timeout, null, null, State.Locked, State.None,
    "Unlocked -> Locked"),
        new TransitionRow(State.Alarm, Event.Reset, () => isAdminCode(Reset_code), () => {
    resetAttempts(Reset_code); }, State.Locked, State.None, "Alarm -> Locked"),
    };
}
```

Listing 6.2: Each event with parameters gets a typed dispatch method. `Lock`/`Unlock` surround every public member.

```
public bool DispatchEnterCode(int code)
{
    Lock();
    try
    {
        EnterCode_code = code;
        return DispatchCore(Event.EnterCode);
    }
    finally
    {
        Unlock();
    }
}

public bool DispatchReset(int code)
{
    Lock();
    try
    {
        Reset_code = code;
        return DispatchCore(Event.Reset);
    }
    finally
    {
        Unlock();
    }
}
```

The hooks you must implement:

Listing 6.3: The hooks to implement.

```
/// <summary>Acquires mutual exclusion around every public member (e.g. Monitor.Enter on a
    private object).</summary>
/// <remarks>Hooks must not call public members of this class unless the lock is re-entrant.</
    remarks>
protected abstract void Lock();
/// <summary>Releases the lock acquired by <see cref="Lock"/>.</summary>
protected abstract void Unlock();
/// <summary>Override to log state machine activity.</summary>
protected virtual void Trace(TraceKind kind, string name)
{
}
protected abstract bool isValidCode(int code);
protected abstract bool tooManyAttempts(int code);
protected abstract bool isAdminCode(int code);
protected abstract void engageBolt();
protected abstract void releaseBolt();
protected abstract void sirenOn();
protected abstract void sirenOff();
protected abstract void resetAttempts(int code);
protected abstract void countFailure(int code);
```

6.6 Step 6 — Implement and run

Listing 6.4: `Program.cs` of the door lock demo.

```
// Tutorial 3: thread-safe DoorLock (table-based C# with tracing and event parameters).
using System;
using System.Threading;
using System.Threading.Tasks;
using Security;

sealed class Door : DoorLock
{
    private const int Secret = 4711;
    private const int Admin = 9999;
    private readonly object gate = new();
    private int failures;

    protected override void Lock() => Monitor.Enter(gate);
    protected override void Unlock() => Monitor.Exit(gate);

    protected override bool isValidCode(int code) => code == Secret;
    protected override bool tooManyAttempts(int code) => failures >= 2 && code != Secret;
    protected override bool isAdminCode(int code) => code == Admin;
    protected override void countFailure(int code) => Console.WriteLine($"    countFailure({code}) -> {++failures} failure(s)");
    protected override void resetAttempts(int code) => failures = 0;

    protected override void engageBolt() => Console.WriteLine("    engageBolt()");
    protected override void releaseBolt() => Console.WriteLine("    releaseBolt()");
    protected override void sirenOn() => Console.WriteLine("    sirenOn()");
    protected override void sirenOff() => Console.WriteLine("    sirenOff()");

    protected override void Trace(TraceKind kind, string name) => Console.WriteLine($"  {kind,-10} {name}");
}

static class Program
{
    static void Main()
    {
        var door = new Door();
        door.Start();

        foreach (var code in new[] { 1234, 1111, 2222 })
        {
            Console.WriteLine($"DispatchEnterCode({code})");
            door.DispatchEnterCode(code);
        }
        Console.WriteLine($"Dispatch(Lock) handled: {door.Dispatch(DoorLock.Event.Lock)}");
        Console.WriteLine($"Dispatch(EnterCode) handled: {door.Dispatch(DoorLock.Event.EnterCode)}   // needs its typed method");
        Console.WriteLine("DispatchReset(9999)");
        door.DispatchReset(9999);
        Console.WriteLine("DispatchEnterCode(4711)");
        door.DispatchEnterCode(4711);
        Console.WriteLine($"state: {door.Current}");

        // Many threads may use the machine at once; Lock()/Unlock() serialise them.
        Parallel.For(0, 4, i => door.Dispatch(i % 2 == 0 ? DoorLock.Event.Lock : DoorLock.Event.Timeout));
        Console.WriteLine($"after concurrent calls: {door.Current}");
    }
}
```

Listing 6.5: Output of the Tutorial 3 program. The order of the last four events depends on thread scheduling.

```
  Start      DoorLock
  Enter      Locked
    engageBolt()
DispatchEnterCode(1234)
  Event      EnterCode
  Transition Locked (internal)
    countFailure(1234) -> 1 failure(s)
DispatchEnterCode(1111)
  Event      EnterCode
  Transition Locked (internal)
    countFailure(1111) -> 2 failure(s)
DispatchEnterCode(2222)
  Event      EnterCode
  Transition Locked -> Alarm
  Exit       Locked
  Enter      Alarm
    sirenOn()
  Event      Lock
  Ignored    Lock
Dispatch(Lock) handled: False
Dispatch(EnterCode) handled: False   // needs its typed method
DispatchReset(9999)
  Event      Reset
  Transition Alarm -> Locked
  Exit       Alarm
    sirenOff()
  Enter      Locked
    engageBolt()
DispatchEnterCode(4711)
  Event      EnterCode
  Transition Locked -> Unlocked
  Exit       Locked
  Enter      Unlocked
    releaseBolt()
state: Unlocked
  Event      Timeout
  Transition Unlocked -> Locked
  Exit       Unlocked
  Enter      Locked
    engageBolt()
  Event      Lock
  Ignored    Lock
  Event      Lock
  Ignored    Lock
  Event      Timeout
  Ignored    Timeout
after concurrent calls: Locked
```

Notice:

- `Dispatch(Event.EnterCode)` returns `false` — events with parameters must be sent with their typed method `DispatchEnterCode(int)`.
- While the alarm sounds, `Lock` is *ignored* because no active state handles it.
- `Parallel.For` calls `Dispatch` from several threads; `Lock()`/`Unlock()` make each call atomic.

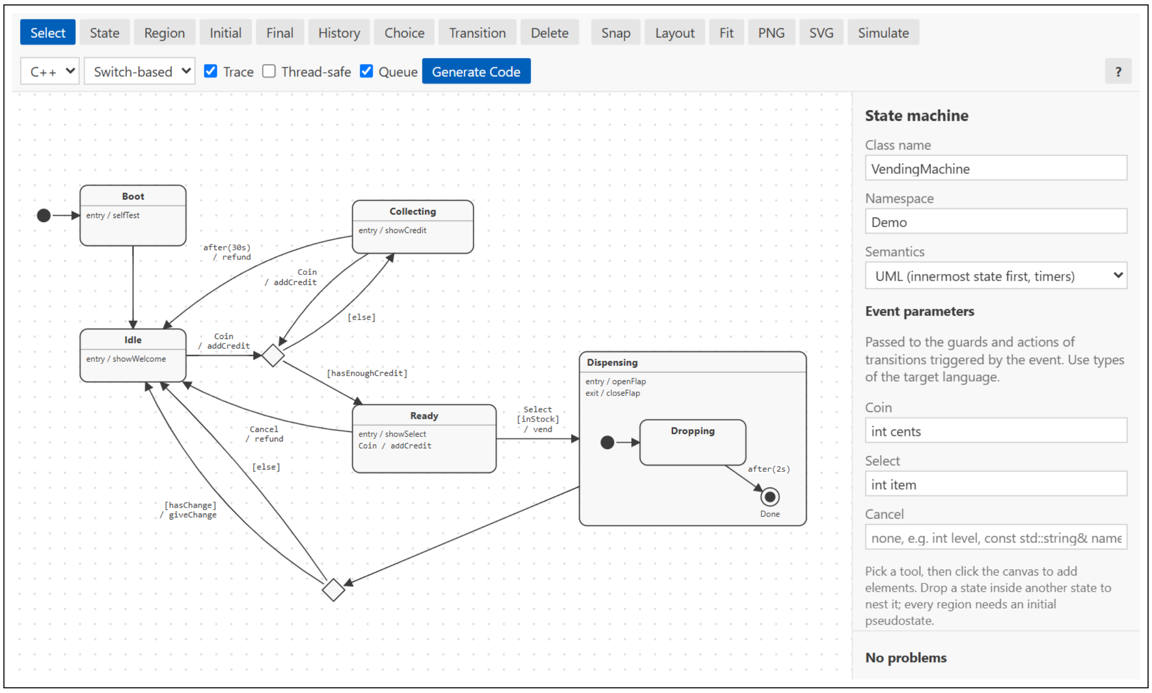
Chapter 7

Tutorial 4: A Vending Machine with Choices, Timeouts and a Queue

The last tutorial uses the newest modelling elements. The vending machine collects coins until the credit covers the price, dispenses the selected item, returns change, and gives up after 30 seconds without a coin. You will meet:

- a **completion transition** that leaves `Boot` as soon as it has been entered;
- a **choice point** that branches on the credit, shared by two incoming transitions;
- **timeouts** with `after(30s)` and `after(2s)`;
- the **completion of a composite state** when its region reaches a final state;
- the **event queue**, used by an action to raise a follow-up event.

The finished diagram is `examples/VendingMachine.machina` (Figure 7.1).


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Boot
entry / selfTest
Collecting
entry / showCredit
after(30s)
/ refund
Coin
/ addCredit
[else]
Idle
entry / showWelcome
Coin
/ addCredit
[hasEnoughCredit]
Dispensing
entry / openFlap
exit / closeFlap
Ready
entry / showSelect
Coin / addCredit
Select
[inStock]
/ vend
Dropping
after(2s)
Done
Cancel
/ refund
[else]
[hasChange]
/ giveChange

C++ · Switch-based · ☑ Trace ☐ Thread-safe ☑ Queue

State machine

Class name: VendingMachine

Namespace: Demo

Semantics: UML (innermost state first, timers)

Event parameters

Passed to the guards and actions of transitions triggered by the event. Use types of the target language.

Coin: int cents

Select: int item

Cancel: none, e.g. int level, const std::string& name

Pick a tool, then click the canvas to add elements. Drop a state inside another state to nest it; every region needs an initial pseudostate.

No problems

Figure 7.1: The finished vending machine.

7.1 Step 1 — Boot with a completion transition

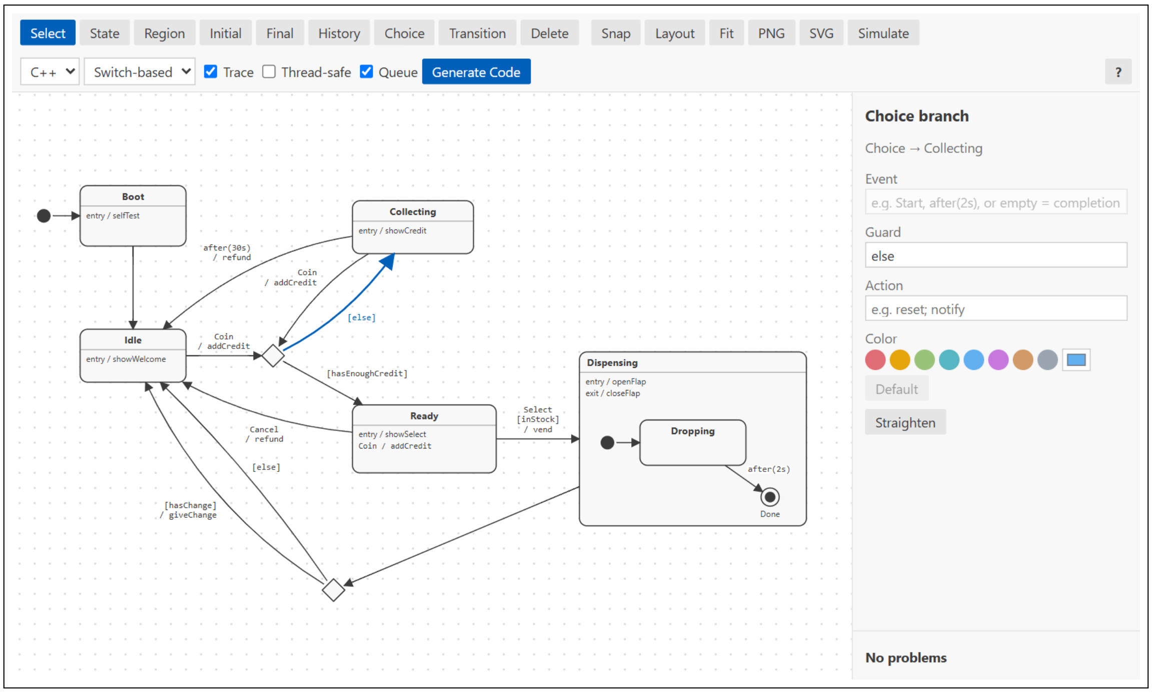
Add an initial pseudostate, a state `Boot` with entry action `selfTest` and a state `Idle` with entry action `showWelcome`. Connect `Boot` to `Idle` and *clear the event field*. A transition without an event is a *completion transition*: it fires on its own as soon as `Boot` has been entered (and its entry action has run). There is no need for an artificial “BootDone” event.

7.2 Step 2 — Collect coins through a choice point

Pick the **Choice** tool and place a diamond to the right of `Idle`. Add the states `Collecting` (entry `showCredit`) and `Ready` (entry `showSelect`) and connect:

1. `Idle` → choice with event `Coin` and action `addCredit`;
2. `Collecting` → choice with the same event and action;
3. choice → `Ready` with guard `hasEnoughCredit`;
4. choice → `Collecting` with guard `else`.

Branches leaving a choice point have no event; the **Event** field is disabled for them. The `[else]` branch is taken when no other guard is true (Figure 7.2).


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Boot
entry / selfTest
Collecting
entry / showCredit
after(30s)
/ refund
Coin
/ addCredit
[else]
Idle
entry / showWelcome
Coin
/ addCredit
[hasEnoughCredit]
Dispensing
entry / openFlap
exit / closeFlap
Ready
entry / showSelect
Coin / addCredit
Select
[inStock]
/ vend
Dropping
after(2s)
Done
Cancel
/ refund
[else]
[hasChange]
/ giveChange
Event
e.g. Start, after(2s), or empty = completion
Action
e.g. reset; notify
Color
Default
Straighten

C++ · Switch-based · ☑ Trace ☐ Thread-safe ☑ Queue

Choice branch

Choice → Collecting

Guard: else

No problems

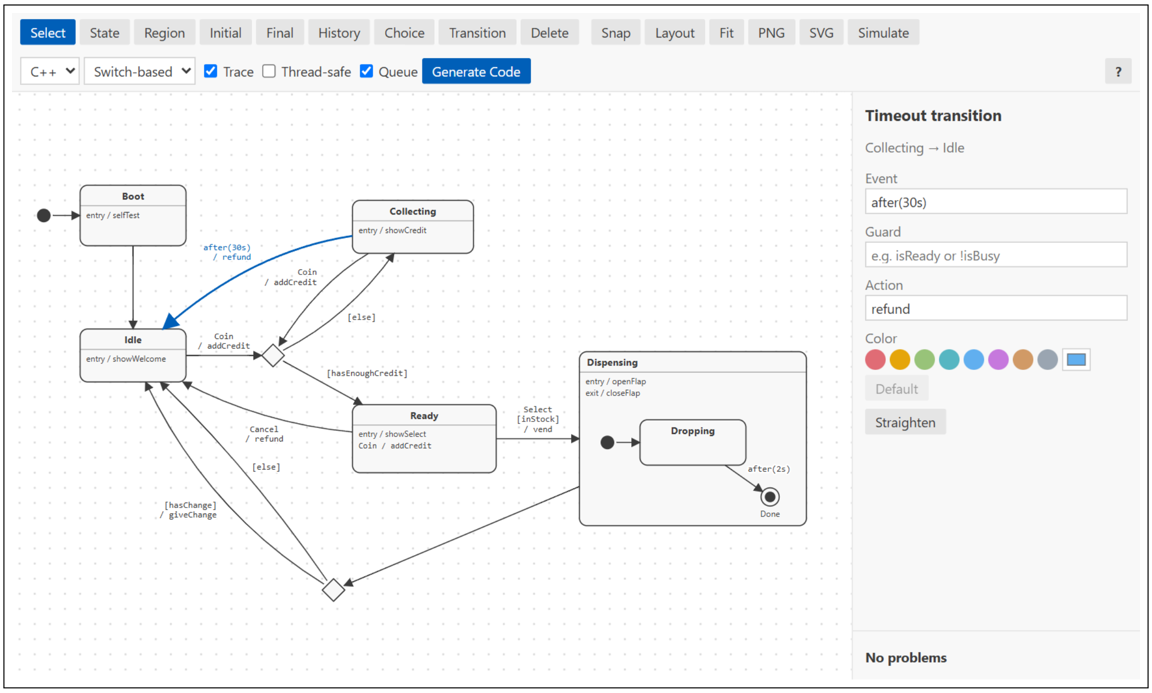
Figure 7.2: The `[else]` branch of the choice point is selected.

With nothing selected, enter the parameter list `int cents` for `Coin` and `int item` for `Select`. Because the choice branches belong to transitions triggered by `Coin`, the guard is generated as `hasEnoughCredit(int cents)` as well.

> **Note**
>
> MachinaState evaluates the guards of a choice point *before* the transition starts, i.e. before any state is exited and before `addCredit` runs. The guard therefore has to add the incoming coin itself: `credit + cents >= price`. Section 10.10 explains why.

7.3 Step 3 — Give up after 30 seconds

Connect `Collecting` → `Idle` with the event `after(30s)` and the action `refund`. A timeout is written as `after(`n`ms|s|min)`. The generated code starts a timer whenever `Collecting` is entered and cancels it whenever `Collecting` is left — so every coin, which leaves and re-enters `Collecting` through the choice point, restarts the 30 seconds (Figure 7.3).


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Boot
entry / selfTest
Collecting
entry / showCredit
after(30s)
/ refund
Coin
/ addCredit
[else]
Idle
entry / showWelcome
Coin
/ addCredit
[hasEnoughCredit]
Dispensing
entry / openFlap
exit / closeFlap
Ready
entry / showSelect
Coin / addCredit
Select
[inStock]
/ vend
Dropping
after(2s)
Done
Cancel
/ refund
[else]
[hasChange]
/ giveChange
Color
Default
Straighten

C++ Switch-based ☑ Trace ☐ Thread-safe ☑ Queue

Timeout transition

Collecting → Idle

Event: after(30s)

Guard: e.g. isReady or !isBusy

Action: refund

No problems

Figure 7.3: A timeout transition.

7.4 Step 4 — Dispense, then return change

Add a composite state `Dispensing` (entry `openFlap`, exit `closeFlap`) containing an initial pseudostate, a state `Dropping` and a final state `Done`, with `Dropping` → `Done` on `after(2s)`. Connect:

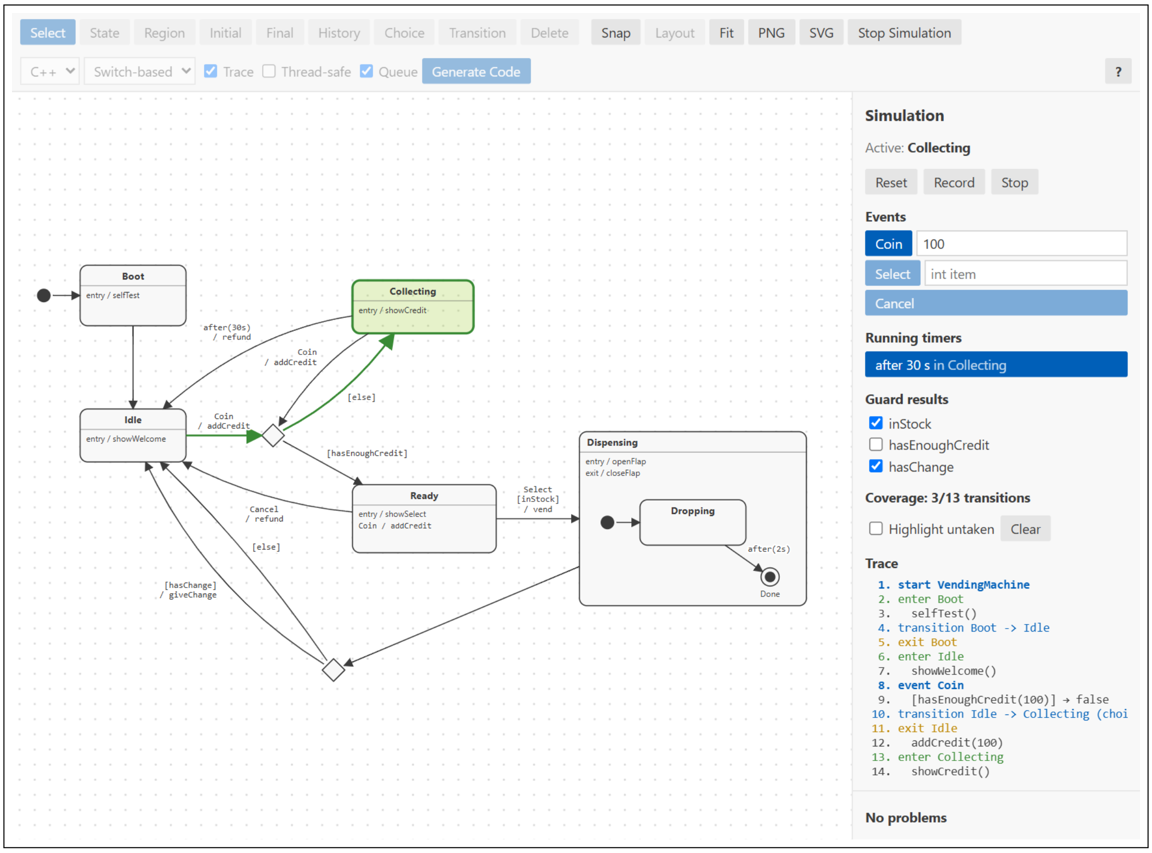
- `Ready` → `Dispensing` on `Select [inStock] / vend`;
- `Ready` → `Idle` on `Cancel / refund`;
- an internal transition `Coin / addCredit` on `Ready` (draw a self transition and tick **Internal**);
- a completion transition from `Dispensing` to a second choice point with the branches `[hasChange] / giveChange` and `[else]`, both to `Idle`.

The completion transition of `Dispensing` fires when its region reaches `Done`.

7.5 Step 5 — Simulate

Press **Simulate**. The simulator starts the machine like `Start()` does: `Boot` is entered, `selfTest` runs, and the completion transition takes the machine straight on to `Idle`.

The simulator cannot run your guard methods, so it lists every guard under **Guard results** with a check box: ticked means the guard returns `true`. Untick **hasEnoughCredit**, type `100` into the argument field next to **Coin** and click **Coin**. The transition reaches the choice point, `[hasEnoughCredit]` is false, so the `[else]` branch leads back to `Collecting` (Figure 7.4). Entering `Collecting` started its 30-second timer, which now appears under **Running timers**; clicking it lets the timer elapse.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Stop Simulation
Generate Code
?
Boot
entry / selfTest
Collecting
entry / showCredit
after(30s)
/ refund
Coin
/ addCredit
[else]
Idle
entry / showWelcome
Coin
/ addCredit
[hasEnoughCredit]
Dispensing
entry / openFlap
exit / closeFlap
Ready
entry / showSelect
Coin / addCredit
Select
[inStock]
/ vend
Dropping
after(2s)
Done
Cancel
/ refund
[else]
[hasChange]
/ giveChange
Simulation
Reset
Record
Stop
Events
Coin
Select
Cancel
Running timers
Guard results
Trace
Clear

C++ · Switch-based · ☑ Trace · ☐ Thread-safe · ☑ Queue

Active: **Collecting**

Coin: 100

Select: int item

after 30 s in Collecting

☑ inStock
☐ hasEnoughCredit
☑ hasChange

Coverage: 3/13 transitions

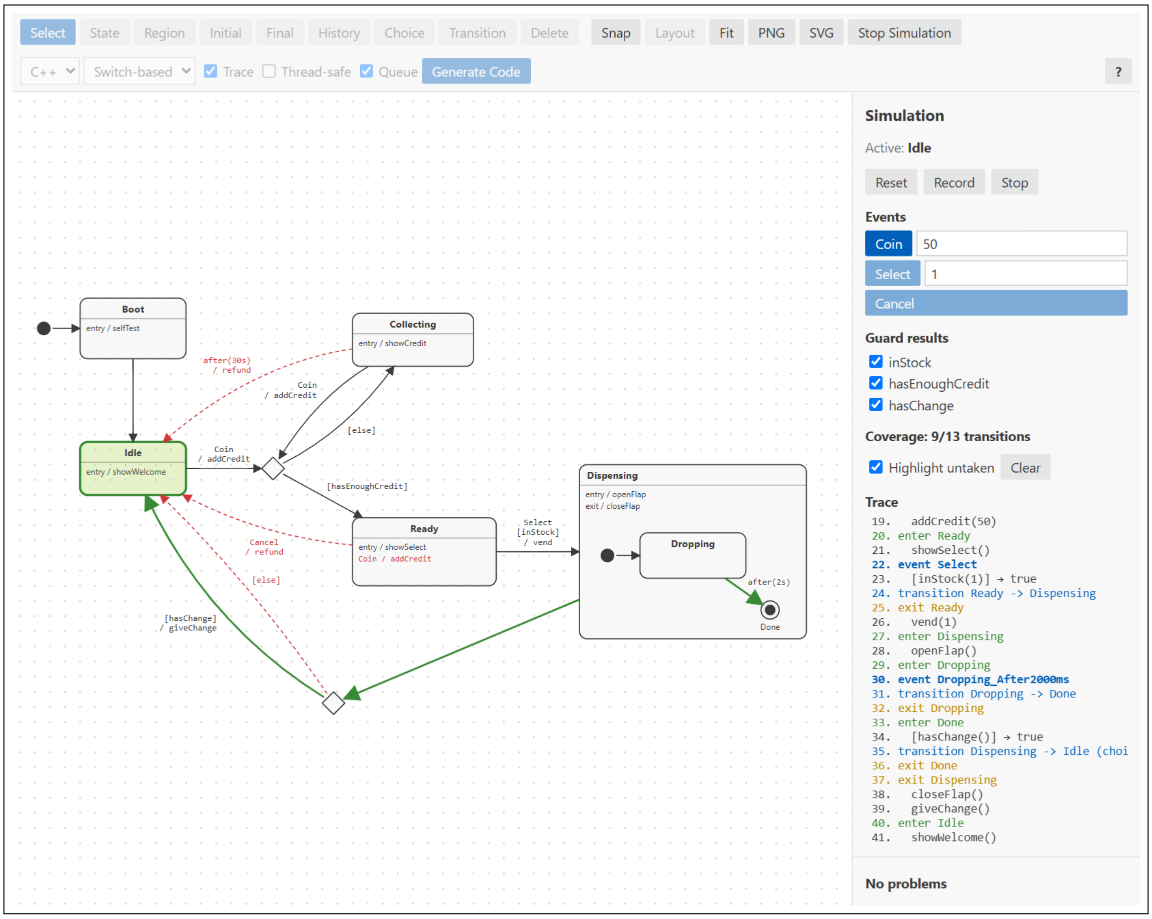
☐ Highlight untaken

```
 1. start VendingMachine
 2. enter Boot
 3.   selfTest()
 4. transition Boot -> Idle
 5. exit Boot
 6. enter Idle
 7.   showWelcome()
 8. event Coin
 9.   [hasEnoughCredit(100)] → false
10. transition Idle -> Collecting (choi
11. exit Idle
12.   addCredit(100)
13. enter Collecting
14.   showCredit()
```

No problems

Figure 7.4: The `[else]` branch was taken. The timer of `Collecting` is running; the trace shows the guard result next to the guard call.

Now tick **hasEnoughCredit**, fire `Coin` with `50` and `Select` with `1`, and click **after 2 s**. Entering the final state `Done` completes `Dispensing`; its completion transition runs through the second choice point and, since `hasChange` is ticked, calls `giveChange` on the way to `Idle` — all within the same step, exactly as in the generated code (Figure 7.5).


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Stop Simulation
C++
Switch-based
☑ Trace
☐ Thread-safe
☑ Queue
Generate Code
?
Boot
entry / selfTest
Collecting
entry / showCredit
after(30s)
/ refund
Coin
/ addCredit
[else]
Idle
entry / showWelcome
Coin
/ addCredit
[hasEnoughCredit]
Ready
entry / showSelect
Coin / addCredit
Cancel
/ refund
[else]
[hasChange]
/ giveChange
Select
[inStock]
/ vend
Dispensing
entry / openFlap
exit / closeFlap
Dropping
after(2s)
Done
Reset
Record
Stop
Clear

Simulation

Active: **Idle**

Events

Coin 50

Select 1

Cancel

Guard results

☑ inStock

☑ hasEnoughCredit

☑ hasChange

Coverage: 9/13 transitions

☑ Highlight untaken

Trace

```
19.   addCredit(50)
20. enter Ready
21.   showSelect()
22. event Select
23.   [inStock(1)] → true
24. transition Ready -> Dispensing
25. exit Ready
26.   vend(1)
27. enter Dispensing
28.   openFlap()
29. enter Dropping
30. event Dropping_After2000ms
31. transition Dropping -> Done
32. exit Dropping
33. enter Done
34.   [hasChange()] → true
35. transition Dispensing -> Idle (choi
36. exit Done
37. exit Dispensing
38.   closeFlap()
39.   giveChange()
40. enter Idle
41.   showWelcome()
```

No problems

Figure 7.5: `Dispensing` has completed and the machine is back in `Idle`. With **Highlight untaken**, the transitions not yet covered are drawn in red.

The coverage counter shows how many transitions have been taken since the simulation started. Tick **Highlight untaken** to see the rest in red: here the timeout, `Cancel`, the internal `Coin` of `Ready` and the `[else]` branch of the change point are still untested.

7.6 Step 6 — Generate with trace and queue

Select **C++** and **Switch-based**, tick **Trace** and **Queue**, and click **Generate Code**. Besides the guard and action hooks, the class now requires two timer hooks and offers `Post`:

```
virtual void StartTimer(Event timeout, unsigned milliseconds, unsigned generation) = 0;
virtual void CancelTimer(Event timeout)                                            = 0;
bool DispatchTimeout(Event timeout, unsigned generation);  // public: report an elapsed timer
bool Post(Event event);                          // protected: queue an event from a hook
bool PostCoin(int cents);                        // typed variant for events with parameters
```

Timeouts are events with generated names such as `Collecting_After30000ms`. When a timer elapses, your code calls `DispatchTimeout` with the event and the `generation` number it was given by `StartTimer`. The machine ignores the call if the timer has been cancelled or restarted in the meantime, or if that timeout has already been dispatched, so a late timer can never fire a newer one early (Section 13.10).

7.7 Step 7 — Implement and run

The program uses a fake timer that main() lets “elapse”. It also shows the queue: when the credit exceeds a limit, `addCredit` posts `Cancel`, which is processed right after the current `Coin` has been handled.

Listing 7.1: `vending.cpp`.

```
// Tutorial 4: VendingMachine (switch-based C++ with trace and event queue).
#include <cstdio>

#include "VendingMachine.hpp"

using Demo::VendingMachine;

class Machine : public VendingMachine {
  public:
    // Demo timer: remembers the running timeout so main() can let it elapse.
    void Elapse() {
        if (timerRunning_) {
            timerRunning_ = false;
            std::printf("timer elapsed -> DispatchTimeout(%s, %u)\n", EventName(timer_),
    generation_);
            DispatchTimeout(timer_, generation_);
        }
    }

  protected:
    static constexpr int kPrice     = 150;
    static constexpr int kMaxCredit = 500;

    void StartTimer(Event timeout, unsigned milliseconds, unsigned generation) override {
        std::printf("    StartTimer(%s, %u, %u)\n", EventName(timeout), milliseconds,
    generation);
        timer_        = timeout;
        generation_   = generation;
        timerRunning_ = true;
    }
    void CancelTimer(Event timeout) override {
        std::printf("    CancelTimer(%s)\n", EventName(timeout));
        timerRunning_ = false;
    }

    // Choice guards are evaluated before the transition’s actions run, so the coin is not yet
    credited.
    bool hasEnoughCredit(int cents) override { return credit_ + cents >= kPrice; }
    bool inStock(int item) override          { return item < 3; }
    bool hasChange() override                { return credit_ > 0; }

    void addCredit(int cents) override {
        credit_ += cents;
        std::printf("    addCredit(%d) -> credit %d\n", cents, credit_);
        if (credit_ > kMaxCredit) {
            std::puts("    credit limit exceeded: Post(Cancel)");
            Post(Event::Cancel);
        }
    }
    void vend(int item) override {
        credit_ -= kPrice;
        std::printf("    vend(%d) -> credit %d\n", item, credit_);
    }
    void refund() override {
        std::printf("    refund() -> %d returned\n", credit_);
        credit_ = 0;
    }
    void giveChange() override {
        std::printf("    giveChange() -> %d returned\n", credit_);
        credit_ = 0;
    }
    void selfTest() override    { std::puts("    selfTest()"); }
    void showWelcome() override { std::puts("    showWelcome()"); }
    void showCredit() override  { std::puts("    showCredit()"); }
    void showSelect() override  { std::puts("    showSelect()"); }
    void openFlap() override    { std::puts("    openFlap()"); }
    void closeFlap() override   { std::puts("    closeFlap()"); }

    void Trace(TraceKind kind, const char* name) override {
        static const char* const kinds[] = { "start", "event", "ignored", "transition", "exit",
     "enter" };
        std::printf("  %-10s %s\n", kinds[static_cast<int>(kind)], name);
    }

  private:
    int      credit_       = 0;
    Event    timer_        = Event::Coin;
    unsigned generation_   = 0;
    bool     timerRunning_ = false;
};

int main() {
    Machine m;
    std::puts("Start()");
    m.Start();
    std::puts("DispatchCoin(100)");
    m.DispatchCoin(100);
    m.Elapse();
    std::puts("DispatchCoin(100)");
    m.DispatchCoin(100);
    std::puts("DispatchCoin(50)");
    m.DispatchCoin(50);
    std::puts("DispatchSelect(7)");
    m.DispatchSelect(7);
    std::puts("DispatchSelect(1)");
    m.DispatchSelect(1);
    m.Elapse();
    std::puts("DispatchCoin(200)");
    m.DispatchCoin(200);
    std::puts("DispatchCoin(400)");
    m.DispatchCoin(400);
    std::printf("final state: %s\n", VendingMachine::StateName(m.Current()));
}
```

Listing 7.2: Output of the Tutorial 4 program.

```
Start()
  start      VendingMachine
  enter      Boot
    selfTest()
  transition Boot -> Idle
  exit       Boot
  enter      Idle
    showWelcome()
DispatchCoin(100)
  event      Coin
  transition Idle -> Collecting (choice)
  exit       Idle
    addCredit(100) -> credit 100
  enter      Collecting
    showCredit()
    StartTimer(Collecting_After30000ms, 30000, 1)
timer elapsed -> DispatchTimeout(Collecting_After30000ms, 1)
  event      Collecting_After30000ms
  transition Collecting -> Idle
  exit       Collecting
    CancelTimer(Collecting_After30000ms)
    refund() -> 100 returned
  enter      Idle
    showWelcome()
DispatchCoin(100)
  event      Coin
  transition Idle -> Collecting (choice)
  exit       Idle
    addCredit(100) -> credit 100
  enter      Collecting
    showCredit()
    StartTimer(Collecting_After30000ms, 30000, 2)
DispatchCoin(50)
  event      Coin
  transition Collecting -> Ready (choice)
  exit       Collecting
    CancelTimer(Collecting_After30000ms)
    addCredit(50) -> credit 150
  enter      Ready
    showSelect()
DispatchSelect(7)
  event      Select
  ignored    Select
DispatchSelect(1)
  event      Select
  transition Ready -> Dispensing
  exit       Ready
    vend(1) -> credit 0
  enter      Dispensing
    openFlap()
  enter      Dropping
    StartTimer(Dropping_After2000ms, 2000, 1)
timer elapsed -> DispatchTimeout(Dropping_After2000ms, 1)
  event      Dropping_After2000ms
  transition Dropping -> Done
  exit       Dropping
    CancelTimer(Dropping_After2000ms)
  enter      Done
  transition Dispensing -> Idle (choice)
  exit       Done
  exit       Dispensing
    closeFlap()
  enter      Idle
    showWelcome()
DispatchCoin(200)
  event      Coin
  transition Idle -> Ready (choice)
  exit       Idle
    addCredit(200) -> credit 200
  enter      Ready
    showSelect()
DispatchCoin(400)
  event      Coin
  transition Ready (internal)
    addCredit(400) -> credit 600
    credit limit exceeded: Post(Cancel)
  event      Cancel
  transition Ready -> Idle
  exit       Ready
    refund() -> 600 returned
  enter      Idle
    showWelcome()
final state: Idle
```

Observe:

- the completion transition `Boot -> Idle` fires inside `Start()`;

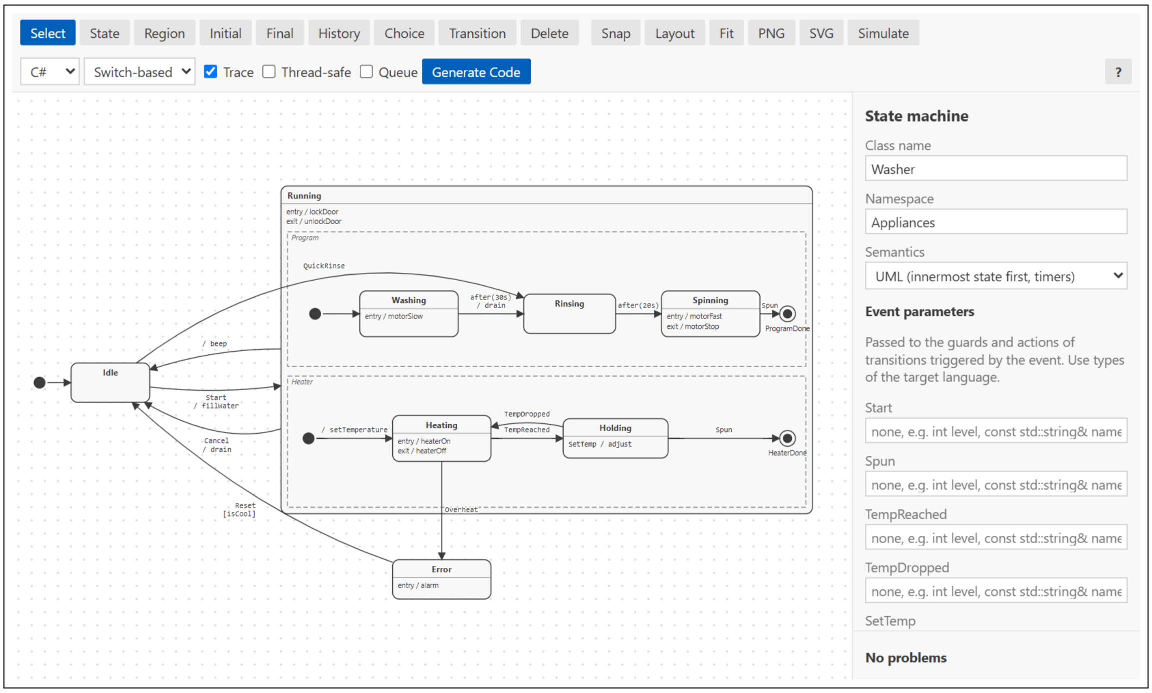
- the choice transitions are traced as `(choice)` and exit the source before running `addCredit`;
- `StartTimer`/`CancelTimer` bracket the time spent in `Collecting` and `Dropping`; the second visit to `Collecting` starts its timer with generation 2;
- reaching `Done` immediately triggers the completion of `Dispensing`;
- the posted `Cancel` is dispatched after the internal `Coin` transition has completed — never in the middle of it.

Chapter 8

Tutorial 5: A Washing Machine with Orthogonal Regions

A washing machine does several things at once: the drum runs through its programme while the heater keeps the water at temperature. Modelling both in one chain of states would need a state for every combination (`Washing+Heating`, `Washing+Holding`, `Rinsing+Heating`, ...). UML *orthogonal regions* keep them apart: a state is divided into regions, and while the state is active, *each* of its regions has its own active substate. This tutorial builds `examples/Washer.machina` (Figure 8.1) and then uses *scenarios* to turn simulator sessions into repeatable tests. You will meet:

- the **Region** tool and orthogonal states;
- events that are handled by **several regions at once**;
- the **completion of an orthogonal state** once every region has finished;
- transitions that **leave all regions** at once, from the state or from inside one region;
- **scenarios**: recording, saving, replaying and stepping;
- **transition coverage**.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Running
entry / lockDoor
exit / unlockDoor
Program
QuickRinse
Washing
entry / motorSlow
after(30s)
/ drain
Rinsing
after(20s)
Spinning
entry / motorFast
exit / motorStop
Spun
ProgramDone
/ beep
Idle
Start
/ fillWater
Cancel
/ drain
Heater
/ setTemperature
Heating
entry / heaterOn
exit / heaterOff
TempDropped
TempReached
Holding
SetTemp / adjust
Spun
HeaterDone
Reset
[isCool]
Overheat
Error
entry / alarm

C# | Switch-based | ☑ Trace ☐ Thread-safe ☐ Queue

State machine

Class name: Washer

Namespace: Appliances

Semantics: UML (innermost state first, timers)

Event parameters

Passed to the guards and actions of transitions triggered by the event. Use types of the target language.

Start: none, e.g. int level, const std::string& name

Spun: none, e.g. int level, const std::string& name

TempReached: none, e.g. int level, const std::string& name

TempDropped: none, e.g. int level, const std::string& name

SetTemp

No problems

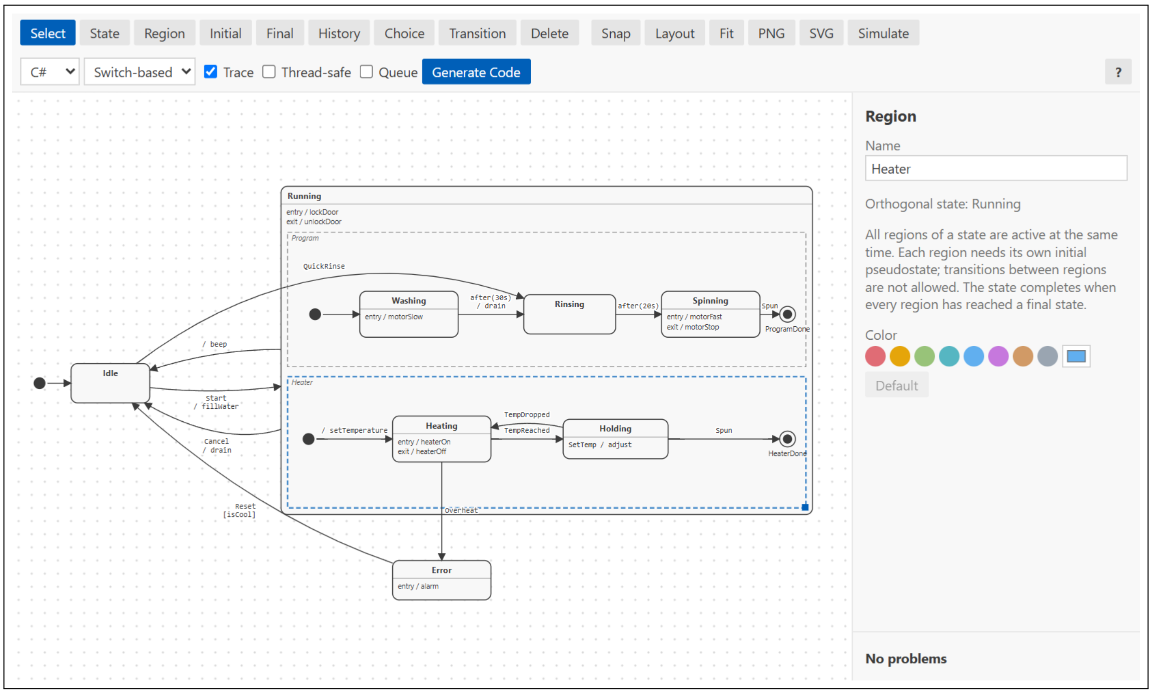
Figure 8.1: The finished washing machine. `Running` has two regions, `Program` and `Heater`.

8.1 Step 1 — The outer states

Create a new diagram with the class name `Washer` and the namespace `Appliances`. Add an initial pseudostate, a state `Idle`, a large state `Running` (entry `lockDoor`, exit `unlockDoor`) and a state `Error` (entry `alarm`). Connect `Idle` to `Running` with `Start / fillWater`.

8.2 Step 2 — Divide Running into two regions

Pick the **Region** tool and click inside `Running`: a dashed rectangle appears. Click again below it for the second region. Select each region and name it — `Program` and `Heater` (Figure 8.2). Region names appear in the diagram and the generated comments; they do not have to be identifiers. Resize the regions so that they fill `Running`; a region can only be placed inside a state, and once a state has regions, all of its substates must be placed inside one of them.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Running
entry / lockDoor
exit / unlockDoor
Program
QuickRinse
Washing
entry / motorSlow
after(30s)
/ drain
Rinsing
after(20s)
Spinning
entry / motorFast
exit / motorStop
Spun
ProgramDone
/ beep
Idle
Start
/ fillWater
Heater
/ setTemperature
Heating
entry / heaterOn
exit / heaterOff
TempDropped
TempReached
Holding
SetTemp / adjust
Spun
HeaterDone
Cancel
/ drain
Reset
[isCool]
Overheat
Error
entry / alarm
Default

C# Switch-based ☑ Trace ☐ Thread-safe ☐ Queue

Region

Name: Heater

Orthogonal state: Running

All regions of a state are active at the same time. Each region needs its own initial pseudostate; transitions between regions are not allowed. The state completes when every region has reached a final state.

Color

No problems

Figure 8.2: The `Heater` region is selected. Its properties name the orthogonal state it belongs to.

8.3 Step 3 — Fill the regions

Each region is a small state machine of its own and needs its own initial pseudostate:

- **Program**: `Washing` (entry `motorSlow`) → `Rinsing` on `after(30s) / drain`, → `Spinning` (entry `motorFast`, exit `motorStop`) on `after(20s)`, and → the final state `ProgramDone` on `Spun`.
- **Heater**: an initial transition with the action `setTemperature` to `Heating` (entry `heaterOn`, exit `heaterOff`); `Heating` → `Holding` on `TempReached` and back on `TempDropped`; an internal transition `SetTemp / adjust` on `Holding` with the parameter `int celsius`; and `Holding` → the final state `HeaterDone` on `Spun`.

Note that `Spun` appears in both regions. An event is offered to every active region, so one `Spun` finishes the programme and switches off the heater in the same step.

8.4 Step 4 — Leaving the orthogonal state

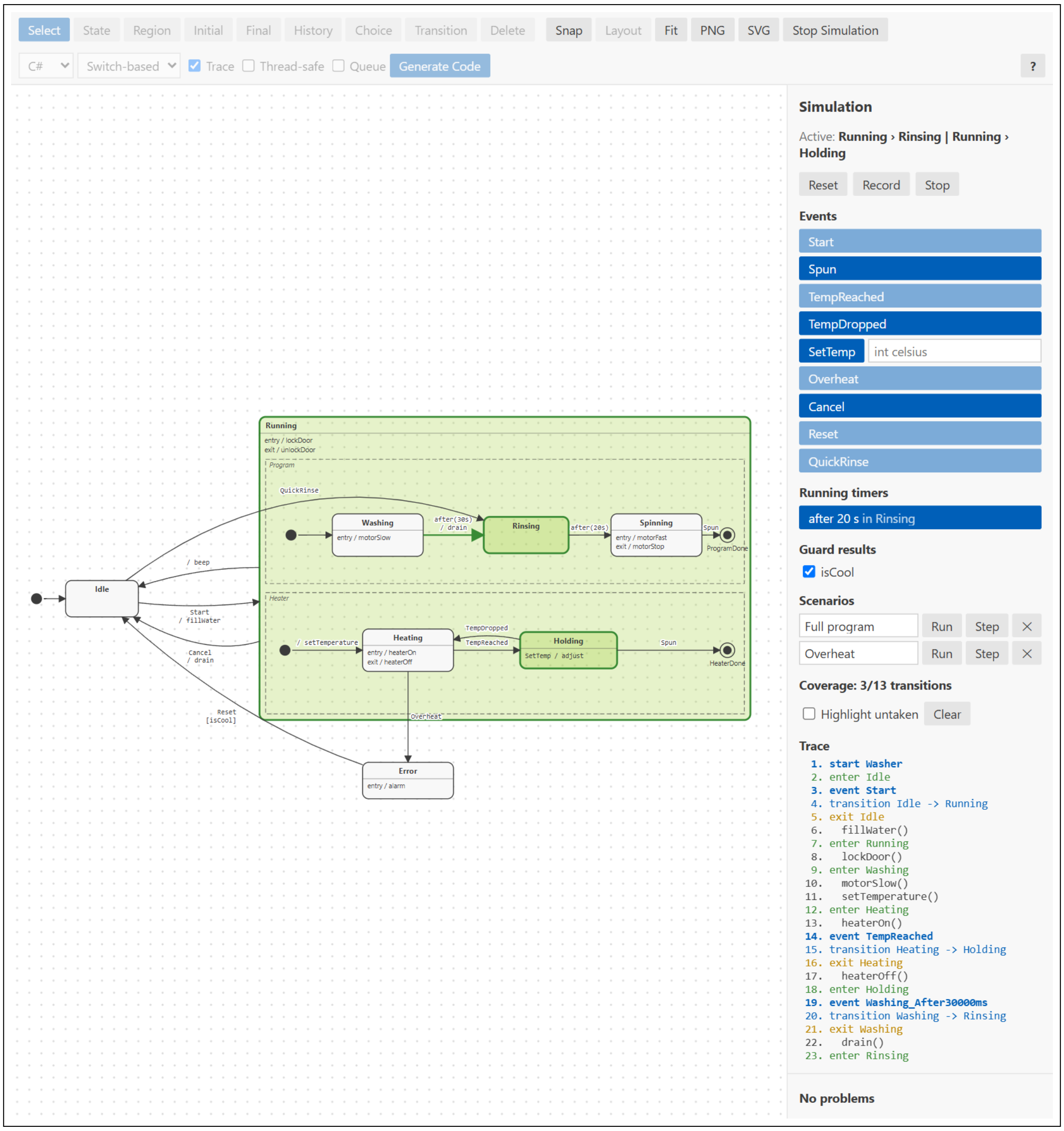
Add four more transitions:

- a **completion transition** `Running` → `Idle` with the action `beep`. An orthogonal state completes when *every* region has reached a final state, so it fires only after both `ProgramDone` and `HeaterDone`;
- `Running` → `Idle` on `Cancel / drain`: from the orthogonal state itself, it leaves both regions whatever their state;
- `Heating` → `Error` on `Overheat`: a transition from inside one region to a state outside `Running` also exits the other region;
- `Error` → `Idle` on `Reset [isCool]`, and `Idle` → `Rinsing` on `QuickRinse`. Entering `Running` through a substate of one region starts the other regions at their initial states (the regions are still entered in their order, `Program` first).

Transitions *between* the regions of one state are not allowed: the regions are independent by definition. History pseudostates are not available in states with several regions; a deep history pseudostate in an enclosing state does restore all of their regions (Section 10.7).

8.5 Step 5 — Simulate

Press **Simulate**, then fire `Start`, `TempReached` and the running timer **after 30 s** (Figure 8.3). **Active** now lists one path per region, `Running > Rinsing | Running > Holding`, and both active leaves are highlighted. The trace shows the order of entry: `Running` and its entry action first, then the regions in order — the `Program` region enters `Washing`, then the `Heater` region runs its initial action `setTemperature` and enters `Heating`. On the way out the order is reversed: the regions are exited last-to-first, each innermost-first, before `Running` itself.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Stop Simulation
Generate Code
?
Running
entry / lockDoor
exit / unlockDoor
Program
QuickRinse
Washing
entry / motorSlow
after(30s)
/ drain
Rinsing
after(20s)
Spinning
entry / motorFast
exit / motorStop
Spun
ProgramDone
/ beep
Idle
Start
/ fillWater
Cancel
/ drain
Heater
/ setTemperature
Heating
entry / heaterOn
exit / heaterOff
TempDropped
TempReached
Holding
SetTemp / adjust
Spun
HeaterDone
Overheat
Reset
[isCool]
Error
entry / alarm
Simulation
Reset
Record
Stop
Events
Start
Spun
TempReached
TempDropped
SetTemp
int celsius
Overheat
Cancel
Reset
QuickRinse
Running timers
after 20 s in Rinsing
Guard results
Scenarios
Run
Step
Run
Step
Clear
Trace

C# · Switch-based · ☑ Trace ☐ Thread-safe ☐ Queue

Active: **Running › Rinsing | Running › Holding**

☑ isCool

Full program

Overheat

Coverage: 3/13 transitions

☐ Highlight untaken

```
 1. start Washer
 2. enter Idle
 3. event Start
 4. transition Idle -> Running
 5. exit Idle
 6.   fillWater()
 7. enter Running
 8.   lockDoor()
 9. enter Washing
10.   motorSlow()
11.   setTemperature()
12. enter Heating
13.   heaterOn()
14. event TempReached
15. transition Heating -> Holding
16. exit Heating
17.   heaterOff()
18. enter Holding
19. event Washing_After30000ms
20. transition Washing -> Rinsing
21. exit Washing
22.   drain()
23. enter Rinsing
```

No problems

Figure 8.3: Both regions are active: `Rinsing` in `Program` and `Holding` in `Heater`. The 20-second timer of `Rinsing` is running.

Now fire `Spun`. Only the `Heater` region handles it — `Program` is still in `Rinsing`, which has no `Spun` transition — so the heater finishes in `HeaterDone` while the drum carries on. Let **after 20 s** elapse and fire `Spun` again: `Program` reaches `ProgramDone`, every region has now finished, and the completion transition returns to `Idle` with `beep`.

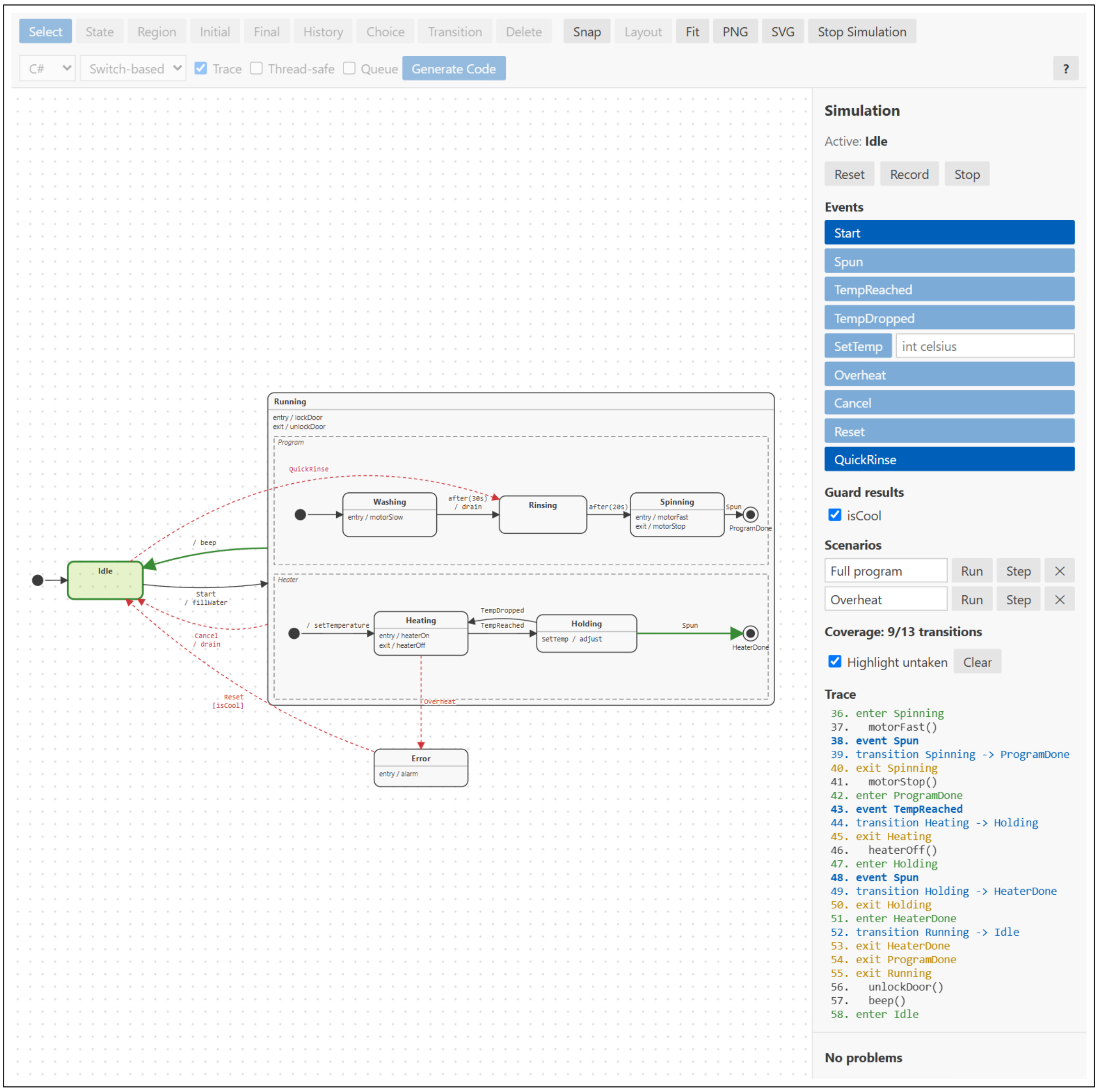
8.6 Step 6 — Record and replay scenarios

Clicking through the same sequence after every change to the diagram is tedious. Instead, record it:

1. Click **Record**. The simulation restarts and the button changes to **Save (0)**.
2. Fire the events of the test case — with the event arguments and guard results it needs. The counter on the button counts the recorded steps.

3. Click **Save**. The steps are stored in the diagram as a *scenario* named `Scenario 1`, together with the arguments and the guards that were unticked. Rename it in the **Scenarios** list; **Discard** stops recording without saving.

Run restarts the simulation and replays all steps of a scenario; **Step** restarts it and replays one step per click, so you can watch each transition. The example contains two scenarios, `Full program` and `Overheat`. Run `Full program` and tick **Highlight untaken** (Figure 8.4): 9 of the 13 transitions were taken. The red ones — `Cancel`, `QuickRinse`, `Overheat` and `Reset` — are exactly what the `Overheat` scenario covers apart from `Cancel`; run it as well and the coverage rises to 12/13. As an exercise, record a third scenario that cancels a running programme to reach 13/13. Coverage accumulates until you press **Clear** or stop the simulator.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Stop Simulation
C#
Switch-based
☑ Trace
☐ Thread-safe
☐ Queue
Generate Code
?
Running
entry / lockDoor
exit / unlockDoor
Program
QuickRinse
Washing
entry / motorSlow
after(30s)
/ drain
Rinsing
after(20s)
Spinning
entry / motorFast
exit / motorStop
Spun
ProgramDone
Heater
/ setTemperature
Heating
entry / heaterOn
exit / heaterOff
TempDropped
TempReached
Holding
SetTemp / adjust
Spun
HeaterDone
Overheat
Error
entry / alarm
Idle
/ beep
Start
/ fillWater
Cancel
/ drain
Reset
[isCool]
Reset
Record
Stop
Events
Start
Spun
TempReached
TempDropped
SetTemp
int celsius
Overheat
Cancel
Reset
QuickRinse
Run
Step
×
Run
Step
×
Clear

Simulation

Active: **Idle**

Guard results

☑ isCool

Scenarios

Full program

Overheat

Coverage: 9/13 transitions

☑ Highlight untaken

Trace

```
36. enter Spinning
37.   motorFast()
38. event Spun
39. transition Spinning -> ProgramDone
40. exit Spinning
41.   motorStop()
42. enter ProgramDone
43. event TempReached
44. transition Heating -> Holding
45. exit Heating
46.   heaterOff()
47. enter Holding
48. event Spun
49. transition Holding -> HeaterDone
50. exit Holding
51. enter HeaterDone
52. transition Running -> Idle
53. exit HeaterDone
54. exit ProgramDone
55. exit Running
56.   unlockDoor()
57.   beep()
58. enter Idle
```

No problems

Figure 8.4: After replaying `Full program`: the trace ends with both regions finishing and the completion transition to `Idle`. Untaken transitions are red.

Tip

Scenarios are saved in the `.machina` file, so they travel with the diagram through version control. After changing the diagram, replay the scenarios: a trace that changed unexpectedly, or a scenario that no longer reaches the end, points straight at the regression. The trace of a replayed scenario is the same trace the generated code produces with **Trace** enabled.

8.7 Step 7 — Generate

Select **C#**, tick **Trace** and generate. Regions change little in the interface:

- `IsIn(State.Heating)` tells whether a state is active in any region;
- `Current` follows the *first* region of an orthogonal state (`Program` here); use `IsIn` for the others;
- internally the class keeps one active state per region, and every hook runs in the order shown by the simulator trace.

The C++ and C generators produce the same structure, with fixed-size arrays and no allocation.

Part III

Reference

Chapter 9

Editor Reference

9.1 Toolbar

Control	Purpose
Select	Select, move and resize elements; drag transition labels to bend transitions.
State	Click on the canvas to add a state. Clicking inside an existing state nests the new state.
Region	Click inside a state to add an orthogonal region to it (Section 10.3).
Initial	Add an initial pseudostate (the black dot that marks a region's default state).
Final	Add a final state.
History	Add a history pseudostate inside a composite state (Section 10.7).
Choice	Add a choice point (Section 10.10).
Transition	Press on the source element, drag to the target and release. Dragging from a state onto itself creates a self transition.
Delete	Delete the selection. Deleting a state also deletes its contents and all connected transitions.
Snap	Toggle snapping of positions and sizes to a 10-pixel grid. The setting is remembered per editor. The dot grid shows the snap positions and pans and zooms with the diagram; zoomed out, only every 2nd, 5th or 10th position is drawn.
Layout	Arrange all states and transitions automatically (Section 9.8). This is a normal edit and can be undone.
Fit	Zoom and pan so that the whole diagram is visible.
PNG, **SVG**	Save the whole diagram as a PNG or SVG image (Section 9.7).
Simulate	Start or stop the simulator (Chapter 12).
Language	**C++**, **C#** or **C**.
Style	**Switch-based** or **Table-based** (Section 13.3).
Trace	Generate a `Trace` method (Section 13.7).
Thread-safe	Generate `Lock`/`Unlock` hooks around the public interface (Section 13.8).
Queue	Generate `Post` so hooks can queue events (Section 13.9).
Generate Code	Generate code with the current options (Chapter 13).
?	At the right end of the toolbar: show the help dialog (Section 9.11).

9.2 Mouse and keyboard

Gesture	Effect
Click an element	Selects it and shows its properties.
`Shift`/`Ctrl`+click a state	Adds it to (or removes it from) the selection.
`Shift`+drag on empty canvas	Selects all elements inside the rectangle.

Gesture	Effect
Drag a state	Moves the state — or all selected states — together with everything nested inside.
Drop a state inside another	Makes it a substate of the state under its centre.
Drag a state out of its parent	Moves it to the enclosing level.
Drag the corner handle	Resizes the selected state. Siblings that end up fully inside it become its children; children whose centre ends up outside it leave it and are nested where they now lie.
Drag a transition label	Bends the transition; near the straight line it snaps back to straight.
Drag a self loop or its label	Moves the loop round its state and makes it larger or smaller; its tip follows the pointer.
Double-click a state	Selects it and focuses the **Name** field.
Drag empty canvas / middle button	Pans the view.
Mouse wheel	Zooms around the mouse pointer.
`Delete` / `Backspace`	Deletes the selection.
`Ctrl`+`C` / `Ctrl`+`X` / `Ctrl`+`V`	Copies, cuts and pastes the selected states with their contents and the transitions between them — also into another diagram. Names that already exist get a number appended.
`Ctrl`+`D`	Duplicates the selection.
`Ctrl`+`A`	Selects all top-level elements.
Arrow keys	Nudge the selection by 1 pixel (10 with **Snap**); with `Shift` by 50 pixels. A burst of key presses is one undo step, and nudged states are nested where they end up, as after a drag.
`Esc`	Cancels the current tool and clears the selection.
`Ctrl`+`Z` / `Ctrl`+`Y`	Undo / redo (handled by VS Code).
`Ctrl`+`S`	Saves the diagram.
`?`	Shows or hides the help dialog (Section 9.11). While it is open, the other shortcuts do nothing.

9.3 Properties panel

Nothing selected

Class name (also the output file name), **Namespace** (`A.B` in C#, `A::B` in C++), and one **Event parameters** field per event.

State

Name, **Entry actions**, **Exit actions**, **Color**; the parent state is shown for information. A state containing other states is labelled *Composite state*.

Region

Name (free text, used in the diagram and in comments of the generated code) and **Color**; the orthogonal state it belongs to is shown for information.

Final state

Name and **Color**.

Initial pseudostate

The region it belongs to and **Color**.

History pseudostate

The composite state it belongs to, **Deep history (H*)** and **Color**.

Choice point

The region it belongs to and **Color**.

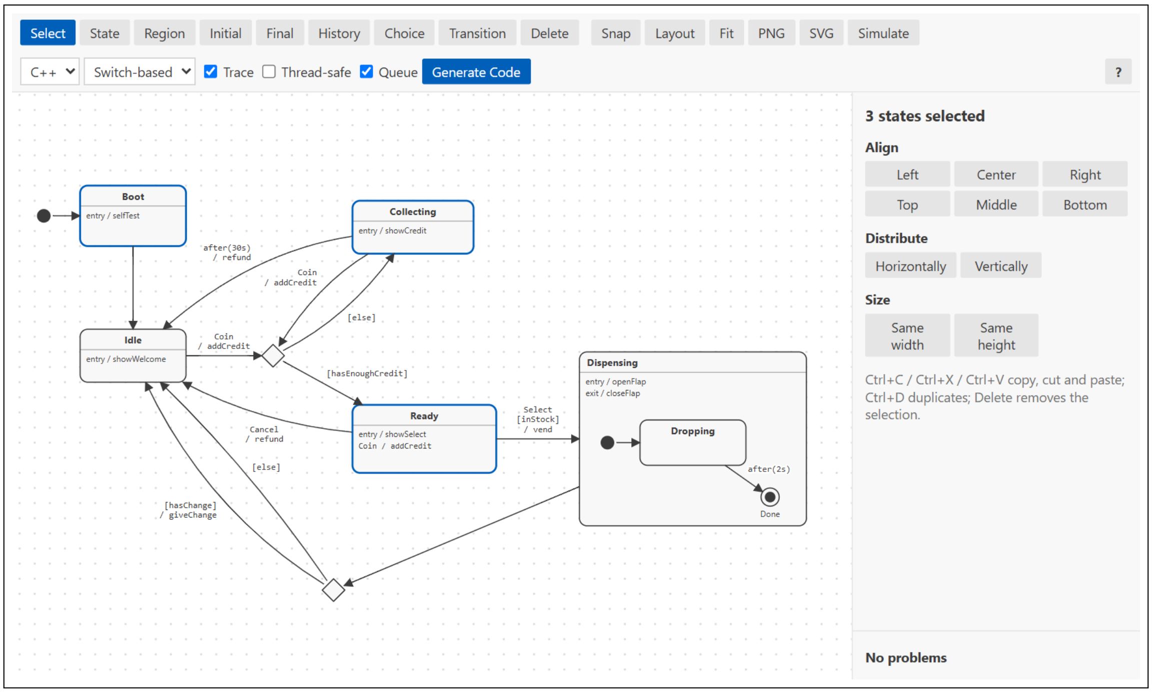
Transition

Event, **Guard**, **Action**, **Color** and **Straighten** (**Reset loop** for a self transition, which returns a dragged loop to its default place). The heading tells the kind: *Transition*, *Timeout transition*, *Completion transition*, *Internal transition* or *Choice branch*. Self transitions additionally offer **Internal (no exit / entry)**. Branches leaving a choice point have no event; their guard may be `else`. Transitions leaving an initial pseudostate only have an action.

Text fields are applied when you press `Enter` or move the focus elsewhere. Every change is a normal document edit: it marks the file as modified and can be undone with `Ctrl`+`Z`.

9.4 Working with several states

When two or more states are selected, the side panel offers alignment, distribution and sizing (Figure 9.1). The first selected state is the reference for **Left**, **Center**, **Right**, **Top**, **Middle** and **Bottom**; **Same width** and **Same height** use the first selected state that is not a pseudostate or region. Pseudostates and regions keep their size, and no state becomes smaller than 60 × 40 pixels. Clicking another member of the selection makes it the reference. **Horizontally** and **Vertically** spread three or more states with equal gaps between their outer ones. States nested in another selected state move with their parent.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Boot
entry / selfTest
Collecting
entry / showCredit
after(30s)
/ refund
Coin
/ addCredit
[else]
Idle
entry / showWelcome
Coin
/ addCredit
[hasEnoughCredit]
Dispensing
entry / openFlap
exit / closeFlap
Dropping
after(2s)
Done
Ready
entry / showSelect
Coin / addCredit
Select
[inStock]
/ vend
Cancel
/ refund
[else]
[hasChange]
/ giveChange
Align
Left
Center
Right
Top
Middle
Bottom
Distribute
Horizontally
Vertically
Size
Same width
Same height

C++ Switch-based ☑ Trace ☐ Thread-safe ☑ Queue

3 states selected

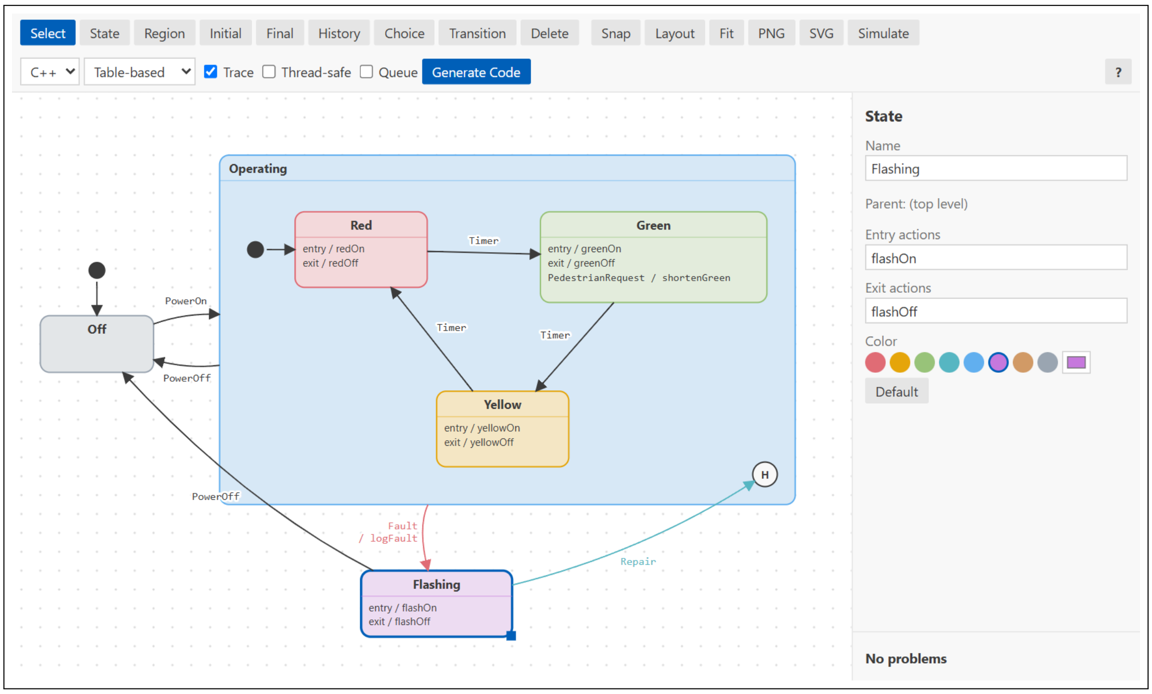
Ctrl+C / Ctrl+X / Ctrl+V copy, cut and paste; Ctrl+D duplicates; Delete removes the selection.

No problems

Figure 9.1: Three states selected; the side panel offers alignment and distribution.

9.5 Colours

Every element can be given one of eight preset colours or any custom colour. States get a coloured outline and a light tint, pseudostates are filled, and transitions colour their line, arrowhead and label. **Default** returns to the theme colour. Colours are purely visual.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Operating
Red
entry / redOn
exit / redOff
Timer
Green
entry / greenOn
exit / greenOff
PedestrianRequest / shortenGreen
Off
PowerOn
PowerOff
Timer
Timer
Yellow
entry / yellowOn
exit / yellowOff
H
PowerOff
Fault
/ logFault
Repair
Flashing
entry / flashOn
exit / flashOff
Color
Default

C++ | Table-based | ☑ Trace | ☐ Thread-safe | ☐ Queue

State

Name: Flashing

Parent: (top level)

Entry actions: flashOn

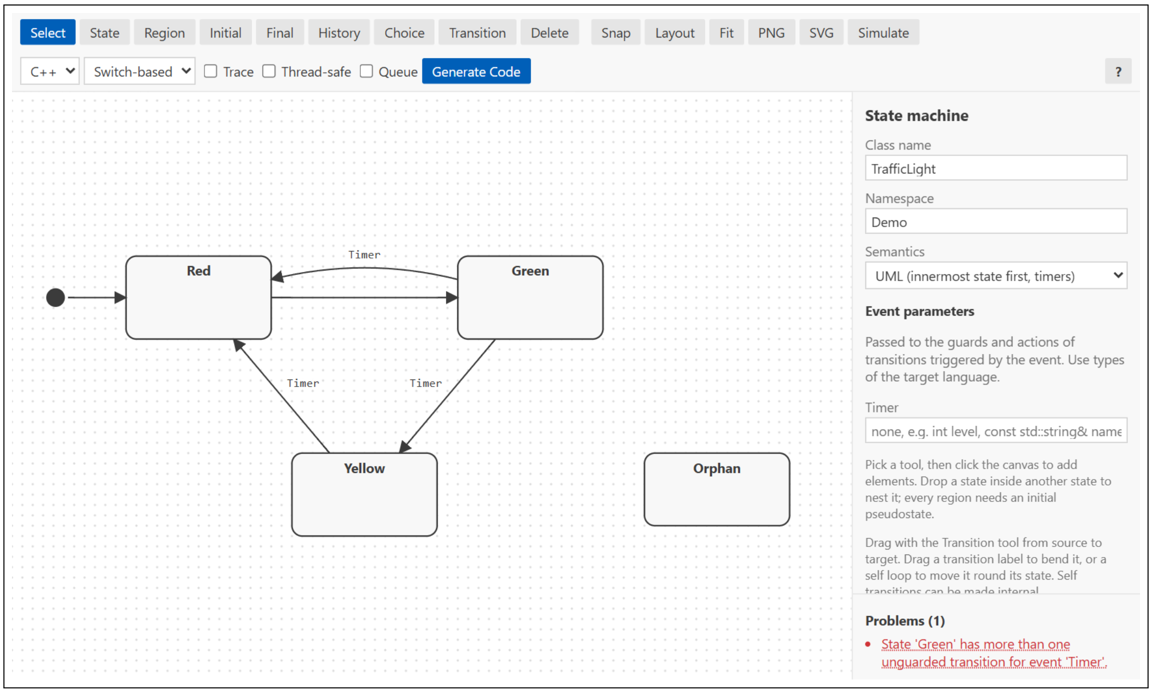
Exit actions: flashOff

No problems

Figure 9.2: Colours highlight the colours of the traffic light and the fault path.

9.6 Problems list

The problems list below the properties shows every violation of the rules in Chapter 15. It updates as you edit. Clicking a problem that refers to an element selects that element. Code can only be generated, and the diagram simulated, when the list contains no errors; warnings (for elements that have no effect, such as a choice point without an incoming transition) do not prevent either. The same problems are reported in VS Code's **Problems** panel (`Ctrl`+`Shift`+`M`), also for diagrams opened as JSON text, where they point at the line of the offending element.



State machine

Class name: TrafficLight

Namespace: Demo

Semantics: UML (innermost state first, timers)

Event parameters

Passed to the guards and actions of transitions triggered by the event. Use types of the target language.

Timer: none, e.g. int level, const std::string& name

Pick a tool, then click the canvas to add elements. Drop a state inside another state to nest it; every region needs an initial pseudostate.

Drag with the Transition tool from source to target. Drag a transition label to bend it, or a self loop to move it round its state. Self transitions can be made internal

Problems (1)

- State 'Green' has more than one unguarded transition for event 'Timer'.

Figure 9.3: A transition without an event, and two unguarded `Timer` transitions leaving `Green`. `Orphan` is not a problem: unreachable states are allowed.

9.7 Exporting images

PNG renders the complete diagram (not only the visible part) at twice the screen resolution with a 20-pixel margin, using the current theme's background colour. **SVG** produces a scalable vector image with a transparent background, ideal for documentation. In both, the selection highlight and the simulation highlights (active states, taken and untaken transitions) are always omitted, even while the simulator is running, and the theme's colours are baked into the image. A save dialog proposes `<ClassName>.png` or `.svg` next to the diagram. For documents, export with a light theme.

9.8 Automatic layout

Layout in the toolbar, or **MachinaState: Auto-Layout Diagram** in the Command Palette, arranges the diagram with the Eclipse Layout Kernel (ELK). Each container — the top level, every composite state and every region — is laid out on its own, innermost first, so that the hierarchy is preserved and containers grow to fit their contents. Each container is arranged three ways — as a layered flow from left to right, as a layered flow from top to bottom, and organically, spread in both directions with the initial state towards the top left — and the arrangement that fits a screen-shaped area most compactly is kept, so that a diagram does not turn into a long strip. Simple states are resized to fit their name and actions, and transition bends are reset. Imported diagrams are laid out this way automatically. The layout runs inside VS Code and does not need a network connection.

9.9 Commands

All commands are available from the Command Palette (Ctrl+Shift+P); some also appear in the context menu of a file in the Explorer.

Command	Description
New State Machine Diagram	Creates a `.machina` file with an initial pseudostate and one state.
Generate State Machine Code	Generates code for the active diagram, or the `.machina` file clicked in the Explorer (Chapter 13).
Auto-Layout Diagram	Arranges the active diagram (Section 9.8).
Export Diagram as PlantUML	Writes a `.puml` file (Chapter 14).
Export Diagram as SCXML	Writes a `.scxml` file (Chapter 14).
Export Diagram as Mermaid	Writes a `.mmd` file, or a `.md` file with a ` ```mermaid ` block (Section 14.2).
Copy Diagram as Mermaid (Markdown)	Copies the diagram as a ` ```mermaid ` block for pasting into Markdown (Section 14.2).
Import PlantUML, SCXML or Mermaid State Machine	Converts a `.puml`, `.scxml`, `.mmd` or Markdown file into a new `.machina` diagram. Also in the Explorer context menu of such files (except Markdown).
Open User Manual	Opens this manual, for the installed version, in the web browser (Section 9.11).
Get Started	Opens the *Get Started* walkthrough (Section 9.11).

9.10 Settings

Setting	Description
`machinaState.generateOnSave`	Default `false`. When enabled, saving a `.machina` file regenerates its code with the language and style stored in the diagram. Nothing is written if the diagram has problems or the code is unchanged.
`machinaState.stateflow.libraryPath`	Folders searched for the Simulink libraries that an imported Stateflow model links to (Section 14.4).
`machinaState.stateflow.parameterFiles`	MATLAB scripts with the values of the parameters of imported Stateflow charts. When empty, the import asks.

9.11 Getting help

Help is available in three places inside VS Code:

Help dialog

Press **?** at the right end of the toolbar, or the ? key on the canvas. The dialog lists the mouse gestures and keyboard shortcuts of this chapter, summarises the transition notation (`Event [guard] / action`, `after(...)` timeouts, completion and internal transitions, history) and has buttons for the manual and the walkthrough. Close it with Esc, ?, its ✕ button or a click outside it. While it is open, keys and clipboard shortcuts do not change the diagram.

Get Started walkthrough

VS Code's **Welcome** page lists **Get Started with MachinaState**, which can also be opened with **MachinaState: Get Started**. Its five steps — create a diagram, add states and transitions, simulate, generate code, learn more — are Tutorial 1 in brief. Steps with a button are ticked off when the button's command runs, the others when you select them.

User manual

MachinaState: Open User Manual, also in the **...** menu of the diagram editor's title bar, opens this manual in the web browser: the copy that matches the installed version, so the manual and the extension describe the same features.

9.12 Working with the JSON source

Use **Reopen Editor With...** → **Text Editor** to edit the JSON directly; both views of the same file stay synchronised. If the JSON is invalid, the diagram editor shows an error message until it is fixed. The format is described in Chapter 16.

Chapter 10

Modelling Reference

10.1 States

A state has a name that is unique in the whole diagram and optional *entry* and *exit* actions. Entry actions run whenever the state is entered, exit actions whenever it is left — including when a transition leaves and re-enters the same state.

10.2 Composite states and regions

A state that contains other states is a *composite state*. While a substate is active, all of its ancestors are active too: the *active configuration* is a chain from a top-level state down to a leaf. The top level and every composite state form a *region* that needs exactly one initial pseudostate. Nesting depth is unlimited. A composite state can also be divided into several *orthogonal* regions that are active at the same time (Section 10.3).

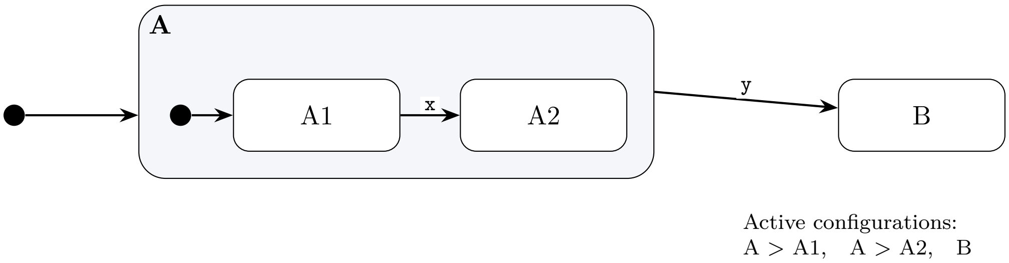


Figure 10.1: A composite state `A` with two substates. Event `y` leaves `A` from either substate.

10.3 Orthogonal regions

A state can be divided into two or more *regions*, drawn as dashed rectangles inside it. While such an *orthogonal state* is active, every one of its regions has exactly one active substate, so the active configuration is a tree rather than a chain (Tutorial 5).

- Each region needs its own initial pseudostate. Once a state has regions, all of its substates must lie inside one of them. A state with a single region behaves like an ordinary composite state.
- Entering the orthogonal state enters all of its regions, in diagram order. A transition that targets a substate of one region enters that region at the target and the other regions at their initial states, still in diagram order.
- An event is offered to every active region; several regions can handle the same event in one step. The orthogonal state itself only sees the event if none of its regions handled it. Once a transition in one region has left the orthogonal state, the remaining regions are not asked, even if the transition entered the state again.
- A transition leaving the orthogonal state — from the state itself, or from a substate of one region to a state outside — exits all regions, last region first, before the state's own exit

actions run.

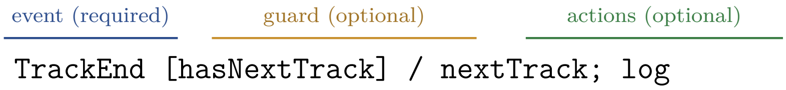
- The orthogonal state *completes* when every region has reached a final state; then its completion transitions can fire.
- Transitions between different regions of the same state are not allowed, regions cannot be the source or target of transitions, and history pseudostates cannot be used in a state with several regions (a state with a single region can have one, inside that region).

10.4 Initial and final states

An *initial pseudostate* has exactly one outgoing transition, without event or guard, to a state inside its region (possibly nested deeper). That transition may have actions, which run before the default state is entered.

A *final state* has no outgoing transitions and no children. Reaching a top-level final state means the machine has finished (`IsFinished` becomes true). A final state inside a composite state is an ordinary leaf; the transitions of its enclosing states remain active.

10.5 Transitions



`TrackEnd [hasNextTrack] / nextTrack; log`

Figure 10.2: Anatomy of a transition label.

On an arrow, the editor shows each part on a line of its own — the event, then `[guard]`, then `/ actions` — beside the arrow, on the outer side of a curve. Internal transitions, listed inside their state, keep the one-line form.

- **Event** — one of:
    - an identifier; all such events form the `Event` enumeration;
    - a timeout `after(`*n*`ms|s|min)` (Section 10.9);
    - empty, for a completion transition (Section 10.8).
- **Guard** — the name of a Boolean method, optionally negated: `isReady`, `!isBusy`. The transition fires only if the guard returns true.
- **Actions** — method names separated by `;` or `,`, run in order.

A transition can start at a composite state; it then applies to all of its substates. A transition can end at a composite state; the composite is then entered through its initial pseudostate. A self transition exits and re-enters its state.

Note

Guards and actions are *names*, not code. This keeps the diagram language-neutral and keeps your logic in normal source files, where it can be debugged and unit-tested.

10.6 Internal transitions

An internal transition is a self transition that runs only its actions — no exit actions, no entry actions, and the active configuration does not change. On a composite state, the active substates stay active. Create a self transition and tick **Internal (no exit / entry)**; it is then listed inside the state.

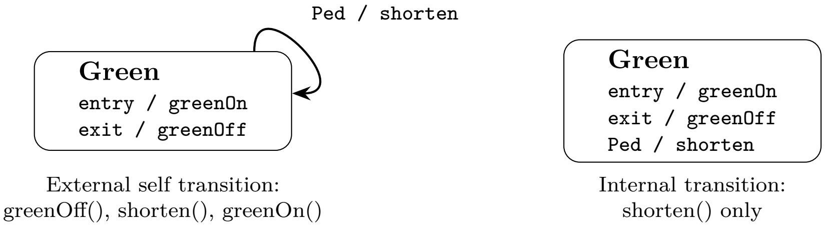

Ped / shorten
Green
entry / greenOn
exit / greenOff
External self transition:
greenOff(), shorten(), greenOn()
Green
entry / greenOn
exit / greenOff
Ped / shorten
Internal transition:
shorten() only

Figure 10.3: External self transition versus internal transition.

10.7 History

A history pseudostate inside a composite state C records the configuration of C every time C is exited. A transition that targets the history pseudostate enters C and restores it:

Shallow history (H)
: restores the last active *direct* substate of C; if that substate is itself composite, it is entered through its initial pseudostate.

Deep history (H*)
: restores the whole configuration below C: the last active substate at every level, entering every state on the way. Below an orthogonal state, every region is restored.

If C has not been exited since `Start`, there is no history and C is entered through its initial pseudostate (`Start` forgets all history). Transitions that target C itself always use the initial pseudostate.

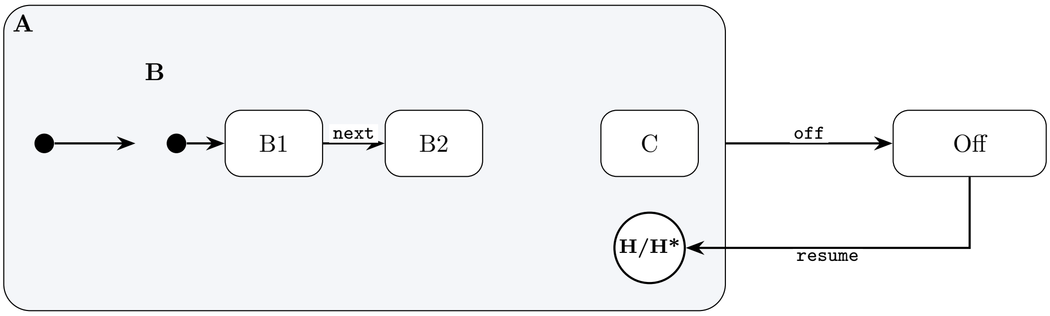

A
B
B1
next
B2
C
off
Off
H/H*
resume

Figure 10.4: Leave `A` from `B2` with `off`, then `resume`: shallow history restores `B` and enters `B1` (its default); deep history restores `B2`.

A composite state can have at most one history pseudostate, and history pseudostates cannot have outgoing transitions. A state with several orthogonal regions cannot have one; a state with a single explicit region can, placed inside that region.

10.8 Completion transitions

A transition without an event is a *completion transition*. It fires automatically when its source state *completes*:

- a simple state completes as soon as it has been entered (after its entry actions);
- a composite state completes when its region reaches a final state.

Completion transitions are processed before `Start` or `Dispatch` returns, before any queued event. They may have guards; if no guard is true the completion is discarded and the state simply stays active. Typical uses are initialisation states (`Boot` in Tutorial 4) and "when finished, continue" for composite states.

Completion transitions that lead back to a state that completes again immediately would fire for ever. A cycle made only of unguarded completion transitions is therefore reported as an error. A cycle that contains a guard is allowed (it ends when the guard becomes false), but in the generated code it keeps `Dispatch` busy for as long as the guard holds; the simulator's engine gives up after 1000 completion transitions in one step.

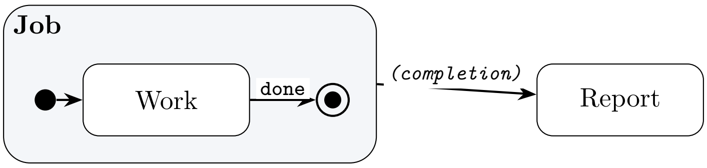


Figure 10.5: Reaching the final state completes `Job`; its event-less transition then fires.

10.9 Timeouts

An event written as `after(500ms)`, `after(2s)` or `after(1min)` (no unit means milliseconds) is a *timeout*. Each distinct timeout of a state becomes an event with a generated name, *State*`_After`*n*`ms`, e.g. `Collecting_After30000ms`. The generated class calls two hooks that you implement with your platform's timers:

- `StartTimer(timeout, milliseconds, generation)` after the state has been entered (after its entry actions);
- `CancelTimer(timeout)` when the state is exited (before its exit actions).

When a timer elapses, call `DispatchTimeout(timeout, generation)` with the generation number that `StartTimer` received. Every start of a timer gets a new generation, and the call is ignored (returning `false`) if that timer has been cancelled or restarted since. A late timer is therefore harmless even when its state has been left and re-entered in the meantime — it cannot cut the new waiting time short. A timer fires once: after its timeout has been dispatched, a repeated call with the same generation is ignored too, even if the timeout's guard was false or its transition internal. An external self transition restarts the timer; an internal transition does not. Timeouts can have guards and actions like any other transition.

10.10 Choice points

A choice point (diamond) splits a transition into alternative branches. The incoming transition carries the event (and optionally a guard and actions); each outgoing branch carries a guard and optionally actions. One branch may have the guard `else` (or no guard), which is taken when no other guard is true. Branches can lead to states, history pseudostates or further choice points.

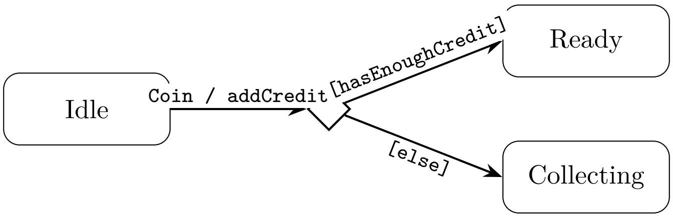


Figure 10.6: A choice point with an `[else]` branch.

MachinaState evaluates all guards on the path *before* the transition starts: first the guard of the incoming transition, then the branch guards in order (`else` last). Only when a complete path is found does it exit the source states, run the actions of all segments (incoming first), and enter the target. If no branch is enabled, nothing happens and the transition counts as not enabled, so the event continues to the next candidate (an outer state, for example). In UML terms this is the semantics of a *junction*; it guarantees that a transition never stops half-way.

Consequently a branch guard cannot see the effect of the incoming transition's actions — in Tutorial 4, `hasEnoughCredit` adds the incoming coin to the credit itself. Branch guards and actions receive the parameters of the triggering event.

10.11 Event parameters

Declare parameters per event with nothing selected, as a comma-separated list such as `int level` or `const std::string& name, int retries`. Each entry must be written as `type name`; default values are ignored. The parameters are passed to the guards and actions of every transition triggered by that event. Consequently:

- a hook used with several events must receive the same parameter list everywhere;
- entry, exit and initial actions never receive parameters.

In the generated code, each event with parameters gets a typed dispatch method, for example `DispatchVolume(int level)`.

10.12 Supported and unsupported UML features

UML feature	**Status**
Simple and composite states, arbitrary nesting	supported
Initial, final, shallow and deep history pseudostates	supported
Entry and exit actions, transition actions	supported
Guards (single, optionally negated, method name)	supported
Internal transitions	supported
Event parameters	supported
Completion (event-less) transitions	supported
Time events (`after`)	supported, through timer hooks
Choice pseudostates	supported, guards evaluated before the transition starts (junction semantics)
Orthogonal (parallel) regions	supported, without history in orthogonal states
Time events (`at`), `do` activities, deferred events	not supported

Chapter 11

Execution Semantics

All six generated variants (C++, C# and C, each switch-based and table-based) and the simulator implement exactly the semantics in this chapter. The simulator runs a reference engine, and the test suite checks that it and all generated variants produce identical traces.

11.1 Starting

`Start()` first resets the machine: it cancels every running timer (calling `CancelTimer`, and giving it a new generation so that a `DispatchTimeout` still on its way is ignored), forgets all history, discards pending completions and queued events, and sets the active state to *none*. Start is a reset, not a transition: states that were active before are not exited, so their exit actions do not run. Then it runs the actions of the top-level initial transition and enters its target. Entering a composite state continues with its initial transition, until a leaf state is reached; entering an orthogonal state enters each of its regions in turn. Completion transitions and queued events are then processed exactly as after a dispatched event.

11.2 Dispatching an event

`Dispatch` processes one event *to completion* before returning:

Listing 11.1: Dispatch algorithm (pseudocode).

```
handled = false
for (s = innermost active state; s != none && !handled; s = parent(s))
    for (t in transitions leaving s for this event,
              those that may not fire first, in diagram order)
        if (guard(t) holds && (t has no choice point || a branch path is enabled))
            fire(t); handled = true; break
if (handled)
    while (a state has completed)         // run to completion
        fire its enabled completion transition, if any
drain the event queue (each queued event is dispatched like this one)
return handled
```

Inner states therefore take priority over outer ones, and within a state guarded transitions are checked before the unguarded one (Figure 11.1). With orthogonal regions, “the innermost active state” is one state per region: the event is offered to each active region in diagram order (each region applying the loop above), and bubbles up to the orthogonal state only if no region handled it. If a transition in one region leaves the orthogonal state, the remaining regions are not asked — even when the transition enters the orthogonal state again (a self transition of the orthogonal state, for example), so that its freshly entered regions do not see the same event a second time.

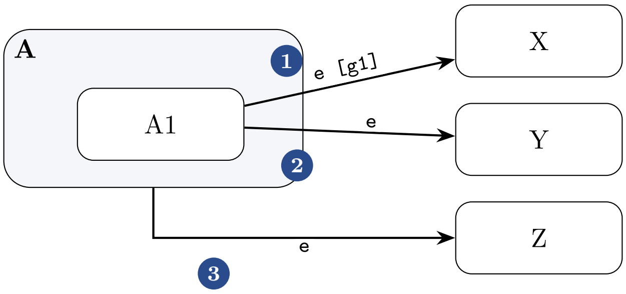


Figure 11.1: Evaluation order for event `e` while `A1` is active: `A1`'s guarded transition, then `A1`'s unguarded one. `A`'s transition ③ is only considered if `A1` has no enabled transition for `e`.

11.3 Firing a transition

For an external transition from source S to target T:

1. Determine the *least common ancestor* (LCA): the innermost state that strictly contains both S and T, or the top level.
2. Exit all active states below the LCA, innermost first, running exit actions (and recording history of composite states being exited).
3. Run the transition's actions.
4. Enter the states from below the LCA down to T, outermost first, running entry actions.
5. While the entered state is composite, follow its initial transition (running its actions).

An internal transition only runs its actions. A transition to a history pseudostate of C uses C as the target for the LCA, but restores the recorded configuration of C if there is one (Section 10.7). A transition through choice points first selects a complete path (Section 10.10), then performs steps 1–5 with the final target and the actions of all segments in order. Exiting a state with timeouts cancels its timers; entering it starts them. Entering a leaf state that has completion transitions — or a final state whose parent has them — marks that state as completed.

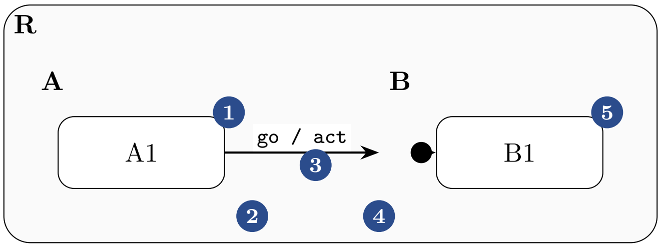


Figure 11.2: Transition `go / act` from `A1` to `B`. The LCA is `R`, so the order is: ① exit `A1`, ② exit `A`, ③ `act()`, ④ enter `B`, ⑤ enter `B1` via `B`'s initial transition. `R` stays active.

11.4 Orthogonal regions

For a state O with regions $R_1, \ldots, R_n$:

- **Entering** O runs its entry actions, then enters $R_1, \ldots, R_n$ in order. A region that contains the transition's target is entered along the path to the target; every other region follows its initial transition. The order does not depend on where the target is: with a target in R_2, R_1 is entered (by default) before R_2.

- **Dispatching** offers the event to $R_1, \ldots, R_n$ in order and stops as soon as a transition taken in one region has exited O, whether or not it re-entered O.
- **Exiting** O exits $R_n, \ldots, R_1$ in reverse order — each one innermost state first — then cancels O's timers and runs its exit actions.
- A final state reached in a region marks O as **completed** once every region's active state is final. The completion is processed after the current step, like any other.

11.5 Worked example

The table traces the hierarchical traffic light from Tutorial 2 (compare with the program output in Section 7 of that tutorial).

Event	Active before	Effect
`Start()`	—	enter `Off`
`PowerOn`	`Off`	exit `Off`; enter `Operating`; enter `Red`, `redOn()`
`Timer`	`Operating>Red`	handled by `Red`: exit `Red`, `redOff()`; enter `Green`, `greenOn()`
`PedestrianRequest`	`Operating>Green`	internal: `shortenGreen()`
`Fault`	`Operating>Green`	not handled by `Green`, bubbles to `Operating`: exit `Green`, `greenOff()`; exit `Operating` (history := `Green`); `logFault()`; enter `Flashing`, `flashOn()`
`Repair`	`Flashing`	exit `Flashing`, `flashOff()`; enter `Operating`; resume `Green`, `greenOn()`
`Timer`	`Operating>Green`	exit `Green`; enter `Yellow`

11.6 Stateflow semantics

A diagram can instead use the execution semantics of MATLAB Stateflow for charts that are woken periodically. Choose **Semantics: Stateflow** in the properties of the state machine (nothing selected), or import a Stateflow chart (Section 14.4). The file then records `"semantics": "stateflow"` and a step event (Chapter 16). Everything above applies, with these differences:

- **Step event.** The application dispatches the step event (default `Step`) once per period, as Simulink wakes the chart once per sample time. Conditions that Stateflow checks at every wake-up are transitions on the step event, such as `Step [isFull]`.
- **Outer state first.** An event is offered to the outermost active state first. Only if none of that state's transitions fires is the event passed on to the active state of each of its regions. In UML semantics the innermost state has priority instead (Section 11.2).
- **During actions.** A state may have **During actions**. They run on every dispatched event while the state is active and none of its own transitions fired, before the event is passed to its substates. During actions do not count as handling the event.
- **after() counts steps.** `after(n steps)` is enabled once its source state has been active for n step events. A time such as `after(500ms)` is converted with the **Step period** and rounded up to whole steps. The step during which a state is entered does not count. There are no timers: `StartTimer`, `CancelTimer` and `DispatchTimeout` are not generated, and a temporal transition is tried in its place among the state's other step transitions.

Per step, an active state therefore does this, from the outermost state down: try its own transitions for the event (in priority order); if one fires, stop; otherwise run its during actions and continue with its substates. Completion transitions, choice points, history and orthogonal regions behave as described above. The generated code, the simulator and the reference engine implement these semantics identically, and the test suite checks this for every language and style.

11.7 Rules for hooks

Guards and actions run *during* `Dispatch`. They must not call `Dispatch` (or `Start`) recursively: the machine is in the middle of a transition. If an action needs to raise a follow-up event, enable **Queue** and call `Post` (Section 13.9). Guards should be free of side effects; they may be evaluated without their transition firing.

Chapter 12

Simulator

Click **Simulate** to start. The simulator runs the diagram on the same reference engine that the test suite compares with every generated variant, so a step in the simulator does exactly what `Dispatch` does in the generated code: choice points, completion transitions and queued events are all processed within the step. The machine is started, editing is disabled and the side panel changes to the simulation view:

Active
the active leaf states with their ancestors, e.g. `Operating > Green`; with orthogonal regions one path per region, separated by `|`. “State machine finished” appears once a top-level final state is reached.

Reset / Record / Stop
restart the machine, start recording a scenario, or leave the simulator.

Events
one button per event. Events that no active state handles right now are shown dimmed; firing one is still allowed and logs that it was ignored. Events with parameters have an argument field: enter the values separated by commas (numbers, `true`/`false`, or text).

Running timers
one button per running timer, e.g. **after 2 s in Dropping**. Clicking it lets the timer elapse, like `DispatchTimeout` with the current generation. The timer then disappears from the list (timers are one-shot) unless the transition re-enters its state and restarts it.

Guard results
one check box per guard. The simulator cannot run your code, so you decide what each guard returns; a ticked box means `true`. Negated guards (`!isBusy`) use the same box. The trace shows each guard call with its arguments and result.

Scenarios
the scenarios saved in the diagram, with **Run**, **Step** and × (delete) (Section 12.1).

Coverage
how many transitions and choice branches have been taken since the simulator was started, out of all that can be taken (initial transitions are not counted). **Highlight untaken** draws the others dashed and in red, also when they have their own colour; **Clear** resets the count.

Trace
every event, transition, exit, action (with arguments) and entry in execution order — the same lines the generated `Trace` hook reports.

Active states are drawn in green and the transitions taken in the last step are highlighted. Pan and zoom remain available while simulating. If the diagram has errors, the panel lists them instead. If completion transitions keep firing without settling (more than 1000 in one step, see Section 10.8), the simulator stops, reports a probable endless loop in the trace and refuses further events until **Reset**.

12.1 Scenarios

A *scenario* is a named sequence of steps saved in the diagram. Each step is an event, with its arguments and the guards that were *false* when it was fired.

- **Record** restarts the simulation and records every event you fire. **Save (*n*)** stores the recording as `Scenario n`; **Discard** drops it.
- **Run** restarts the simulation and replays all steps, setting the guard results recorded for each step. **Step** restarts it and replays one step per click.
- The name field renames a scenario; **×** deletes it. Both are document edits and can be undone.

Because a replay sets the guard results itself, a scenario always takes the same path through an unchanged diagram. After a change, replaying the scenarios and comparing the trace is a quick regression test; a step that is now ignored shows up as `ignored` in the trace. A replayed timeout whose timer is not running at that point (its state was left, or the timer already elapsed) is not dispatched; the trace marks it as a stale timeout.

Tip

Use the simulator as an executable specification: walk through each requirement and compare the trace with what you expect. A trace that is surprising in the simulator will be equally surprising in the generated code.

Chapter 13

Code Generation

13.1 Generating

Choose language, style and options in the toolbar and click **Generate Code**. Alternatively run **MachinaState: Generate State Machine Code** from the Command Palette, the editor title bar or the context menu of a `.machina` file in the Explorer; these use the language and style stored in the diagram and ask only if none are stored. The file `<ClassName>.hpp` or `<ClassName>.cs` — for C the pair `<ClassName>.h` and `<ClassName>.c` — is written next to the diagram (overwriting a previous version) and opened beside the editor. A diagram that has never been saved asks for an output folder instead, and confirms before overwriting existing files.

Caution

Never edit the generated file — it is overwritten on every generation. Put your code in a subclass. Generated files can be checked into version control or generated as part of your build.

13.2 Generated interface

Member	Description
`enum State`	All states in pre-order (parents before children), plus `None`.
`enum Event`	All events used by transitions.
`Start()`	Resets the machine (cancels running timers, forgets history) and enters the initial configuration through the top-level initial transition.
`Dispatch(Event)`	Processes one event; returns `true` if a transition fired. Events with parameters are rejected (`false`).
`Dispatch`*`Event`*`(...)`	Typed method per event with parameters, e.g. `DispatchEnterCode(int code)`.
`Completion` event	Present when the diagram has completion transitions; used internally and rejected by `Dispatch`.
State`_After`*n*`ms` events	One per timeout, reported by `StartTimer`.
`DispatchTimeout(Event, generation)`	Generated with timeouts; call it when a timer elapses (Section 13.10). Each timer fires at most once per start.
`Current`	Innermost active state, following the first region of orthogonal states (method in C++, property in C#).
`IsIn(State)`	True if the state is active, also as an ancestor of an active state or in any region.
`IsFinished`	True when a top-level final state is active (method in C++, property in C#).
`ParentOf(State)`	Static; the enclosing state or `None`.
`StateName`, `EventName`	Static, C++ only; use `ToString()` in C#.
`kStateCount`, `kEventCount`	C++ only; number of states and events.

Member	Description
Guards and actions	Pure virtual (C++) / `protected abstract` (C#) hooks.
`Trace(TraceKind, name)`	Optional, virtual with empty default (Section 13.7).
`Lock()`, `Unlock()`	Optional, abstract (Section 13.8).
`StartTimer`, `CancelTimer`	Abstract; generated when the diagram has timeouts (Section 13.10).
`Post(Event)`, `Post`*`Event`*`(...)`	Optional, protected; queue an event from a hook (Section 13.9).
`kQueueCapacity`	With **Queue**; size of the queue (`codegen.queueCapacity`, default 16). `QueueCapacity` in C#, `..._QueueCapacity` in C.

13.3 Switch-based versus table-based

Both styles have the same interface and identical behaviour.

	Switch-based	Table-based
Event handling	Nested `switch` over state and event	Loop over a constant transition table
Hierarchy	`ParentOf` as a `switch`	Parent lookup array
Entry/exit	`switch` in `OnEntry`/`OnExit`	Arrays of member-function pointers (C++) or delegates (C#)
Code size	Grows with the number of transitions	Mostly data; small fixed code
Debugging	Step through readable control flow	Inspect table rows
Best for	Small to medium machines, readability	Large machines, data-driven inspection

Listing 13.1: Switch-based: event handling for the flat traffic light.

```
bool HandleEvent(State state, Event event) {
    switch (state) {
        case State::Red:
            switch (event) {
                case Event::Timer:
                    ExitState(State::Red);
                    EnterDown(State::None, State::Green);
                    return true;
                default:
                    break;
            }
            break;
        case State::Green:
            switch (event) {
                case Event::Timer:
                    ExitState(State::Green);
                    EnterDown(State::None, State::Yellow);
                    return true;
                default:
                    break;
            }
            break;
        case State::Yellow:
            switch (event) {
                case Event::Timer:
                    ExitState(State::Yellow);
                    EnterDown(State::None, State::Red);
                    return true;
                default:
                    break;
            }
            break;
        default:
            break;
    }
    return false;
}
```

Listing 13.2: Table-based: the same transitions as data.

```
// source, event, guard, negate guard, action, target, least common ancestor
inline const TrafficLight::TransitionRow TrafficLight::kTransitions[3] = {
    { State::Red, Event::Timer, nullptr, false, nullptr, State::Green, State::None },
    { State::Green, Event::Timer, nullptr, false, nullptr, State::Yellow, State::None },
    { State::Yellow, Event::Timer, nullptr, false, nullptr, State::Red, State::None },
};
```

13.4 C++ specifics

- Single self-contained header, C++17 (uses `inline` variables for the tables). No heap allocation, no exceptions, no RTTI — suitable for embedded targets.
- Hooks are pure virtual member functions; derive and `override` them.
- Event arguments are passed through a pointer to a stack-allocated argument struct that is valid only during the call, so reference parameters such as `const std::string&` are never copied.
- Namespaces written with dots are converted to nested C++ namespaces.

13.5 C specifics

The C target generates a header and a source file in C99, for compilers and code bases that cannot use C++. Everything is prefixed with the namespace and class name (`Demo_TrafficLight_...`) to avoid clashes.

- The machine is a plain `struct` that you allocate (statically, on the stack or inside your own structure); the generated code never allocates memory. Treat its members as private.
- Guards, actions and the optional services (`Trace`, `Lock`/`Unlock`, `StartTimer`/`CancelTimer`) are function pointers in a `..._Hooks` structure. Each receives the `void* context` passed to `..._Init`, followed by the event parameters. All members must be set, except `Trace`, which may be `NULL`.
- The public functions mirror the C++ interface: `..._Init`, `..._Start`, `..._Dispatch`, `..._Dispatch`*`Event`*`(machine, args...)`, `..._DispatchTimeout`, `..._Current`, `..._IsIn`, `..._IsFinished`, `..._ParentOf`, `..._StateName`, `..._EventName`. With **Queue**, hooks can call `..._Post` and the typed `..._Post`*`Event`* functions; the queue is a fixed array of `codegen.queueCapacity` entries.
- Switch-based and table-based styles are available as in C++; the table-based variant uses constant arrays of function pointers.

The traffic light of Tutorial 1, generated as C with **Trace**, is driven like this:

Listing 13.3: `traffic_flat.c`, compiled with `gcc -std=c99 -Wall -Wextra -Werror -pedantic` together with the generated `TrafficLight.c`.

```c
/* The Tutorial 1 traffic light generated as C99 (switch-based, with Trace). */
#include <stdio.h>

#include "TrafficLight.h"

/* Application data reached through the context pointer. */
typedef struct {
    int switches;
} Lamps;

static void Show(void* context, const char* text) {
    Lamps* lamps = (Lamps*)context;
    lamps->switches++;
    printf("  [%s]\n", text);
}

static void redOn(void* c)     { Show(c, "RED    on "); }
static void redOff(void* c)    { Show(c, "RED    off"); }
static void yellowOn(void* c)  { Show(c, "YELLOW on "); }
static void yellowOff(void* c) { Show(c, "YELLOW off"); }
static void greenOn(void* c)   { Show(c, "GREEN  on "); }
static void greenOff(void* c)  { Show(c, "GREEN  off"); }

static void Trace(void* context, Demo_TrafficLight_TraceKind kind, const char* name) {
    (void)context;
    if (kind == Demo_TrafficLight_Trace_Transition) {
        printf("transition %s\n", name);
    }
}

int main(void) {
    static const Demo_TrafficLight_Hooks hooks = {
        .redOn = redOn,
        .redOff = redOff,
        .greenOn = greenOn,
        .greenOff = greenOff,
        .yellowOn = yellowOn,
        .yellowOff = yellowOff,
        .Trace = Trace,
    };
    Lamps lamps = { 0 };
    Demo_TrafficLight light;

    Demo_TrafficLight_Init(&light, &hooks, &lamps);
    Demo_TrafficLight_Start(&light);
    for (int tick = 1; tick <= 3; ++tick) {
        Demo_TrafficLight_Dispatch(&light, Demo_TrafficLight_Event_Timer);
        printf("tick %d -> state: %s\n", tick, Demo_TrafficLight_StateName(
    Demo_TrafficLight_Current(&light)));
    }
    printf("%d lamp switches\n", lamps.switches);
    return 0;
}
```

Listing 13.4: Output of the C program.

```
  [RED    on ]
transition Red -> Green
  [RED    off]
  [GREEN  on ]
```

```
tick 1 -> state: Green
transition Green -> Yellow
  [GREEN  off]
  [YELLOW on ]
tick 2 -> state: Yellow
transition Yellow -> Red
  [YELLOW off]
  [RED    on ]
tick 3 -> state: Red
7 lamp switches
```

The generated header is listed in Appendix F.

13.6 C# specifics

- The class is `public abstract` and uses nullable annotations (`#nullable enable`).
- Hooks are `protected abstract` methods; `Trace` is `protected virtual`.
- Event arguments are stored in private fields before dispatch.
- Table-based code builds its tables in the protected constructor using method groups and lambdas.

13.7 Tracing

With **Trace** ticked, the class gets a `TraceKind` enumeration and a virtual `Trace(TraceKind kind, name)` method that does nothing by default:

Kind	Reported when / name
`Start`	`Start()` is called; the class name.
`Event`	`Dispatch` receives an event; the event name.
`Ignored`	No state handled the event; the event name.
`Transition`	A transition fires, before any exit: `"S -> T"`, `"S (internal)"` or `"S -> T (history)"`.
`Exit`	A state is about to be exited (before its exit actions).
`Enter`	A state has been entered (before its entry actions).

In C++ the names are string literals (no allocation). Tutorials 2 and 3 show complete examples.

13.8 Thread safety

By default the machine is not thread-safe. With **Thread-safe** ticked, the class declares two hooks that you implement with any synchronisation primitive (mutex, critical section, RTOS semaphore, interrupt masking):

C++	`virtual void Lock() const = 0;  virtual void Unlock() const = 0;`
C#	`protected abstract void Lock();  protected abstract void Unlock();`

`Start`, `Dispatch`, the typed dispatch methods, `DispatchTimeout`, `Current`, `IsIn` and `IsFinished` acquire the lock on entry and release it on exit, also when an exception is thrown (RAII guard in C++, `try`/`finally` in C#). Hooks therefore run while the lock is held. `Post`, the static members (`ParentOf`, `StateName`, `EventName`) and, in C, `..._Init` do not lock.

```
class Player : public Demo::MediaPlayer {
  protected:
    void Lock() const override   { mutex_.lock(); }
    void Unlock() const override { mutex_.unlock(); }
  private:
    mutable std::mutex mutex_;     // const hooks need a mutable mutex
};
```

Caution

Hooks must not call public members of the machine (`IsIn`, `Current`, . . .) unless the lock is re-entrant (`std::recursive_mutex`; `Monitor` in C# already is). Never call `Dispatch` from a hook, even with a re-entrant lock.

13.9 Event queue

Actions often need to raise further events — “credit limit exceeded, cancel”, “self-test failed, enter error mode”. Calling `Dispatch` from a hook is not allowed, because the machine is in the middle of a transition. With **Queue** ticked, the class instead offers protected `Post` methods:

```
bool Post(Event event);               // events without parameters
bool PostCoin(int cents);             // one typed method per event with parameters
```

A posted event is stored and dispatched after the current `Start`/`Dispatch` call has finished its transition and all completion transitions — still inside the same call, before it returns. Events posted while the queue is being drained are appended and processed in order.

- In C++ the queue is a ring buffer of `kQueueCapacity` entries inside the object (no heap), and in C a ring buffer of `..._QueueCapacity` entries inside the machine structure. Parameters are copied by value: a reference and a top-level `const` are removed from the declared type (`const std::string&` is stored as `std::string`, `const int` as `int`), but a pointer is stored as it is (`const char*`), so what it points to must outlive the queued event.
- In C# the queue holds at most `QueueCapacity` events; the closure stored for a typed `Post`*`Event`* captures its arguments.
- `Post` returns `false` when the queue is full, and for events that cannot be posted (`Completion`, or an event with parameters passed to the untyped `Post`).
- `Post` does not lock; it is meant to be called from hooks, which already run under the lock when **Thread-safe** is enabled. From other threads, call `Dispatch`.

Tutorial 4 shows `Post` in action.

13.10 Timers

For diagrams with `after(...)` transitions the class declares

```
virtual void StartTimer(Event timeout, unsigned milliseconds, unsigned generation) = 0;   // C
    ++
virtual void CancelTimer(Event timeout) = 0;
```

(in C# the same as `protected abstract` methods with `int` parameters; in C as `StartTimer` and `CancelTimer` members of the hooks structure). Implement them with whatever timer your platform offers. When a timer elapses, pass the event and the generation it was started with to `DispatchTimeout`:

Listing 13.5: `DispatchTimeout` in the generated vending machine.

```
bool DispatchTimeout(Event timeout, unsigned generation) {
    const int index = TimerIndex(timeout);
    if (index < 0 || !timerArmed_[index] || timerGeneration_[index] != generation) {
        return false;
    }
    args_ = nullptr;
    const bool handled = DispatchCore(timeout);
    args_ = nullptr;
    DrainQueue();
    return handled;
}
```

Each timeout event has a generation counter that is incremented whenever its timer is started, and an *armed* flag that is cleared when the timer is cancelled or has fired, i.e. as soon as its timeout is dispatched: a timer fires at most once per start, even if the timeout's guard is false or its transition is internal. A timeout that arrives after its state was left, after the state was left and entered again, or a second time for the same start, does not match and is ignored — even if your timer implementation cannot cancel a callback that is already on its way. `Start` cancels every running timer (through `CancelTimer`) and makes outstanding timeouts stale.

Calling `Dispatch(timeout)` (or `Post(timeout)`) directly is still possible, for example to drive timeouts by hand in a test: it skips the generation check, but disarms the timer like `DispatchTimeout` does, so the timer's own callback is ignored when it arrives later. In C#:

```
private readonly Dictionary<Event, Timer> timers = new();

protected override void StartTimer(Event timeout, int milliseconds, int generation) =>
    timers[timeout] = new Timer(_ => DispatchTimeout(timeout, generation), null, milliseconds,
    Timeout.Infinite);

protected override void CancelTimer(Event timeout)
{
    if (timers.Remove(timeout, out var timer))
        timer.Dispose();
}
```

The timer callback runs on a thread-pool thread, so combine this with **Thread-safe**. On embedded systems, record the deadline in `StartTimer`, check it in the main loop or a timer interrupt, and call `DispatchTimeout` from the main loop.

13.11 Hook stubs

For a diagram that records the code behind its hooks (an imported Stateflow chart), **Generate Code** also writes a hook stub next to the generated files: `<Name>Hooks.hpp`, `<Name>Hooks.cs` or `<Name>Hooks.c`. It is written only if it does not exist yet, and never overwritten after that: it is your file. The stub implements every guard and action. The C++ and C# stubs are a class derived from the machine; the C stub has one function per hook and a hooks structure `<Prefix>_HookImpl` to pass to `<Prefix>_Init`. Each hook's body is the code it replaced, and the chart's data are declared as members (or file-scope variables in C) with corresponding types. The code is copied as written, so functions, bus types and literals of the source tool have to be supplied or adapted. The one exception is Stateflow's `send(E, ...)`, which raises an event: in C++ and C# it becomes `Post(Event::E)` or `Post(Event.E)`. A hook without recorded code gets an empty body, and a guard without code returns `false`.

13.12 Integration patterns

13.12.1 Unit testing

Because all hooks are virtual, a test double can record calls and control guards:

```
struct TestLight : Demo::TrafficLight {
    std::vector<std::string> calls;
    void redOn() override     { calls.push_back("redOn"); }
    void redOff() override    { calls.push_back("redOff"); }
    void greenOn() override   { calls.push_back("greenOn"); }
    void greenOff() override  { calls.push_back("greenOff"); }
    void yellowOn() override  { calls.push_back("yellowOn"); }
    void yellowOff() override { calls.push_back("yellowOff"); }
};

TEST(TrafficLight, TimerAdvancesFromRedToGreen) {
    TestLight light;
    light.Start();
    ASSERT_EQ(light.Current(), TestLight::State::Red);
    EXPECT_TRUE(light.Dispatch(TestLight::Event::Timer));
    EXPECT_EQ(light.Current(), TestLight::State::Green);
    EXPECT_EQ(light.calls, (std::vector<std::string>{ "redOn", "redOff", "greenOn" }));
}
```

Chapter 14

Import and Export

MachinaState reads and writes three textual formats, so that diagrams can move to and from other tools and into documentation, and it imports Stateflow charts from Simulink models. Export writes a file next to the diagram; import creates a new `.machina` file, lays it out automatically (Section 9.8) and opens it. Problems found during import are reported as warnings; the elements concerned are skipped.

14.1 PlantUML

Export Diagram as PlantUML writes a PlantUML state diagram that renders with any PlantUML installation or documentation tool that embeds it. States, nesting, regions (separated by `--`), initial and final states, history (`[H]`, `[H*]`), choice points (`<<choice>>`), entry and exit actions, internal transitions and labels `event [guard] / action` are written in PlantUML syntax. Details that PlantUML has no notation for — the namespace, event parameters and region names — are kept in special comments, so an exported file imports back without loss. This is the export of the washing machine of Tutorial 5:

```
@startuml Washer
’ @namespace Appliances
’ @event SetTemp(int celsius)
state Idle
state Running {
  ’ @region Program
  state Washing
  Washing : entry / motorSlow
  ...
  [*] --> Washing
  --
  ’ @region Heater
  state Heating
  ...
}
Running : entry / lockDoor
Idle --> Running : Start / fillWater
Washing --> Rinsing : after(30s) / drain
Running --> Idle : / beep
@enduml
```

Import PlantUML, SCXML or Mermaid State Machine reads the common subset of PlantUML's state diagram syntax: `state` declarations with nested blocks, aliases (`state "Long name" as X`), `[*]` for initial and final states, `<<choice>>`, `<<end>>`, `[H]` and `[H*]`, `--` region separators, `X : entry / a` and `X : exit / b`, and transitions with any arrow style (`->`, `-->`, `-up->`, …). It also reads internal transitions written as description lines (`X : event [guard] / action`; an internal transition with neither guard nor action is exported as `X : event /`). Notes, legends, skin parameters and other layout commands are skipped silently; a note without its `end note` and other lines it does not understand are reported and skipped.

14.2 Mermaid

Mermaid diagrams render directly in GitHub and GitLab Markdown, in many wikis and in documentation generators, which makes Mermaid the easiest way to show a state machine in a `README`, an issue or a pull request.

- **Copy Diagram as Mermaid (Markdown)** puts the diagram on the clipboard as a `` ```mermaid `` block, ready to paste into Markdown. It is also in the `...` menu of the diagram editor's title bar.
- **Export Diagram as Mermaid** writes a `.mmd` file, or a Markdown file with the block when you choose a `.md` file name.

The export is a `stateDiagram-v2` with the diagram name as its title. States, nesting, regions (`--`), initial and final states (`[*]`), choice points (`<<choice>>`), entry and exit actions, internal transitions, labels `event [guard] / action` and state colours are written in Mermaid syntax. Where Mermaid differs from MachinaState:

History
Mermaid has no history pseudostate. It is drawn as a small state labelled `H` or `H*` in its composite state and marked with a `%% @history` comment.

Final states
Mermaid draws one end node per composite state. The name of a final state is kept in a `%% @end` comment; a second final state in the same container is drawn as an ordinary state and marked `%% @final`.

Composite state actions
Mermaid does not allow description lines on composite states, so their entry and exit actions and internal transitions are shown in a note beside the state, marked `%% @describe`.

Special characters
`;`, `#`, `<` and `>` in labels are written as Mermaid entity codes (`#59;`, `#35;`, `#60;`, `#62;`); otherwise Mermaid would cut a label at a semicolon or drop text between angle brackets. They render as the original characters.

Reserved names
A state named after a Mermaid keyword (`Note`, `State`, `Default`, `Class`, `Style`, ...) is written under an alias with its name as display text.

As with PlantUML, the namespace, event parameters and region names travel in `%%` comments, which Mermaid ignores, so an exported diagram imports back without loss. An excerpt of the washing machine's export:

```
---
title: Washer
---
stateDiagram-v2
  %% @namespace Appliances
  %% @event SetTemp(int celsius)
  Idle
  state Running {
    %% @region Program
    state "Washing" as Washing
    Washing : entry / motorSlow
    ...
    [*] --> Washing
    %% @end ProgramDone
```

```
  Spinning --> [*] : Spun
  --
  %% @region Heater
  ...
}
%% @describe Running
note right of Running
  entry / lockDoor
  exit / unlockDoor
end note
Idle --> Running : Start / fillWater
Washing --> Rinsing : after(30s) / drain
```

Import reads `.mmd` and `.mermaid` files, and Markdown files, from which it takes the first ` ```mermaid ` block that holds a state diagram. It understands `stateDiagram-v2` and `stateDiagram` with front matter (the title becomes the diagram name), `state` declarations and blocks, display names (`state "Text" as id`; the text names the state if it is an identifier), `[*]`, `<<choice>>`, `--` region separators, description lines (`X : entry / a`, internal transitions as for PlantUML), and `classDef / class / :::` colours. Notes, `direction`, `style`, `click`, accessibility lines and directives are skipped. `<<fork>>` and `<<join>>` have no MachinaState equivalent; they are imported as ordinary states and reported.

14.3 SCXML

Export Diagram as SCXML writes a W3C State Chart XML document. The mapping is:

MachinaState	SCXML
State, composite state	`<state>`; orthogonal states become `<parallel>`
Initial transition	`initial` attribute, or `<initial>` when it has actions
Final state	`<final>`
History	`<history type="shallow|deep">`
Guard	`cond="isReady()"`, negated `cond="!isReady()"`
Actions	`<script>name();</script>` in `<onentry>`, `<onexit>` and transitions
Internal transition	`<transition>` without `target`, marked `type="internal"`
Timeout	`<send delay="...">` on entry and `<cancel>` on exit, with a matching event
Choice point	a transient state with eventless transitions, marked `machina:kind="choice"`
Namespace, event parameters	`machina:` attributes on `<scxml>`

Import accepts the same constructs. Every transition without a target is imported as an internal transition. A transition with `type="internal"` whose target lies inside its source state (an SCXML local transition) has no MachinaState equivalent; it is imported as an external transition and reported. Entry and exit actions of a region (a child `<state>` of `<parallel>`) are moved to the parallel state and reported. `machina:` attributes are recognised under any prefix bound to `urn:machinastate:scxml`. Executable content other than function calls, the data model, `<invoke>` and similar elements cannot be represented in a diagram; they are reported and skipped.

14.4 Stateflow (Simulink models)

Import PlantUML, SCXML, Mermaid or Stateflow State Machine also accepts a Simulink model (`.slx`) that contains a Stateflow chart. The result uses Stateflow semantics (Section 11.6), so the imported diagram behaves like the chart.

Libraries. A chart that a model uses through a library link, and every atomic subchart, is read from its library file `<library>.slx`. Libraries are found by name, as MATLAB finds them on its path: first in the folders of the setting `machinaState.stateflow.libraryPath`, then below the model's own folder and below each of its ancestors, up to three levels up.

Parameters and step period. Durations of `after(...)` and the chart's sample time are often names of MATLAB variables. Their values are read from the MATLAB scripts listed in `machinaState.stateflow.parameterFiles`. When that setting is empty, the import asks for scripts (**Cancel** skips them). Statements of the form `name = 0.25;` or `name = single(0.25);` are understood; computed values are not. The step period is the sample time of the chart's block or, when the chart inherits it, the model's fixed solver step. If it is not known (for example with a variable-step solver), the import asks for it, and it can also be set later in the diagram's properties. A model with several charts asks which one to import.

Mapping.

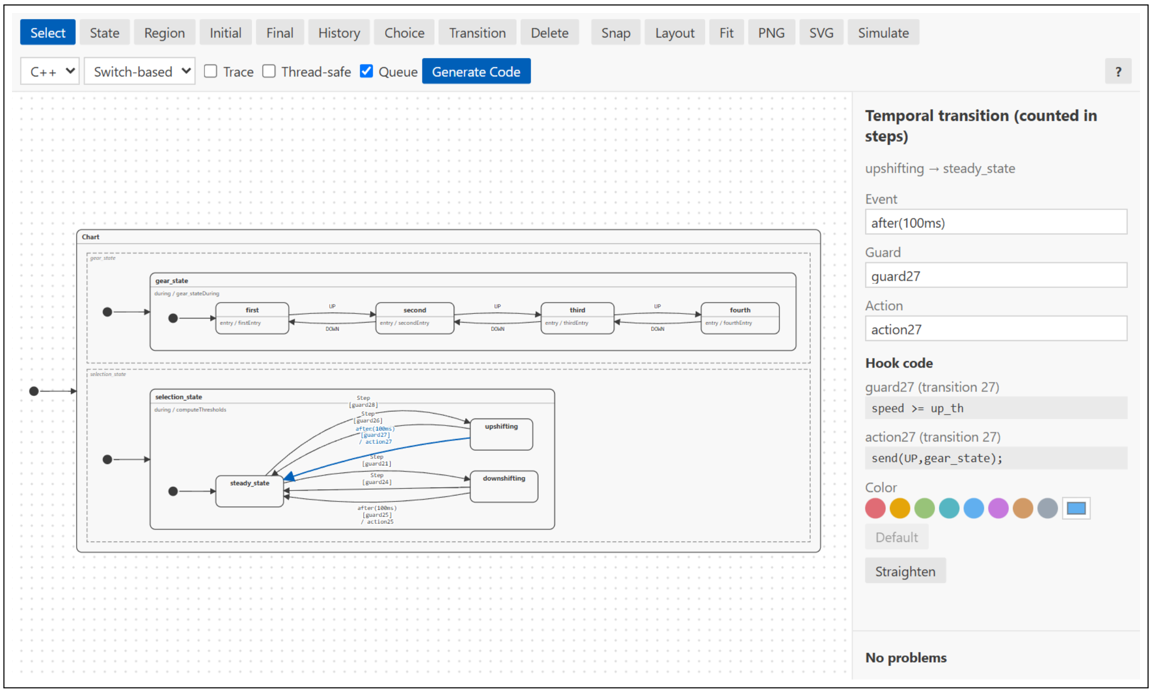
Stateflow	**MachinaState**
State, subchart	state; a subchart is only a separate view of a composite state
Parallel (AND) decomposition	one region per parallel state, containing a state of the same name that keeps its actions and substates; a parallel chart gets a top-level state `Chart` to hold the regions
Box	its contents move into the enclosing state
Atomic subchart	the library chart, imported into the state
Default transition	initial pseudostate; into a junction: through a transient state *State*`Start` with a completion transition. A flow chart of junctions in a state without substates becomes entry code of that state
Connective junction	choice point
Junction without exit	the path becomes an internal transition of its source state
History junction	shallow history pseudostate
Event `E`	event `E`; `E1 | E2` gives one transition per event
`send(E, ...)`	an action hook that raises `E` with `Post`; the event queue is enabled
Condition only, or no label	transition on the step event
`after(n,sec)`, `msec`, `usec`	`after(...ms)` or `after(...s)`
`after(n,tick)`	`after(n steps)`
`[c || after(...)]`	a step transition with guard `c` plus an `after(...)` transition
Expressions, statements	guard hooks `guard`*SSID* and action hooks `action`*SSID*, *State*`Entry`, *State*`During`, *State*`Exit`
`entry`, `during`, `exit`	entry, during and exit actions
Data	recorded in the diagram and declared in the hook stub
Comments	removed: `//` and `/* */`, in the MATLAB action language `%` and `%{ %}`

Guards and actions are hooks in MachinaState, not code. Each expression or statement list therefore becomes a hook, named after the Stateflow SSID or state so that it can be traced back. The original code is kept in the diagram, shown read-only under **Hook code** in the properties panel, and written into the hook stub when code is generated (Section 13.11).

Limitations. Graphical, MATLAB and Simulink functions are not imported; the hook code calls them, so the hook stub needs their implementation. `on` and `bind` sections of state labels are dropped. A transition from a state to one of its own substates (a Stateflow inner transition) is imported as an external transition, which exits the state. Condition actions on junction paths run once a complete path has been chosen, not while Stateflow tests the path. An event raised with `send()` is processed after the current event and offered to the whole machine, while Stateflow processes it at once and only in the state that `send()` names. Each of these cases is reported as a warning.

14.4.1 Example: a gear shift chart

The folder `examples/stateflow` of the MachinaState repository contains `gear_logic.slx`, a Simulink model with the Stateflow chart `Gear_Logic`: a simplified controller for a four-gear automatic transmission.[1] To import it, run **Import PlantUML, SCXML, Mermaid or Stateflow State Machine**, pick the file, press **Cancel** when asked for parameter scripts (the chart has no symbolic durations), and save the result as `Gear_Logic.machina`. The same file is in the folder for comparison.


Select
State
Region
Initial
Final
History
Choice
Transition
Delete
Snap
Layout
Fit
PNG
SVG
Simulate
Generate Code
?
Chart
gear_state
gear_state
during / gear_stateDuring
first
entry / firstEntry
UP
DOWN
second
entry / secondEntry
UP
DOWN
third
entry / thirdEntry
UP
DOWN
fourth
entry / fourthEntry
selection_state
selection_state
during / computeThresholds
Step
[guard28]
Step
[guard26]
after(100ms)
[guard27]
/ action27
upshifting
Step
[guard21]
Step
[guard24]
downshifting
steady_state
after(100ms)
[guard25]
/ action25
Default
Straighten

C++ ▾ Switch-based ▾ ☐ Trace ☐ Thread-safe ☑ Queue

Temporal transition (counted in steps)

upshifting → steady_state

Event: after(100ms)

Guard: guard27

Action: action27

Hook code

guard27 (transition 27)

```
speed >= up_th
```

action27 (transition 27)

```
send(UP,gear_state);
```

Color: ● ● ● ● ● ● ● ● ▭

No problems

Figure 14.1: The imported gear shift chart; the panel shows the Stateflow code behind the selected upshift transition.

The import shows how the chart was mapped:

- The chart has two parallel states, which become two regions of a top-level state `Chart`. `gear_state` keeps the current gear, `first` to `fourth`, and changes it on the events `UP` and `DOWN`. `selection_state` compares the speed with two thresholds and decides when to shift.
- The chart runs at the model's fixed solver step of 0.01 s, so the step period is 10 ms. `after(0.1, sec)` becomes `after(100ms)`, which is 10 steps.

---

[1] `gear_logic.slx` is copyright © 2021 iCub Tech and licensed under the BSD 3-Clause License, see `examples/stateflow/LICENSE`. It is unmodified from https://github.com/mesh-iit/training-matlab-rev-lights.

- Conditions and statements become hooks named after their transition or state. For example, `guard27` stands for `[speed >= up_th]`, `secondEntry` for `gear=2;`, and `gear_stateDuring` for the during action of `gear_state`. The properties panel shows the code behind a hook under **Hook code**.
- The chart raises `UP` and `DOWN` with `send()`. The import enables the event queue, and the hook stub raises them with `Post`.
- The lines of `gear_state` that start with `%on` are MATLAB comments and are ignored. `computeThresholds`, a graphical function, is not imported; its call stays an action hook.

Generate Code then writes `Gear_Logic.hpp` and, once, the hook stub `Gear_LogicHooks.hpp` (Section 13.11). An excerpt:

Listing 14.1: Part of the hook stub of the gear shift chart.

```
// Variables used by the hook code.
double down_th{};   // local, Inherit: From definition in chart
double up_th{};   // local, Inherit: From definition in chart
double delta{};   // local, Inherit: From definition in chart
double speed{};   // local, Inherit: From definition in chart
double gear{};   // local, Inherit: From definition in chart

void gear_stateDuring() override {
    speed = max(0,min(speed+delta,100));
}

void action27() override {
    Post(Event::UP);
}
```

The code is MATLAB, so some of it needs adapting: `max` and `min` become `std::max` and `std::min`, `computeThresholds` needs an implementation, and the application sets `delta` from its inputs. The machine is then driven by one step event per period:

```
Gear_LogicHooks gears;
gears.Start();
// every 10 ms:
gears.Dispatch(Gear_Logic::Event::Step);
```

The test suite imports this model, compares the result with `Gear_Logic.machina`, and runs the code generated from it in every language and style against the reference engine.

14.5 Round trips

The test suite exports every example to PlantUML, Mermaid and SCXML, imports the result, and requires the imported diagram to produce exactly the same trace as the original in the reference engine. It also renders every Mermaid export with Mermaid itself. Element positions are not part of any of the formats; they are recomputed by the automatic layout.

None of the three formats can express Stateflow semantics. Exporting a diagram that uses them writes the UML reading of the diagram, without during actions, and shows a warning.

Chapter 15

Validation Rules

Every problem is an error unless it is marked as a warning. Errors prevent code generation and simulation; warnings point at parts of the diagram that are valid but have no effect.

Problem	**Cause and fix**
Machine name is not a valid class name	Use an identifier that is not a C++/C# keyword.
Namespace is not valid	Parts separated by `.` or `::` must be identifiers.
State: name must be a valid identifier	Letters, digits and `_`, not starting with a digit, not a keyword, not `None`.
name is not unique	State names share one enumeration, even when nested. Rename one.
needs an initial pseudostate	Add an initial pseudostate to the top level, the composite state or the region named.
A region must be placed inside a state	Move the region into the state it divides.
must be placed inside one of the regions of . . .	A state with regions cannot have substates outside them; move the element into a region.
A region of . . . contains no states	Add states to the region, or delete it.
transitions between orthogonal regions are not allowed	Regions are independent; leave the orthogonal state instead, or use one region.
History pseudostates are not supported in orthogonal states	A state with several regions cannot have a history pseudostate; remove it (a deep history pseudostate in an enclosing state restores all regions).
more than one initial pseudostate	Delete one of them.
initial pseudostate must have exactly one outgoing transition	Connect it to the default state (once).
initial transition must target a state inside the same region	Point it at a state inside its own composite state.
initial transition cannot have an event or guard	Clear them.
event '. . . ' is not a valid identifier (or is reserved)	Use an identifier, `after(...)` or leave the event empty for a completion transition. `Completion` and `None` are reserved.
timeout must be a positive whole number of milliseconds	e.g. `after(250ms)`, `after(2s)`; fractions of a millisecond are not possible.
clashes with a generated timeout event	An event is named like a generated timeout event (*State*`_After`*n*`ms`); rename it.
Transitions cannot target an initial pseudostate	Target the composite state instead.
during actions need Stateflow semantics	During actions exist only under Stateflow semantics; switch the semantics or clear them.
after(. . . steps) needs Stateflow semantics	Use a time, such as `after(5s)`, or switch to Stateflow semantics.
needs a step period to be counted in steps	Set the step period in the state machine's properties, or write `after(n steps)`.

Problem	**Cause and fix**
The step period must be a positive number	Enter the period in milliseconds, e.g. `10`.
Step event . . . is not a valid identifier	Name the step event with an identifier other than `None` and `Completion`.
The step event . . . cannot have parameters	Remove its parameter list.
Transitions cannot start at a region	Draw the transition from the orthogonal state or from a state inside the region.
a region cannot be a target	Target the orthogonal state or a state inside the region.
the target history pseudostate is not inside a composite state	Move the history pseudostate into a composite state, or retarget the transition.
Final state cannot have outgoing transitions / contain other states	Use a normal state.
more than one unguarded transition for event	Add guards or remove one — the second could never fire.
more than one unguarded completion transition	The same for transitions without an event.
Completion transitions of . . . form an endless loop	Each of these states completes immediately after the previous one's completion transition, for ever. Give one of the transitions a guard or an event.
A choice point needs at least one outgoing branch	Connect the choice point to its targets.
at most one [else] (unguarded) branch	Give all other branches a guard.
branches leaving a choice point cannot have an event	Clear the event; the incoming transition carries it.
Choice branch: invalid target	Branches cannot lead to initial pseudostates (or to regions).
Choice point has no incoming transition (warning)	The choice point is never used and its branches are not checked; connect a transition to it or delete it.
Choice points form a cycle	A chain of choice points must end in states.
guard must be a method name, optionally negated	Expressions are not allowed; create a guard method.
action must be a method name	Remove parentheses with arguments or operators.
only self transitions with an event can be internal	Untick **Internal**, reconnect the transition or give it an event.
A history pseudostate must be placed inside a composite state	Move it into a state that has substates.
more than one history pseudostate	One per composite state; tick **Deep history** instead.
History pseudostates cannot have outgoing transitions	The initial pseudostate defines the default.
parameter must be written as 'type name'	e.g. `int level`.

Problem	**Cause and fix**
Parameters are declared for event . . . , which is not used by any transition (warning)	The declaration has no effect; remove it, or use the event. The same warning appears for generated (timeout and completion) events, which cannot have parameters.
used with different parameter lists	A hook is shared by events with different parameters, or used both as entry/exit action and with parameters. Use separate hooks.
used both as a guard and as an action	Use different names.
clashes with a generated member name	Avoid `Start`, `Dispatch...`, `Post...`, `Current`, `IsIn`, `Trace`, `Lock`, `StartTimer`, `CancelTimer`, . . . and the prefixes `Entry_`, `Exit_`, `Init_`, `Guard_`, `Fire_`, `Transition_`.
Queue capacity must be a whole number between 1 and 65535	Fix `codegen.queueCapacity` in the JSON source.

Chapter 16

File Format

A `.machina` file is a UTF-8 JSON document. Coordinates are diagram pixels; positions are absolute, and nesting is expressed through `parent`.

Field	**Meaning**
`formatVersion`	Optional; absent means 1. Version 2 adds Stateflow semantics. A file with a higher version than the installed MachinaState supports is refused instead of misread.
`semantics`	Optional: `uml` (default) or `stateflow` (Section 11.6).
`step`	Stateflow semantics: `event` (default `Step`) and `periodMs`.
`name`	Class name and output file name.
`namespace`	Optional namespace.
`states[]`	Elements: `id`, `name`, `kind` (`state`, `region`, `initial`, `final`, `history`, `choice`), `parent` (id or `null`), `x`, `y`, `w`, `h`; states also `entry`, `exit` and, under Stateflow semantics, `during`; history also `deep`; optional `color`. The parent of a region is a state; elements inside a region have the region as parent.
`transitions[]`	`id`, `source`, `target`, `event` (name, `after(...)`, under Stateflow semantics also `after(n steps)`, or empty), `guard` (may be `else` on choice branches), `action`; optional `bend` (curvature), `internal`, `color`.
`events`	Map from event name to parameter list, e.g. `{ "Volume": "int level" }`.
`hooks`	Optional map from hook name to `code` and `origin`: the code a hook stands for (filled by the Stateflow import).
`data[]`	Optional variables used by the hook code: `name`, `scope`, `type`, optional `size`.
`scenarios[]`	Optional simulator scenarios: `name` and `steps[]`, each step an `event` with optional `args` (array) and `guards` (map of guard name to `false`).
`codegen`	`language` (`cpp`/`csharp`/`c`), `style` (`switch`/`table`), `trace`, `threadSafe`, `queue`, `queueCapacity` (default 16).

Listing 16.1: The beginning of `examples/DoorLock.machina`.

```
{
  "name": "DoorLock",
  "namespace": "Security",
  "states": [
    {
      "id": "init",
      "name": "Initial",
      "kind": "initial",
      "parent": null,
      "x": 60,
      "y": 168,
      "w": 24,
      "h": 24
    },
    {
      "id": "Locked",
      "name": "Locked",
      "kind": "state",
      "parent": null,
      "x": 140,
      "y": 130,
      "w": 210,
      "h": 100,
      "entry": "engageBolt",
      "exit": ""
    },
    {
      "id": "Unlocked",
      "name": "Unlocked",
      "kind": "state",
```

The `id` values are internal and never shown in the editor. Because the file is plain JSON with one property per line, it diffs and merges well in version control.

Chapter 17

Troubleshooting and FAQ

Generate Code reports problems.
Fix all entries in the problems list first; click an entry to select the element concerned.

A state does not become nested.
Nesting is decided by the position of the state's *centre*. Move the state so that its centre lies inside the intended parent.

The editor shows "The file is not valid JSON".
The file was edited by hand and contains a syntax error. Reopen it with the text editor, fix it, and switch back.

Guards are not evaluated in the simulator.
The simulator cannot run your code; set each guard's result with its check box under **Guard results**.

A timer fires too early after re-entering a state.
Pass the `generation` received by `StartTimer` to `DispatchTimeout` instead of calling `Dispatch` (Section 13.10).

An imported diagram looks different from the original drawing.
None of PlantUML, Mermaid and SCXML stores positions; imported diagrams are laid out automatically. Rearrange them, or use **Layout** again after changes.

`Dispatch` returns `false` for an event with parameters.
Use the typed method, e.g. `DispatchEnterCode(1234)`.

My program deadlocks with Thread-safe.
A hook calls a public member of the machine while the lock is held. Use a recursive lock or avoid the call.

Can I use the generated code without exceptions or RTTI?
Yes. The C++ code uses neither, and allocates nothing on the heap.

How do I model a timeout?
Use an event such as `after(5s)` and implement the timer hooks (Sections 10.9 and 13.10).

An action needs to trigger another event.
Tick **Queue** and call `Post` (Section 13.9).

My choice point guard does not see the effect of the incoming action.
Choice guards are evaluated before any action runs (Section 10.10); compute the result from the event parameters and the current data instead.

Generated code is not updated automatically.
Enable `machinaState.generateOnSave`.

Copy and paste does not work.
Click on the canvas first so that it has the keyboard focus; text fields in the side panel handle the clipboard themselves.

How do I share one action between states?
Use the same name; the generator declares it once.

Can two diagrams generate into one project?

Yes; give them different class names, or the same class name in different namespaces.

Chapter 18

Development and Testing

The repository contains an automated test suite that every change to MachinaState must pass.

```
npm test              # compile, then run all tests in test/
npm run test:unit     # model and golden tests only (no compilers needed)
npm run test:ui       # editor and simulator tests in a browser
npm run test:update   # regenerate test/golden after an intended generator change
```

Model tests
check validation rules, transition ordering, LCA computation, history, internal transitions, choice points, completion transitions, timeouts and event parameters.

Golden tests
generate every example in every language, style and option combination and compare the result with the files in `test/golden`. Any change to generated code shows up as a reviewable diff.

Compile tests
generate the same variants, compile them with `g++ -Wall -Wextra -Werror -pedantic`, `gcc -std=c99 -Wall -Wextra -Werror -pedantic` and `dotnet` (warnings as errors), and drive each with an identical pseudo-random event script — including stale timeouts — through an automatically generated harness whose hooks print every call. All six variants of a machine must produce byte-identical traces, identical to the trace of the reference engine that the simulator uses. The tests are skipped when a compiler is not installed.

Import/export tests
export every example to PlantUML, Mermaid and SCXML, import it again and compare the engine traces with the original's. The Mermaid exports are also rendered with Mermaid itself in a headless browser (`test/mermaid-render.test.js`).

UI tests
(`test/ui.test.js`) drive the real editor and simulator in a headless browser with Playwright: drawing states, regions and transitions, nesting, multi-selection, copy and paste, code generation options, simulation with regions, timers, guards and arguments, scenario recording and replay, and coverage. They use an installed Chromium, Edge or Chrome (`MACHINA_BROWSER` selects one) and are skipped if none is found; CI requires them.

The GitHub Actions workflow `.github/workflows/ci.yml` runs the tests on Linux and Windows, packages the VSIX, builds this manual, and on version tags (`v*`) publishes to the Visual Studio Marketplace and Open VSX when the secrets `VSCE_PAT` and `OVSX_PAT` are configured.

The documentation assets are reproducible: `node docs/tools/build.js` writes the example diagrams, the screenshot scenarios, the generated code and the code excerpts shown in this manual, and `node docs/tools/capture.js` renders every screenshot scenario in `docs/tools/harness.html` and saves it to `docs/images`.

Appendix A

Generated Code: Flat Traffic Light, C++ Switch-Based

```cpp
// TrafficLight: switch-based hierarchical state machine generated by MachinaState.
// Do not edit by hand. Derive from this class and implement the guard and action hooks.
#pragma once

#include <cstddef>

namespace Demo {

    class TrafficLight {
      public:
        enum class State {
            Red,
            Green,
            Yellow,
            None
        };

        enum class Event {
            Timer
        };

        static constexpr std::size_t kStateCount  = 3;
        static constexpr std::size_t kEventCount  = 1;
        static constexpr std::size_t kRegionCount = 1;   // region 0 is the top level

        virtual ~TrafficLight() = default;

        // Resets the machine (running timers are cancelled, history is forgotten) and enters the initial state configuration.
        void Start() {
            for (State& state : active_) {
                state = State::None;
            }
            EnterDown(State::None, State::Red);
        }

        // Dispatches an event to the innermost active states first. Returns false if the event was not handled.
        bool Dispatch(Event event) {
            const bool handled = DispatchCore(event);
            return handled;
        }

        // Innermost active state (following the first region of orthogonal states).
        State Current() const {
            return Innermost(active_[0]);
        }

        // True if the given state is active.
        bool IsIn(State state) const {
            return state != State::None && active_[RegionOf(state)] == state;
        }

    // True once a top-level final state has been reached.
    bool IsFinished() const {
        return false;
    }

    static State ParentOf(State state) {
        (void)state;
        return State::None;
    }

    static const char* StateName(State state) {
        switch (state) {
            case State::Red:
                return "Red";
            case State::Green:
                return "Green";
            case State::Yellow:
                return "Yellow";
            default:
                return "None";
        }
    }

    static const char* EventName(Event event) {
        switch (event) {
            case Event::Timer:
                return "Timer";
            default:
                return "";
        }
    }

  protected:
    // Actions
    virtual void redOn() = 0;
    virtual void redOff() = 0;
    virtual void greenOn() = 0;
    virtual void greenOff() = 0;
    virtual void yellowOn() = 0;
    virtual void yellowOff() = 0;

  private:
    bool DispatchCore(Event event) {
        if (active_[0] != State::None && DispatchState(active_[0], event)) {
            return true;
        }
        return false;
    }

    // Offers the event to every active region below `state` first, then to `state` itself.
    bool DispatchState(State state, Event event) {
        const std::size_t first = FirstRegionOf(state);
        for (std::size_t region = first; region < first + RegionsOf(state); ++region) {
            if (active_[region] != State::None && DispatchState(active_[region], event)) {
                return true;
            }
        }
        return HandleEvent(state, event);
    }

    bool HandleEvent(State state, Event event) {
        switch (state) {
            case State::Red:
                switch (event) {
                        case Event::Timer:
                            ExitState(State::Red);
                            EnterDown(State::None, State::Green);
                            return true;
                        default:
                            break;
                    }
                    break;
                case State::Green:
                    switch (event) {
                        case Event::Timer:
                            ExitState(State::Green);
                            EnterDown(State::None, State::Yellow);
                            return true;
                        default:
                            break;
                    }
                    break;
                case State::Yellow:
                    switch (event) {
                        case Event::Timer:
                            ExitState(State::Yellow);
                            EnterDown(State::None, State::Red);
                            return true;
                        default:
                            break;
                    }
                    break;
                default:
                    break;
            }
            return false;
        }

        static std::size_t RegionOf(State state) {
            (void)state;
            return 0;
        }

        static std::size_t FirstRegionOf(State state) {
            (void)state;
            return 0;
        }

        static std::size_t RegionsOf(State state) {
            (void)state;
            return 0;
        }

        void EnterRegion(std::size_t region) {
            (void)region;
        }

        void OnEntry(State state) {
            switch (state) {
                case State::Red:
                    redOn();
                    break;
                case State::Green:
                    greenOn();
                    break;
                case State::Yellow:
                    yellowOn();
                    break;
                default:
                    break;
            }
        }

        void OnExit(State state) {
            switch (state) {
                case State::Red:
                    redOff();
                    break;
                case State::Green:
                    greenOff();
                    break;
                case State::Yellow:
                    yellowOff();
                    break;
                default:
                    break;
            }
        }

        void ExitState(State state) {
            const std::size_t first = FirstRegionOf(state);
            for (std::size_t region = first + RegionsOf(state); region-- > first;) {
                if (active_[region] != State::None) {
                    ExitState(active_[region]);
                }
            }
            OnExit(state);
            active_[RegionOf(state)] = State::None;
        }

        // Enters every state from below ‘from‘ down to ‘target‘, then follows default (initial) transitions
        // below ‘target‘.
        void EnterDown(State from, State target) {
            State state = target;
            State next  = State::None;
            while (ParentOf(state) != from) {
                next  = state;
                state = ParentOf(state);
            }
            EnterState(state);
            if (next == State::None) {
                EnterDefault(state);
                return;
            }
            EnterDown(state, target);
        }

        void EnterState(State state) {
            active_[RegionOf(state)] = state;
            OnEntry(state);
        }

        void EnterDefault(State state) {
            const std::size_t first = FirstRegionOf(state);
            for (std::size_t region = first; region < first + RegionsOf(state); ++region) {
                EnterRegion(region);
            }
        }

        State Innermost(State state) const {
            while (state != State::None && RegionsOf(state) > 0 && active_[FirstRegionOf(state)] != State::None) {
                state = active_[FirstRegionOf(state)];
            }
            return state;
        }

        State active_[kRegionCount] = { State::None };   // active state per region
    };
}
```

Appendix B

Generated Code: Flat Traffic Light, C++ Table-Based

```cpp
// TrafficLight: table-based hierarchical state machine generated by MachinaState.
// Do not edit by hand. Derive from this class and implement the guard and action hooks.
#pragma once

#include <cstddef>

namespace Demo {

    class TrafficLight {
      public:
        enum class State {
            Red,
            Green,
            Yellow,
            None
        };

        enum class Event {
            Timer
        };

        static constexpr std::size_t kStateCount  = 3;
        static constexpr std::size_t kEventCount  = 1;
        static constexpr std::size_t kRegionCount = 1;   // region 0 is the top level

        virtual ~TrafficLight() = default;

        // Resets the machine (running timers are cancelled, history is forgotten) and enters the initial state configuration.
        void Start() {
            for (State& state : active_) {
                state = State::None;
            }
            EnterDown(State::None, State::Red);
        }

        // Dispatches an event to the innermost active states first. Returns false if the event was not handled.
        bool Dispatch(Event event) {
            const bool handled = DispatchCore(event);
            return handled;
        }

        // Innermost active state (following the first region of orthogonal states).
        State Current() const {
            return Innermost(active_[0]);
        }

        // True if the given state is active.
        bool IsIn(State state) const {
            return state != State::None && active_[RegionOf(state)] == state;
        }

        // True once a top-level final state has been reached.
        bool IsFinished() const {
            return false;
        }

        static State ParentOf(State state) {
            return state == State::None ? State::None : kParent[Index(state)];
        }

        static const char* StateName(State state) {
            return state == State::None ? "None" : kStateNames[Index(state)];
        }

        static const char* EventName(Event event) {
            return kEventNames[static_cast<std::size_t>(event)];
        }

      protected:
        // Actions
        virtual void redOn() = 0;
        virtual void redOff() = 0;
        virtual void greenOn() = 0;
        virtual void greenOff() = 0;
        virtual void yellowOn() = 0;
        virtual void yellowOff() = 0;

      private:
        bool DispatchCore(Event event) {
            if (active_[0] != State::None && DispatchState(active_[0], event)) {
                return true;
            }
            return false;
        }

        // Offers the event to every active region below ‘state‘ first, then to ‘state‘ itself.
        bool DispatchState(State state, Event event) {
            const std::size_t first = FirstRegionOf(state);
            for (std::size_t region = first; region < first + RegionsOf(state); ++region) {
                if (active_[region] != State::None && DispatchState(active_[region], event)) {
                    return true;
                }
            }
            return HandleEvent(state, event);
        }

        using Handler = void (TrafficLight::*)();
        using GuardFn = bool (TrafficLight::*)();

        struct TransitionRow {
            State   source;
            Event   event;
            GuardFn guard;
            bool    negate;
            Handler action;
            State   target;   // None for internal transitions
            State   lca;
        };

        struct RegionRow {
            State   owner;
            State   initial;
            Handler action;
        };

        static const State   kParent[kStateCount];
        static const Handler kEntry[kStateCount];
        static const Handler kExit[kStateCount];
        static const char* const kStateNames[kStateCount];
        static const unsigned char kRegionOf[kStateCount];     // region that contains the state
        static const unsigned char kFirstRegion[kStateCount];  // first own region
        static const unsigned char kRegionsOf[kStateCount];    // number of own regions
        static const bool    kIsFinal[kStateCount];
        static const RegionRow kRegions[kRegionCount];
        static const char* const kEventNames[kEventCount];
        static const TransitionRow kTransitions[3];

        static std::size_t Index(State state) {
            return static_cast<std::size_t>(state);
        }

        static std::size_t RegionOf(State state) {
            return kRegionOf[Index(state)];
        }

        static std::size_t FirstRegionOf(State state) {
            return kFirstRegion[Index(state)];
        }

        static std::size_t RegionsOf(State state) {
            return kRegionsOf[Index(state)];
        }

        static bool IsFinalState(State state) {
            return state != State::None && kIsFinal[Index(state)];
        }

        void Invoke(Handler handler) {
            if (handler != nullptr) {
                (this->*handler)();
            }
        }

        bool HandleEvent(State state, Event event) {
            for (const TransitionRow& row : kTransitions) {
                if (row.source == state && row.event == event && TryRow(row)) {
                    return true;
                }
            }
            return false;
        }

        bool TryRow(const TransitionRow& row) {
            if (row.guard != nullptr && (this->*row.guard)() == row.negate) {
                return false;
            }
            State root = row.source;
            while (ParentOf(root) != row.lca) {
                root = ParentOf(root);
            }
            ExitState(root);
            Invoke(row.action);
            EnterDown(row.lca, row.target);
            return true;
        }

        void EnterRegion(std::size_t region) {
            Invoke(kRegions[region].action);
        EnterDown(kRegions[region].owner, kRegions[region].initial);
    }

    void ExitState(State state) {
        const std::size_t first = FirstRegionOf(state);
        for (std::size_t region = first + RegionsOf(state); region-- > first;) {
            if (active_[region] != State::None) {
                ExitState(active_[region]);
            }
        }
        Invoke(kExit[Index(state)]);
        active_[RegionOf(state)] = State::None;
    }

    // Enters every state from below ‘from‘ down to ‘target‘, then follows default (initial) transitions
    // below ‘target‘.
    void EnterDown(State from, State target) {
        State state = target;
        State next  = State::None;
        while (ParentOf(state) != from) {
            next  = state;
            state = ParentOf(state);
        }
        EnterState(state);
        if (next == State::None) {
            EnterDefault(state);
            return;
        }
        EnterDown(state, target);
    }

    void EnterState(State state) {
        active_[RegionOf(state)] = state;
        Invoke(kEntry[Index(state)]);
    }

    void EnterDefault(State state) {
        const std::size_t first = FirstRegionOf(state);
        for (std::size_t region = first; region < first + RegionsOf(state); ++region) {
            EnterRegion(region);
        }
    }

    State Innermost(State state) const {
        while (state != State::None && RegionsOf(state) > 0 && active_[FirstRegionOf(state)] != State::None) {
            state = active_[FirstRegionOf(state)];
        }
        return state;
    }

    State active_[kRegionCount] = { State::None };   // active state per region
};

inline const TrafficLight::State TrafficLight::kParent[TrafficLight::kStateCount] = {
    State::None,
    State::None,
    State::None,
};

inline const TrafficLight::Handler TrafficLight::kEntry[TrafficLight::kStateCount] = {
    &TrafficLight::redOn,
    &TrafficLight::greenOn,
        &TrafficLight::yellowOn,
    };

    inline const TrafficLight::Handler TrafficLight::kExit[TrafficLight::kStateCount] = {
        &TrafficLight::redOff,
        &TrafficLight::greenOff,
        &TrafficLight::yellowOff,
    };

    inline const char* const TrafficLight::kStateNames[TrafficLight::kStateCount] = {
        "Red",
        "Green",
        "Yellow",
    };

    inline const unsigned char TrafficLight::kRegionOf[TrafficLight::kStateCount] = {
        0,
        0,
        0,
    };

    inline const unsigned char TrafficLight::kFirstRegion[TrafficLight::kStateCount] = {
        0,
        0,
        0,
    };

    inline const unsigned char TrafficLight::kRegionsOf[TrafficLight::kStateCount] = {
        0,
        0,
        0,
    };

    inline const bool TrafficLight::kIsFinal[TrafficLight::kStateCount] = {
        false,
        false,
        false,
    };

    // owner, initial target, initial action
    inline const TrafficLight::RegionRow TrafficLight::kRegions[TrafficLight::kRegionCount] = {
        { State::None, State::Red, nullptr },
    };

    inline const char* const TrafficLight::kEventNames[TrafficLight::kEventCount] = {
        "Timer",
    };

    // source, event, guard, negate guard, action, target, least common ancestor
    inline const TrafficLight::TransitionRow TrafficLight::kTransitions[3] = {
        { State::Red, Event::Timer, nullptr, false, nullptr, State::Green, State::None },
        { State::Green, Event::Timer, nullptr, false, nullptr, State::Yellow, State::None },
        { State::Yellow, Event::Timer, nullptr, false, nullptr, State::Red, State::None },
    };

}
```

Appendix C

Generated Code: Flat Traffic Light, C# Switch-Based

```
// <auto-generated>
//     TrafficLight: switch-based hierarchical state machine generated by MachinaState.
//     Do not edit by hand. Derive from this class and implement the guard and action hooks.
// </auto-generated>
#nullable enable
using System;

namespace Demo
{
    public abstract class TrafficLight
    {
        public enum State
        {
            Red,
            Green,
            Yellow,
            None,
        }

        public enum Event
        {
            Timer,
        }

        public const int StateCount = 3;
        public const int RegionCount = 1; // region 0 is the top level

        private readonly State[] active = { State.None }; // active state per region

        /// <summary>Innermost active state (following the first region of orthogonal states).</summary>
        public State Current => Innermost(active[0]);

        /// <summary>True once a top-level final state has been reached.</summary>
        public bool IsFinished => false;

        /// <summary>Resets the machine (running timers are cancelled, history is forgotten) and enters the initial state configuration.</summary>
        public void Start()
        {
            Array.Fill(active, State.None);
            EnterDown(State.None, State.Red);
        }

        /// <summary>Dispatches an event to the innermost active states first. Returns false if the event was not handled.</summary>
        public bool Dispatch(Event trigger)
        {
            return DispatchCore(trigger);
        }

        /// <summary>True if the given state is active.</summary>
        public bool IsIn(State state)
        {
            return state != State.None && active[RegionOf(state)] == state;
        }

        public static State ParentOf(State state) => State.None;

        protected abstract void redOn();
        protected abstract void redOff();
        protected abstract void greenOn();
        protected abstract void greenOff();
        protected abstract void yellowOn();
        protected abstract void yellowOff();

        private bool DispatchCore(Event trigger)
        {
            if (active[0] != State.None && DispatchState(active[0], trigger))
            {
                return true;
            }
            return false;
        }

        // Offers the event to every active region below ‘state‘ first, then to ‘state‘ itself.
        private bool DispatchState(State state, Event trigger)
        {
            var first = FirstRegionOf(state);
            for (var region = first; region < first + RegionsOf(state); region++)
            {
                if (active[region] != State.None && DispatchState(active[region], trigger))
                {
                    return true;
                }
            }
            return HandleEvent(state, trigger);
        }

        private bool HandleEvent(State state, Event trigger)
        {
            switch (state)
            {
                case State.Red:
                    switch (trigger)
                    {
                        case Event.Timer:
                            ExitState(State.Red);
                            EnterDown(State.None, State.Green);
                            return true;
                    }
                    break;
                case State.Green:
                    switch (trigger)
                    {
                        case Event.Timer:
                            ExitState(State.Green);
                            EnterDown(State.None, State.Yellow);
                            return true;
                    }
                    break;
                case State.Yellow:
                    switch (trigger)
                    {
                        case Event.Timer:
                            ExitState(State.Yellow);
                            EnterDown(State.None, State.Red);
                        return true;
                    }
                    break;
            }
            return false;
        }

        private static int RegionOf(State state) => 0;

        private static int FirstRegionOf(State state) => 0;

        private static int RegionsOf(State state) => 0;

        private void EnterRegion(int region)
        {
        }

        private void OnEntry(State state)
        {
            switch (state)
            {
                case State.Red:
                    redOn();
                    break;
                case State.Green:
                    greenOn();
                    break;
                case State.Yellow:
                    yellowOn();
                    break;
            }
        }

        private void OnExit(State state)
        {
            switch (state)
            {
                case State.Red:
                    redOff();
                    break;
                case State.Green:
                    greenOff();
                    break;
                case State.Yellow:
                    yellowOff();
                    break;
            }
        }

        private void ExitState(State state)
        {
            var first = FirstRegionOf(state);
            for (var region = first + RegionsOf(state) - 1; region >= first; region--)
            {
                if (active[region] != State.None)
                {
                    ExitState(active[region]);
                }
            }
            OnExit(state);
            active[RegionOf(state)] = State.None;
        }

        // Enters every state from below ‘from‘ down to ‘target‘, then follows default (initial
    ) transitions
        // below ‘target‘.
        private void EnterDown(State from, State target)
        {
            var state = target;
            var next = State.None;
            while (ParentOf(state) != from)
            {
                next = state;
                state = ParentOf(state);
            }
            EnterState(state);
            if (next == State.None)
            {
                EnterDefault(state);
                return;
            }
            EnterDown(state, target);
        }

        private void EnterState(State state)
        {
            active[RegionOf(state)] = state;
            OnEntry(state);
        }

        private void EnterDefault(State state)
        {
            var first = FirstRegionOf(state);
            for (var region = first; region < first + RegionsOf(state); region++)
            {
                EnterRegion(region);
            }
        }

        private State Innermost(State state)
        {
            while (state != State.None && RegionsOf(state) > 0 && active[FirstRegionOf(state)]
    != State.None)
            {
                state = active[FirstRegionOf(state)];
            }
            return state;
        }
    }
}
```

Appendix D

Generated Code: Door Lock, C# Table-Based

```
// <auto-generated>
//     DoorLock: table-based hierarchical state machine generated by MachinaState.
//     Do not edit by hand. Derive from this class and implement the guard and action hooks.
// </auto-generated>
#nullable enable
using System;

namespace Security
{
    public abstract class DoorLock
    {
        public enum State
        {
            Locked,
            Unlocked,
            Alarm,
            None,
        }

        public enum Event
        {
            EnterCode,
            Lock,
            Timeout,
            Reset,
        }

        public enum TraceKind
        {
            Start,      // name = machine name
            Event,      // name = event being dispatched
            Ignored,    // name = event no state handled
            Transition, // name = "Source -> Target"
            Exit,       // name = state being exited
            Enter,      // name = state being entered
        }

        public const int StateCount = 3;
        public const int RegionCount = 1; // region 0 is the top level

        private sealed class TransitionRow
        {
            public TransitionRow(State source, Event trigger, Func<bool>? guard, Action? action
    , State target, State lca, string name)
            {
                Source = source;
                Trigger = trigger;
                Guard = guard;
                Action = action;
                Target = target;
                Lca = lca;
                Name = name;
            }

        public State Source { get; }
        public Event Trigger { get; }
        public Func<bool>? Guard { get; }
        public Action? Action { get; }
        /// <summary>None for internal transitions.</summary>
        public State Target { get; }
        public State Lca { get; }
        public string Name { get; }
    }

    private static readonly State[] Parents = { State.None, State.None, State.None };
    // Region that contains each state, its first own region and its number of own regions.
    private static readonly int[] RegionOfState = { 0, 0, 0 };
    private static readonly int[] FirstRegions = { 0, 0, 0 };
    private static readonly int[] RegionCounts = { 0, 0, 0 };
    private static readonly bool[] Finals = { false, false, false };
    // Owner and initial target of each region.
    private static readonly State[] RegionOwners = { State.None };
    private static readonly State[] RegionTargets = { State.Locked };

    private readonly Action?[] regionActions;
    private readonly Action?[] entryActions;
    private readonly Action?[] exitActions;
    private readonly TransitionRow[] transitions;
    private readonly State[] active = { State.None }; // active state per region

    // Arguments of the event being dispatched.
    private int EnterCode_code = default!;
    private int Reset_code = default!;

    protected DoorLock()
    {
        regionActions = new Action?[] { null };
        entryActions = new Action?[] { engageBolt, releaseBolt, sirenOn };
        exitActions = new Action?[] { null, null, sirenOff };
        // source, event, guard, action, target, least common ancestor, trace name
        transitions = new TransitionRow[]
        {
            new TransitionRow(State.Locked, Event.EnterCode, () => isValidCode(EnterCode_code), () => { resetAttempts(EnterCode_code); }, State.Unlocked, State.None, "Locked -> Unlocked"),
            new TransitionRow(State.Locked, Event.EnterCode, () => tooManyAttempts(EnterCode_code), null, State.Alarm, State.None, "Locked -> Alarm"),
            new TransitionRow(State.Locked, Event.EnterCode, null, () => { countFailure(EnterCode_code); }, State.None, State.None, "Locked (internal)"),
            new TransitionRow(State.Unlocked, Event.Lock, null, null, State.Locked, State.None, "Unlocked -> Locked"),
            new TransitionRow(State.Unlocked, Event.Timeout, null, null, State.Locked, State.None, "Unlocked -> Locked"),
            new TransitionRow(State.Alarm, Event.Reset, () => isAdminCode(Reset_code), () => { resetAttempts(Reset_code); }, State.Locked, State.None, "Alarm -> Locked"),
        };
    }

    /// <summary>Innermost active state (following the first region of orthogonal states).</summary>
    public State Current
    {
        get
        {
            Lock();
            try
            {
                    return Innermost(active[0]);
                }
                finally
                {
                    Unlock();
                }
            }
        }

        /// <summary>True once a top-level final state has been reached.</summary>
        public bool IsFinished
        {
            get
            {
                Lock();
                try
                {
                    return false;
                }
                finally
                {
                    Unlock();
                }
            }
        }

        /// <summary>Resets the machine (running timers are cancelled, history is forgotten) and enters the initial state configuration.</summary>
        public void Start()
        {
            Lock();
            try
            {
                Array.Fill(active, State.None);
                Trace(TraceKind.Start, "DoorLock");
                EnterDown(State.None, State.Locked);
            }
            finally
            {
                Unlock();
            }
        }

        /// <summary>Dispatches an event to the innermost active states first. Returns false if the event was not handled.</summary>
        /// <remarks>Events with parameters must be sent with their typed Dispatch method; they are rejected here.</remarks>
        public bool Dispatch(Event trigger)
        {
            Lock();
            try
            {
                if (trigger == Event.EnterCode || trigger == Event.Reset)
                {
                    return false;
                }
                return DispatchCore(trigger);
            }
            finally
            {
                Unlock();
            }
        }

        public bool DispatchEnterCode(int code)
        {
            Lock();
            try
            {
                EnterCode_code = code;
                return DispatchCore(Event.EnterCode);
            }
            finally
            {
                Unlock();
            }
        }

        public bool DispatchReset(int code)
        {
            Lock();
            try
            {
                Reset_code = code;
                return DispatchCore(Event.Reset);
            }
            finally
            {
                Unlock();
            }
        }

        /// <summary>True if the given state is active.</summary>
        public bool IsIn(State state)
        {
            Lock();
            try
            {
                return state != State.None && active[RegionOf(state)] == state;
            }
            finally
            {
                Unlock();
            }
        }

        public static State ParentOf(State state) => state == State.None ? State.None : Parents[(int)state];

        /// <summary>Acquires mutual exclusion around every public member (e.g. Monitor.Enter on a private object).</summary>
        /// <remarks>Hooks must not call public members of this class unless the lock is re-entrant.</remarks>
        protected abstract void Lock();
        /// <summary>Releases the lock acquired by <see cref="Lock"/>.</summary>
        protected abstract void Unlock();
        /// <summary>Override to log state machine activity.</summary>
        protected virtual void Trace(TraceKind kind, string name)
        {
        }
        protected abstract bool isValidCode(int code);
        protected abstract bool tooManyAttempts(int code);
        protected abstract bool isAdminCode(int code);
        protected abstract void engageBolt();
        protected abstract void releaseBolt();
        protected abstract void sirenOn();
        protected abstract void sirenOff();
        protected abstract void resetAttempts(int code);
        protected abstract void countFailure(int code);

        private bool DispatchCore(Event trigger)
        {
            Trace(TraceKind.Event, trigger.ToString());
            if (active[0] != State.None && DispatchState(active[0], trigger))
            {
                return true;
            }
            Trace(TraceKind.Ignored, trigger.ToString());
            return false;
        }

        // Offers the event to every active region below ‘state‘ first, then to ‘state‘ itself.
        private bool DispatchState(State state, Event trigger)
        {
            var first = FirstRegionOf(state);
            for (var region = first; region < first + RegionsOf(state); region++)
            {
                if (active[region] != State.None && DispatchState(active[region], trigger))
                {
                    return true;
                }
            }
            return HandleEvent(state, trigger);
        }

        private static int RegionOf(State state) => RegionOfState[(int)state];
        private static int FirstRegionOf(State state) => FirstRegions[(int)state];
        private static int RegionsOf(State state) => RegionCounts[(int)state];
        private static bool IsFinalState(State state) => state != State.None && Finals[(int)state];

        private bool HandleEvent(State state, Event trigger)
        {
            foreach (var row in transitions)
            {
                if (row.Source == state && row.Trigger == trigger && TryRow(row))
                {
                    return true;
                }
            }
            return false;
        }

        private bool TryRow(TransitionRow row)
        {
            if (row.Guard != null && !row.Guard())
            {
                return false;
            }
            Trace(TraceKind.Transition, row.Name);
            if (row.Target == State.None)
            {
                row.Action?.Invoke();
                return true;
            }
            var root = row.Source;
            while (ParentOf(root) != row.Lca)
            {
                root = ParentOf(root);
            }
            ExitState(root);
            row.Action?.Invoke();
            EnterDown(row.Lca, row.Target);
            return true;
        }

        private void EnterRegion(int region)
        {
            regionActions[region]?.Invoke();
            EnterDown(RegionOwners[region], RegionTargets[region]);
        }

        private void ExitState(State state)
        {
            var first = FirstRegionOf(state);
            for (var region = first + RegionsOf(state) - 1; region >= first; region--)
            {
                if (active[region] != State.None)
                {
                    ExitState(active[region]);
                }
            }
            Trace(TraceKind.Exit, state.ToString());
            exitActions[(int)state]?.Invoke();
            active[RegionOf(state)] = State.None;
        }

        // Enters every state from below ‘from‘ down to ‘target‘, then follows default (initial) transitions
        // below ‘target‘.
        private void EnterDown(State from, State target)
        {
            var state = target;
            var next = State.None;
            while (ParentOf(state) != from)
            {
                next = state;
                state = ParentOf(state);
            }
            EnterState(state);
            if (next == State.None)
            {
                EnterDefault(state);
                return;
            }
            EnterDown(state, target);
        }

        private void EnterState(State state)
        {
            active[RegionOf(state)] = state;
            Trace(TraceKind.Enter, state.ToString());
            entryActions[(int)state]?.Invoke();
        }

        private void EnterDefault(State state)
        {
            var first = FirstRegionOf(state);
            for (var region = first; region < first + RegionsOf(state); region++)
            {
                EnterRegion(region);
            }
        }

        private State Innermost(State state)
        {
            while (state != State.None && RegionsOf(state) > 0 && active[FirstRegionOf(state)] != State.None)
            {
                state = active[FirstRegionOf(state)];
            }
            return state;
        }
    }
}
```

Appendix E

Generated Code: Vending Machine, C++ Switch-Based with Trace and Queue

```
// VendingMachine: switch-based hierarchical state machine generated by MachinaState.
// Do not edit by hand. Derive from this class and implement the guard and action hooks.
#pragma once

#include <cstddef>

namespace Demo {

    class VendingMachine {
      public:
        enum class State {
            Boot,
            Idle,
            Collecting,
            Ready,
            Dispensing,
            Dropping,   // in Dispensing
            Done,   // in Dispensing
            None
        };

        enum class Event {
            Coin,
            Select,
            Cancel,
            Collecting_After30000ms,   // after(30000 ms) in Collecting
            Dropping_After2000ms,   // after(2000 ms) in Dropping
            Completion   // internal: completion of a state
        };

        enum class TraceKind {
            Start,        // name = machine name
            Event,        // name = event being dispatched
            Ignored,      // name = event no state handled
            Transition,   // name = "Source -> Target"
            Exit,         // name = state being exited
            Enter         // name = state being entered
        };

        static constexpr std::size_t kStateCount  = 7;
        static constexpr std::size_t kEventCount  = 6;
        static constexpr std::size_t kRegionCount = 2;   // region 0 is the top level
        static constexpr std::size_t kTimerCount  = 2;
        static constexpr std::size_t kQueueCapacity = 8;

        virtual ~VendingMachine() = default;

        // Resets the machine (running timers are cancelled, history is forgotten) and enters the initial state configuration.
        void Start() {
            if (timerArmed_[0]) {
```

```cpp
            timerArmed_[0] = false;
            ++timerGeneration_[0];
            CancelTimer(Event::Collecting_After30000ms);
        }
        if (timerArmed_[1]) {
            timerArmed_[1] = false;
            ++timerGeneration_[1];
            CancelTimer(Event::Dropping_After2000ms);
        }
        for (State& state : active_) {
            state = State::None;
        }
        for (bool& flag : pending_) {
            flag = false;
        }
        queueHead_  = 0;
        queueCount_ = 0;
        Trace(TraceKind::Start, "VendingMachine");
        EnterDown(State::None, State::Boot);
        RunCompletions();
        DrainQueue();
    }

    // Dispatches an event to the innermost active states first. Returns false if the event was not handled.
    bool Dispatch(Event event) {
        switch (event) {
            case Event::Coin:
            case Event::Select:
            case Event::Completion:
                return false;   // use the typed Dispatch method / internal event
            default:
                break;
        }
        args_ = nullptr;
        const bool handled = DispatchCore(event);
        args_ = nullptr;
        DrainQueue();
        return handled;
    }

    bool DispatchCoin(int cents) {
        const CoinArgs args { cents };
        args_ = &args;
        const bool handled = DispatchCore(Event::Coin);
        args_ = nullptr;
        DrainQueue();
        return handled;
    }

    bool DispatchSelect(int item) {
        const SelectArgs args { item };
        args_ = &args;
        const bool handled = DispatchCore(Event::Select);
        args_ = nullptr;
        DrainQueue();
        return handled;
    }

    // Dispatches a timeout reported by a timer started with StartTimer; stale timeouts are ignored.
    bool DispatchTimeout(Event timeout, unsigned generation) {
        const int index = TimerIndex(timeout);
        if (index < 0 || !timerArmed_[index] || timerGeneration_[index] != generation) {
                return false;
            }
            args_ = nullptr;
            const bool handled = DispatchCore(timeout);
            args_ = nullptr;
            DrainQueue();
            return handled;
        }

        // Innermost active state (following the first region of orthogonal states).
        State Current() const {
            return Innermost(active_[0]);
        }

        // True if the given state is active.
        bool IsIn(State state) const {
            return state != State::None && active_[RegionOf(state)] == state;
        }

        // True once a top-level final state has been reached.
        bool IsFinished() const {
            return false;
        }

        static State ParentOf(State state) {
            switch (state) {
                case State::Dropping:
                case State::Done:
                    return State::Dispensing;
                default:
                    return State::None;
            }
        }

        static const char* StateName(State state) {
            switch (state) {
                case State::Boot:
                    return "Boot";
                case State::Idle:
                    return "Idle";
                case State::Collecting:
                    return "Collecting";
                case State::Ready:
                    return "Ready";
                case State::Dispensing:
                    return "Dispensing";
                case State::Dropping:
                    return "Dropping";
                case State::Done:
                    return "Done";
                default:
                    return "None";
            }
        }

        static const char* EventName(Event event) {
            switch (event) {
                case Event::Coin:
                    return "Coin";
                case Event::Select:
                    return "Select";
                case Event::Cancel:
                    return "Cancel";
                case Event::Collecting_After30000ms:
```

```
                return "Collecting_After30000ms";
            case Event::Dropping_After2000ms:
                return "Dropping_After2000ms";
            case Event::Completion:
                return "Completion";
            default:
                return "";
        }
    }

  protected:
    // Timers for after(...) transitions: when a timer elapses, call DispatchTimeout(
timeout, generation).
    virtual void StartTimer(Event timeout, unsigned milliseconds, unsigned generation) = 0;
    virtual void CancelTimer(Event timeout)                                            = 0;
    // Override to log state machine activity.
    virtual void Trace(TraceKind kind, const char* name) {
        (void)kind;
        (void)name;
    }
    // Queues an event from inside a hook; it is processed after the current event. Returns
 false if the queue is full.
    bool Post(Event event) {
        switch (event) {
            case Event::Coin:
            case Event::Select:
            case Event::Completion:
                return false;
            default:
                break;
        }
        QueuedEvent entry {};
        entry.event = event;
        return Enqueue(entry);
    }
    bool PostCoin(int cents) {
        QueuedEvent entry {};
        entry.event = Event::Coin;
        entry.Coin_cents = cents;
        return Enqueue(entry);
    }
    bool PostSelect(int item) {
        QueuedEvent entry {};
        entry.event = Event::Select;
        entry.Select_item = item;
        return Enqueue(entry);
    }
    // Guards
    virtual bool inStock(int item) = 0;
    virtual bool hasEnoughCredit(int cents) = 0;
    virtual bool hasChange() = 0;
    // Actions
    virtual void selfTest() = 0;
    virtual void showWelcome() = 0;
    virtual void showCredit() = 0;
    virtual void showSelect() = 0;
    virtual void openFlap() = 0;
    virtual void closeFlap() = 0;
    virtual void addCredit(int cents) = 0;
    virtual void refund() = 0;
    virtual void vend(int item) = 0;
    virtual void giveChange() = 0;

  private:
        // Arguments of the event being dispatched; only valid during Dispatch.
        struct CoinArgs {
            int cents;
        };
        const CoinArgs& ArgsOfCoin() const {
            return *static_cast<const CoinArgs*>(args_);
        }

        struct SelectArgs {
            int item;
        };
        const SelectArgs& ArgsOfSelect() const {
            return *static_cast<const SelectArgs*>(args_);
        }

        const void* args_ = nullptr;

        struct QueuedEvent {
            Event event;
            int Coin_cents;
            int Select_item;
        };

        bool Enqueue(const QueuedEvent& entry) {
            if (queueCount_ == kQueueCapacity) {
                return false;
            }
            queue_[(queueHead_ + queueCount_) % kQueueCapacity] = entry;
            ++queueCount_;
            return true;
        }

        void DrainQueue() {
            while (queueCount_ > 0) {
                const QueuedEvent entry = queue_[queueHead_];
                queueHead_              = (queueHead_ + 1) % kQueueCapacity;
                --queueCount_;
                switch (entry.event) {
                    case Event::Coin: {
                        const CoinArgs args { entry.Coin_cents };
                        args_ = &args;
                        DispatchCore(entry.event);
                        args_ = nullptr;
                        break;
                    }
                    case Event::Select: {
                        const SelectArgs args { entry.Select_item };
                        args_ = &args;
                        DispatchCore(entry.event);
                        args_ = nullptr;
                        break;
                    }
                    default:
                        DispatchCore(entry.event);
                        break;
                }
            }
        }

        bool DispatchCore(Event event) {
            const int timer = TimerIndex(event);
            if (timer >= 0) {
                timerArmed_[timer] = false;    // a timer fires once
            }
            Trace(TraceKind::Event, EventName(event));
            if (active_[0] != State::None && DispatchState(active_[0], event)) {
                RunCompletions();
                return true;
            }
            Trace(TraceKind::Ignored, EventName(event));
            return false;
        }

        // Offers the event to every active region below `state` first, then to `state` itself.
        bool DispatchState(State state, Event event) {
            const std::size_t first = FirstRegionOf(state);
            for (std::size_t region = first; region < first + RegionsOf(state); ++region) {
                if (active_[region] != State::None && DispatchState(active_[region], event)) {
                    return true;
                }
            }
            return HandleEvent(state, event);
        }

        // Event Coin in Idle: choice point.
        bool Fire_1() {
            if (hasEnoughCredit(ArgsOfCoin().cents)) {
                Trace(TraceKind::Transition, "Idle -> Ready (choice)");
                ExitState(State::Idle);
                addCredit(ArgsOfCoin().cents);
                EnterDown(State::None, State::Ready);
                return true;
            }
            Trace(TraceKind::Transition, "Idle -> Collecting (choice)");
            ExitState(State::Idle);
            addCredit(ArgsOfCoin().cents);
            EnterDown(State::None, State::Collecting);
            return true;
        }

        // Event Coin in Collecting: choice point.
        bool Fire_2() {
            if (hasEnoughCredit(ArgsOfCoin().cents)) {
                Trace(TraceKind::Transition, "Collecting -> Ready (choice)");
                ExitState(State::Collecting);
                addCredit(ArgsOfCoin().cents);
                EnterDown(State::None, State::Ready);
                return true;
            }
            Trace(TraceKind::Transition, "Collecting -> Collecting (choice)");
            ExitState(State::Collecting);
            addCredit(ArgsOfCoin().cents);
            EnterDown(State::None, State::Collecting);
            return true;
        }

        // Completion of Dispensing: choice point.
        bool Fire_7() {
            if (hasChange()) {
                Trace(TraceKind::Transition, "Dispensing -> Idle (choice)");
                ExitState(State::Dispensing);
                giveChange();
                EnterDown(State::None, State::Idle);
                return true;
            }
            Trace(TraceKind::Transition, "Dispensing -> Idle (choice)");
            ExitState(State::Dispensing);
            EnterDown(State::None, State::Idle);
            return true;
        }

        // Completion transitions fire once a state is entered, or a composite state has
    reached final states in all regions.
        void RunCompletions() {
            for (;;) {
                std::size_t index = 0;
                while (index < kStateCount && !pending_[index]) {
                    ++index;
                }
                if (index == kStateCount) {
                    return;
                }
                pending_[index]   = false;
                const State state = static_cast<State>(index);
                if (active_[RegionOf(state)] == state) {
                    HandleEvent(state, Event::Completion);
                }
            }
        }

        void MarkCompletion(State state) {
            switch (state) {
                case State::Boot:
                    pending_[0] = true;   // Boot
                    break;
                case State::Done:
                    pending_[4] = true;   // Dispensing
                    break;
                default:
                    break;
            }
        }

        void StartTimers(State state) {
            switch (state) {
                case State::Collecting:
                    ++timerGeneration_[0];
                    timerArmed_[0] = true;
                    StartTimer(Event::Collecting_After30000ms, 30000u, timerGeneration_[0]);
                    break;
                case State::Dropping:
                    ++timerGeneration_[1];
                    timerArmed_[1] = true;
                    StartTimer(Event::Dropping_After2000ms, 2000u, timerGeneration_[1]);
                    break;
                default:
                    break;
            }
        }

        void CancelTimers(State state) {
            switch (state) {
                case State::Collecting:
                    timerArmed_[0] = false;
                    CancelTimer(Event::Collecting_After30000ms);
                    break;
                case State::Dropping:
                    timerArmed_[1] = false;
                    CancelTimer(Event::Dropping_After2000ms);
                    break;
                default:
                    break;
        }
    }

    static int TimerIndex(Event event) {
        switch (event) {
            case Event::Collecting_After30000ms:
                return 0;
            case Event::Dropping_After2000ms:
                return 1;
            default:
                return -1;
        }
    }

    bool HandleEvent(State state, Event event) {
        switch (state) {
            case State::Boot:
                switch (event) {
                    case Event::Completion:
                        Trace(TraceKind::Transition, "Boot -> Idle");
                        ExitState(State::Boot);
                        EnterDown(State::None, State::Idle);
                        return true;
                    default:
                        break;
                }
                break;
            case State::Idle:
                switch (event) {
                    case Event::Coin:
                        return Fire_1();
                    default:
                        break;
                }
                break;
            case State::Collecting:
                switch (event) {
                    case Event::Coin:
                        return Fire_2();
                    case Event::Collecting_After30000ms:
                        Trace(TraceKind::Transition, "Collecting -> Idle");
                        ExitState(State::Collecting);
                        refund();
                        EnterDown(State::None, State::Idle);
                        return true;
                    default:
                        break;
                }
                break;
            case State::Ready:
                switch (event) {
                    case Event::Coin:
                        Trace(TraceKind::Transition, "Ready (internal)");
                        addCredit(ArgsOfCoin().cents);
                        return true;
                    case Event::Select:
                        if (inStock(ArgsOfSelect().item)) {
                            Trace(TraceKind::Transition, "Ready -> Dispensing");
                            ExitState(State::Ready);
                            vend(ArgsOfSelect().item);
                            EnterDown(State::None, State::Dispensing);
                            return true;
                        }
                        break;
                        case Event::Cancel:
                            Trace(TraceKind::Transition, "Ready -> Idle");
                            ExitState(State::Ready);
                            refund();
                            EnterDown(State::None, State::Idle);
                            return true;
                        default:
                            break;
                    }
                    break;
                case State::Dispensing:
                    switch (event) {
                        case Event::Completion:
                            return Fire_7();
                        default:
                            break;
                    }
                    break;
                case State::Dropping:
                    switch (event) {
                        case Event::Dropping_After2000ms:
                            Trace(TraceKind::Transition, "Dropping -> Done");
                            ExitState(State::Dropping);
                            EnterDown(State::Dispensing, State::Done);
                            return true;
                        default:
                            break;
                    }
                    break;
                default:
                    break;
            }
            return false;
        }

        static std::size_t RegionOf(State state) {
            switch (state) {
                case State::Dropping:
                case State::Done:
                    return 1;
                default:
                    return 0;
            }
        }

        static std::size_t FirstRegionOf(State state) {
            switch (state) {
                case State::Dispensing:
                    return 1;
                default:
                    return 0;
            }
        }

        static std::size_t RegionsOf(State state) {
            switch (state) {
                case State::Dispensing:
                    return 1;
                default:
                    return 0;
            }
        }

        void EnterRegion(std::size_t region) {
            switch (region) {
                case 1:
                    EnterDown(State::Dispensing, State::Dropping);
                    break;
                default:
                    break;
            }
        }

        void OnEntry(State state) {
            switch (state) {
                case State::Boot:
                    selfTest();
                    break;
                case State::Idle:
                    showWelcome();
                    break;
                case State::Collecting:
                    showCredit();
                    break;
                case State::Ready:
                    showSelect();
                    break;
                case State::Dispensing:
                    openFlap();
                    break;
                default:
                    break;
            }
        }

        void OnExit(State state) {
            switch (state) {
                case State::Dispensing:
                    closeFlap();
                    break;
                default:
                    break;
            }
        }

        void ExitState(State state) {
            const std::size_t first = FirstRegionOf(state);
            for (std::size_t region = first + RegionsOf(state); region-- > first;) {
                if (active_[region] != State::None) {
                    ExitState(active_[region]);
                }
            }
            Trace(TraceKind::Exit, StateName(state));
            CancelTimers(state);
            OnExit(state);
            active_[RegionOf(state)] = State::None;
        }

        // Enters every state from below ‘from‘ down to ‘target‘, then follows default (initial) transitions
        // below ‘target‘.
        void EnterDown(State from, State target) {
            State state = target;
            State next  = State::None;
            while (ParentOf(state) != from) {
                next  = state;
                state = ParentOf(state);
            }
            EnterState(state);
            if (next == State::None) {
                EnterDefault(state);
                return;
            }
            EnterDown(state, target);
        }

        void EnterState(State state) {
            active_[RegionOf(state)] = state;
            Trace(TraceKind::Enter, StateName(state));
            OnEntry(state);
            StartTimers(state);
        }

        void EnterDefault(State state) {
            const std::size_t first = FirstRegionOf(state);
            for (std::size_t region = first; region < first + RegionsOf(state); ++region) {
                EnterRegion(region);
            }
            if (RegionsOf(state) == 0) {
                MarkCompletion(state);
            }
        }

        State Innermost(State state) const {
            while (state != State::None && RegionsOf(state) > 0 && active_[FirstRegionOf(state)] != State::None) {
                state = active_[FirstRegionOf(state)];
            }
            return state;
        }

        State active_[kRegionCount] = { State::None, State::None };   // active state per region
        bool pending_[kStateCount] = {};   // completion events not yet processed
        unsigned timerGeneration_[kTimerCount] = {};
        bool timerArmed_[kTimerCount] = {};
        QueuedEvent queue_[kQueueCapacity] = {};
        std::size_t queueHead_  = 0;
        std::size_t queueCount_ = 0;
    };
}
```

Appendix F

Generated Code: Flat Traffic Light, C Header (Switch-Based with Trace)

```c
/* TrafficLight: hierarchical state machine generated by MachinaState (C99). */
/* Do not edit by hand. Fill in a hooks structure and pass it to Demo_TrafficLight_Init. */
#ifndef DEMO_TRAFFICLIGHT_H
#define DEMO_TRAFFICLIGHT_H

#include <stdbool.h>
#include <stddef.h>

#ifdef __cplusplus
extern "C" {
#endif

typedef enum {
    Demo_TrafficLight_State_Red,
    Demo_TrafficLight_State_Green,
    Demo_TrafficLight_State_Yellow,
    Demo_TrafficLight_State_None
} Demo_TrafficLight_State;

typedef enum {
    Demo_TrafficLight_Event_Timer
} Demo_TrafficLight_Event;

typedef enum {
    Demo_TrafficLight_Trace_Start,   /* name = machine name */
    Demo_TrafficLight_Trace_Event,   /* name = event being dispatched */
    Demo_TrafficLight_Trace_Ignored,   /* name = event no state handled */
    Demo_TrafficLight_Trace_Transition,   /* name = "Source -> Target" */
    Demo_TrafficLight_Trace_Exit,   /* name = state being exited */
    Demo_TrafficLight_Trace_Enter    /* name = state being entered */
} Demo_TrafficLight_TraceKind;

enum {
    Demo_TrafficLight_StateCount = 3,
    Demo_TrafficLight_EventCount = 1,
    Demo_TrafficLight_RegionCount = 1
};

/* Guards, actions and services supplied by the application. Every member must be set
   (except Trace, which may be NULL). ‘context‘ is the pointer passed to Init. */
typedef struct {
    /* Actions */
    void (*redOn)(void* context);
    void (*redOff)(void* context);
    void (*greenOn)(void* context);
    void (*greenOff)(void* context);
    void (*yellowOn)(void* context);
    void (*yellowOff)(void* context);
    /* Logs state machine activity (optional). */
    void (*Trace)(void* context, Demo_TrafficLight_TraceKind kind, const char* name);
} Demo_TrafficLight_Hooks;

/* Machine instance; treat all members as private. */
typedef struct Demo_TrafficLight {
    const Demo_TrafficLight_Hooks* hooks;
    void* context;
    Demo_TrafficLight_State active[Demo_TrafficLight_RegionCount];   /* active state per region
        ; region 0 is the top level */
} Demo_TrafficLight;

/* Binds the hooks; call once before Start. */
void Demo_TrafficLight_Init(Demo_TrafficLight* machine, const Demo_TrafficLight_Hooks* hooks,
    void* context);
/* Resets the machine (running timers are cancelled, history is forgotten) and enters the
    initial state configuration. */
void Demo_TrafficLight_Start(Demo_TrafficLight* machine);
/* Dispatches an event to the innermost active states first. Returns false if it was not
    handled. */
bool Demo_TrafficLight_Dispatch(Demo_TrafficLight* machine, Demo_TrafficLight_Event event);
/* Innermost active state (following the first region of orthogonal states). */
Demo_TrafficLight_State Demo_TrafficLight_Current(const Demo_TrafficLight* machine);
bool Demo_TrafficLight_IsIn(const Demo_TrafficLight* machine, Demo_TrafficLight_State state);
/* True once a top-level final state has been reached. */
bool Demo_TrafficLight_IsFinished(const Demo_TrafficLight* machine);
Demo_TrafficLight_State Demo_TrafficLight_ParentOf(Demo_TrafficLight_State state);
const char* Demo_TrafficLight_StateName(Demo_TrafficLight_State state);
const char* Demo_TrafficLight_EventName(Demo_TrafficLight_Event event);

#ifdef __cplusplus
}
#endif

#endif /* DEMO_TRAFFICLIGHT_H */
```